SciPy

Knowledge-Base Reference

Version 1.18.1

Optimization and curve fitting, probability distributions and hypothesis testing, integration, interpolation, linear algebra, sparse matrices and spatial structures, plus an API quick reference.

Source: official documentation, https://docs.scipy.org/doc/scipy/
Content © 2001-2026 SciPy Developers (BSD-3-Clause)
Compiled October 29, 2026

Contents

About this reference **3**

Optimization (scipy.optimize) **4**
- Local minimization of multivariate scalar functions (`minimize`) . . . 4
- Global optimization . . . 16
- Least-squares minimization (`least_squares`) . . . 18
- Univariate function minimizers (`minimize_scalar`) . . . 20
- Custom minimizers . . . 21
- Root finding . . . 23
- Linear programming (`linprog`) . . . 26
- Assignment problems . . . 29
- Mixed integer linear programming . . . 30
- Parallel execution support . . . 32

Statistics (scipy.stats): overview **35**

Probability distributions **36**
- Getting help . . . 36
- Common methods . . . 37
- Random number generation . . . 38
- Shifting and scaling . . . 38
- Shape parameters . . . 39
- Freezing a distribution . . . 40
- Broadcasting . . . 40
- Specific points for discrete distributions . . . 41
- Fitting distributions . . . 41
- Performance issues and cautionary remarks . . . 42
- Remaining issues . . . 42
- Building specific distributions . . . 42

Analysing one sample **46**
- Descriptive statistics . . . 46
- T-test and KS-test . . . 46
- Tails of the distribution . . . 47
- Special tests for normal distributions . . . 48

Kernel density estimation **50**
- Univariate estimation . . . 50
- Multivariate estimation . . . 51

Integration (scipy.integrate) **53**
- General integration (`quad`) . . . 53
- General multiple integration (`dblquad`, `tplquad`, `nquad`) . . . 55
- Gaussian quadrature . . . 57
- Integrating using Samples . . . 57
- Faster integration using low-level callback functions . . . 58
- Ordinary differential equations (`solve_ivp`) . . . 59

Interpolation (scipy.interpolate): overview **65**
- Smoothing and approximation of data . . . 65

1-D interpolation **67**
Piecewise linear interpolation . . . 67
Cubic splines . . . 67
Monotone interpolants . . . 68
Interpolation with B-splines . . . 68
Batches of `y` . . . 69
Parametric spline curves . . . 70
Missing data . . . 71
Out-of-bounds behavior . . . 71
Legacy interface for 1-D interpolation (`interp1d`) . . . 74

Linear algebra (scipy.linalg) **76**
numpy.matrix vs 2-D numpy.ndarray . . . 76
Basic routines . . . 77
Decompositions . . . 82
Matrix functions . . . 86
Special matrices . . . 88
Advanced Features . . . 89

Sparse arrays (scipy.sparse) **91**
Introduction . . . 91
Getting started with sparse arrays . . . 91
Understanding sparse array formats . . . 92
Sparse arrays, implicit zeros, and duplicates . . . 93
Canonical formats . . . 95
Next steps with sparse arrays . . . 95

Spatial data structures and algorithms (scipy.spatial) **96**
Convex hulls . . . 97
Voronoi diagrams . . . 98

API quick reference **102**
Statistics: describing and testing . . . 102
Optimization . . . 121
Integration and interpolation . . . 137
Linear algebra, sparse and spatial . . . 145

About this reference

This knowledge-base reference collects core chapters of the official **SciPy 1.18.1** documentation (https://docs.scipy.org/doc/scipy/).

- **Chapters 2 onward** are converted from the project's own documentation source files (the reStructuredText files the website is built from), taken from the `v1.18.1` release tag of github.com/scipy/scipy. Text and code are the project's; Sphinx-only elements such as cross-reference links, rendered plot output and interactive widgets were simplified for print.
- **The API quick reference** at the end is generated from the docstrings of the installed 1.18.1 package, the same text the official API pages are built from. Each entry is trimmed to signature, parameters, returns, notes and a short excerpt of the notes and examples.

Note on figures. Some SciPy tutorial sections draw plots by running their example code when the website is built. Those generated plots are not included; the text, formulas and code that produce them are.

Built October 29, 2026 for a personal knowledge base. Selection and assembly by Claude; no content was rewritten.

License. SciPy and its documentation are distributed under the BSD 3-Clause License, Copyright (c) 2001-2002 Enthought, Inc. 2003, SciPy Developers. Redistribution with this notice is permitted. Full text: https://github.com/scipy/scipy/blob/main/LICENSE.txt

Chapter sources

- Optimization (scipy.optimize): https://docs.scipy.org/doc/scipy/tutorial/optimize.html
- Statistics (scipy.stats): overview: https://docs.scipy.org/doc/scipy/tutorial/stats.html
- Probability distributions: https://docs.scipy.org/doc/scipy/tutorial/stats/probability_distributions.html
- Analysing one sample: https://docs.scipy.org/doc/scipy/tutorial/stats/analysing_one_sample.html
- Kernel density estimation: https://docs.scipy.org/doc/scipy/tutorial/stats/kernel_density_estimation.html
- Integration (scipy.integrate): https://docs.scipy.org/doc/scipy/tutorial/integrate.html
- Interpolation (scipy.interpolate): overview: https://docs.scipy.org/doc/scipy/tutorial/interpolate.html
- 1-D interpolation: https://docs.scipy.org/doc/scipy/tutorial/interpolate/1D.html
- Linear algebra (scipy.linalg): https://docs.scipy.org/doc/scipy/tutorial/linalg.html
- Sparse arrays (scipy.sparse): https://docs.scipy.org/doc/scipy/tutorial/sparse.html
- Spatial data structures and algorithms (scipy.spatial): https://docs.scipy.org/doc/scipy/tutorial/spatial.html

Optimization (scipy.optimize)

Source: https://docs.scipy.org/doc/scipy/tutorial/optimize.html

The `scipy.optimize` package provides several commonly used optimization algorithms. A detailed listing is available: `scipy.optimize` (can also be found by `help(scipy.optimize)`).

Local minimization of multivariate scalar functions (`minimize`)

The `minimize` function provides a common interface to unconstrained and constrained minimization algorithms for multivariate scalar functions in `scipy.optimize`. To demonstrate the minimization function, consider the problem of minimizing the Rosenbrock function of N variables:

$$f\left(\mathbf{x}\right) = \sum_{i=1}^{N-1} 100\left(x_{i+1} - x_i^2\right)^2 + \left(1 - x_i\right)^2.$$

The minimum value of this function is 0 which is achieved when $x_i = 1$.

Note that the Rosenbrock function and its derivatives are included in `scipy.optimize`. The implementations shown in the following sections provide examples of how to define an objective function as well as its jacobian and hessian functions. Objective functions in `scipy.optimize` expect a numpy array as their first parameter which is to be optimized and must return a float value. The exact calling signature must be `f(x, *args)` where `x` represents a numpy array and `args` a tuple of additional arguments supplied to the objective function.

Unconstrained minimization

Nelder-Mead Simplex algorithm (`method='Nelder-Mead'`)

In the example below, the `minimize` routine is used with the *Nelder-Mead* simplex algorithm (selected through the `method` parameter):

```
>>> import numpy as np
>>> from scipy.optimize import minimize
```

```
>>> def rosen(x):
...     """The Rosenbrock function"""
...     return sum(100.0*(x[1:]-x[:-1]**2.0)**2.0 + (1-x[:-1])**2.0)
```

```
>>> x0 = np.array([1.3, 0.7, 0.8, 1.9, 1.2])
>>> res = minimize(rosen, x0, method='nelder-mead',
...                options={'xatol': 1e-8, 'disp': True})
Optimization terminated successfully.
         Current function value: 0.000000
         Iterations: 339
         Function evaluations: 571
```

```
>>> print(res.x)
[1. 1. 1. 1. 1.]
```

The simplex algorithm is probably the simplest way to minimize a fairly well-behaved function. It requires only function evaluations and is a good choice for simple minimization problems. However, because it does not use any gradient evaluations, it may take longer to find the minimum.

Another optimization algorithm that needs only function calls to find the minimum is *Powell*'s method available by setting `method='powell'` in `minimize`.

To demonstrate how to supply additional arguments to an objective function, let us minimize the Rosenbrock function with an additional scaling factor `a` and an offset `b`:

$$f(\mathbf{x}, a, b) = \sum_{i=1}^{N-1} a\left(x_{i+1} - x_i^2\right)^2 + \left(1 - x_i\right)^2 + b.$$

Again using the `minimize` routine this can be solved by the following code block for the example parameters `a=0.5` and `b=1`.

```
>>> def rosen_with_args(x, a, b):
...     """The Rosenbrock function with additional arguments"""
...     return sum(a*(x[1:]-x[:-1]**2.0)**2.0 + (1-x[:-1])**2.0) + b
```

```
>>> x0 = np.array([1.3, 0.7, 0.8, 1.9, 1.2])
>>> res = minimize(rosen_with_args, x0, method='nelder-mead',
...                   args=(0.5, 1.), options={'xatol': 1e-8, 'disp': True})
Optimization terminated successfully.
         Current function value: 1.000000
         Iterations: 319 # may vary
         Function evaluations: 525 # may vary
```

```
>>> print(res.x)
[1.         1.         1.         1.         0.99999999]
```

As an alternative to using the `args` parameter of `minimize`, simply wrap the objective function in a new function that accepts only `x`. This approach is also useful when it is necessary to pass additional parameters to the objective function as keyword arguments.

```
>>> def rosen_with_args(x, a, *, b):  # b is a keyword-only argument
...     return sum(a*(x[1:]-x[:-1]**2.0)**2.0 + (1-x[:-1])**2.0) + b
>>> def wrapped_rosen_without_args(x):
...     return rosen_with_args(x, 0.5, b=1.)  # pass in ``a`` and ``b``
>>> x0 = np.array([1.3, 0.7, 0.8, 1.9, 1.2])
>>> res = minimize(wrapped_rosen_without_args, x0, method='nelder-mead',
...                options={'xatol': 1e-8,})
>>> print(res.x)
[1.         1.         1.         1.         0.99999999]
```

Another alternative is to use `functools.partial`.

```
>>> from functools import partial
>>> partial_rosen = partial(rosen_with_args, a=0.5, b=1.)
>>> res = minimize(partial_rosen, x0, method='nelder-mead',
...                options={'xatol': 1e-8,})
>>> print(res.x)
[1.         1.         1.         1.         0.99999999]
```

Broyden-Fletcher-Goldfarb-Shanno algorithm (`method='BFGS'`)

In order to converge more quickly to the solution, this routine uses the gradient of the objective function. If the gradient is not given by the user, then it is estimated using first-differences. The

Broyden-Fletcher-Goldfarb-Shanno (BFGS) method typically requires fewer function calls than the simplex algorithm even when the gradient must be estimated.

To demonstrate this algorithm, the Rosenbrock function is again used. The gradient of the Rosenbrock function is the vector:

$$\begin{aligned}\frac{\partial f}{\partial x_j} &= \sum_{i=1}^{N} 200\left(x_i - x_{i-1}^2\right)\left(\delta_{i,j} - 2x_{i-1}\delta_{i-1,j}\right) - 2\left(1 - x_{i-1}\right)\delta_{i-1,j}. \\ &= 200\left(x_j - x_{j-1}^2\right) - 400x_j\left(x_{j+1} - x_j^2\right) - 2\left(1 - x_j\right).\end{aligned}$$

This expression is valid for the interior derivatives. Special cases are

$$\begin{aligned}\frac{\partial f}{\partial x_0} &= -400x_0\left(x_1 - x_0^2\right) - 2\left(1 - x_0\right), \\ \frac{\partial f}{\partial x_{N-1}} &= 200\left(x_{N-1} - x_{N-2}^2\right).\end{aligned}$$

A Python function which computes this gradient is constructed by the code-segment:

```
>>> def rosen_der(x):
...     xm = x[1:-1]
...     xm_m1 = x[:-2]
...     xm_p1 = x[2:]
...     der = np.zeros_like(x)
...     der[1:-1] = 200*(xm-xm_m1**2) - 400*(xm_p1 - xm**2)*xm - 2*(1-xm)
...     der[0] = -400*x[0]*(x[1]-x[0]**2) - 2*(1-x[0])
...     der[-1] = 200*(x[-1]-x[-2]**2)
...     return der
```

This gradient information is specified in the `minimize` function through the `jac` parameter as illustrated below.

```
>>> res = minimize(rosen, x0, method='BFGS', jac=rosen_der,
...                options={'disp': True})
Optimization terminated successfully.
         Current function value: 0.000000
         Iterations: 25                    # may vary
         Function evaluations: 30
         Gradient evaluations: 30
>>> res.x
array([1., 1., 1., 1., 1.])
```

Avoiding Redundant Calculation

It is common for the objective function and its gradient to share parts of the calculation. For instance, consider the following problem.

```
>>> def f(x):
...     return -expensive(x[0])**2
>>>
>>> def df(x):
...     return -2 * expensive(x[0]) * dexpensive(x[0])
>>>
>>> def expensive(x):
...     # this function is computationally expensive!
...     expensive.count += 1  # let's keep track of how many times it runs
...     return np.sin(x)
```

```
>>> expensive.count = 0
>>>
>>> def dexpensive(x):
...     return np.cos(x)
>>>
>>> res = minimize(f, 0.5, jac=df)
>>> res.fun
-0.9999999999999174
>>> res.nfev, res.njev
6, 6
>>> expensive.count
12
```

Here, `expensive` is called 12 times: six times in the objective function and six times from the gradient. One way of reducing redundant calculations is to create a single function that returns both the objective function and the gradient.

```
>>> def f_and_df(x):
...     expensive_value = expensive(x[0])
...     return (-expensive_value**2,  # objective function
...             -2*expensive_value*dexpensive(x[0]))  # gradient
>>>
>>> expensive.count = 0  # reset the counter
>>> res = minimize(f_and_df, 0.5, jac=True)
>>> res.fun
-0.9999999999999174
>>> expensive.count
6
```

When we call minimize, we specify `jac==True` to indicate that the provided function returns both the objective function and its gradient. While convenient, not all `scipy.optimize` functions support this feature, and moreover, it is only for sharing calculations between the function and its gradient, whereas in some problems we will want to share calculations with the Hessian (second derivative of the objective function) and constraints. A more general approach is to memoize the expensive parts of the calculation. In simple situations, this can be accomplished with the `functools.lru_cache` wrapper.

```
>>> from functools import lru_cache
>>> expensive.count = 0  # reset the counter
>>> expensive = lru_cache(expensive)
>>> res = minimize(f, 0.5, jac=df)
>>> res.fun
-0.9999999999999174
>>> expensive.count
6
```

Newton-Conjugate-Gradient algorithm (`method='Newton-CG'`)

Newton-Conjugate Gradient algorithm is a modified Newton's method and uses a conjugate gradient algorithm to (approximately) invert the local Hessian [NW]. Newton's method is based on fitting the function locally to a quadratic form:

$$f\left(\mathbf{x}\right) \approx f\left(\mathbf{x}_0\right) + \nabla f\left(\mathbf{x}_0\right) \cdot \left(\mathbf{x} - \mathbf{x}_0\right) + \frac{1}{2}\left(\mathbf{x} - \mathbf{x}_0\right)^T \mathbf{H}\left(\mathbf{x}_0\right)\left(\mathbf{x} - \mathbf{x}_0\right).$$

where $\mathbf{H}\left(\mathbf{x}_0\right)$ is a matrix of second-derivatives (the Hessian). If the Hessian is positive definite then the local minimum of this function can be found by setting the gradient of the quadratic

form to zero, resulting in

$$\mathbf{x}_{\mathrm{opt}} = \mathbf{x}_0 - \mathbf{H}^{-1}\nabla f.$$

The inverse of the Hessian is evaluated using the conjugate-gradient method. An example of employing this method to minimizing the Rosenbrock function is given below. To take full advantage of the Newton-CG method, a function which computes the Hessian must be provided. The Hessian matrix itself does not need to be constructed, only a vector which is the product of the Hessian with an arbitrary vector needs to be available to the minimization routine. As a result, the user can provide either a function to compute the Hessian matrix, or a function to compute the product of the Hessian with an arbitrary vector.

Full Hessian example

The Hessian of the Rosenbrock function is

$$\begin{aligned} H_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j} &= 200\left(\delta_{i,j} - 2x_{i-1}\delta_{i-1,j}\right) - 400x_i\left(\delta_{i+1,j} - 2x_i\delta_{i,j}\right) - 400\delta_{i,j}\left(x_{i+1} - x_i^2\right) + 2\delta_{i,j}, \\ &= \left(202 + 1200x_i^2 - 400x_{i+1}\right)\delta_{i,j} - 400x_i\delta_{i+1,j} - 400x_{i-1}\delta_{i-1,j}, \end{aligned}$$

if $i, j \in [1, N-2]$ with $i, j \in [0, N-1]$ defining the $N \times N$ matrix. Other non-zero entries of the matrix are

$$\begin{aligned} \frac{\partial^2 f}{\partial x_0^2} &= 1200x_0^2 - 400x_1 + 2, \\ \frac{\partial^2 f}{\partial x_0 \partial x_1} = \frac{\partial^2 f}{\partial x_1 \partial x_0} &= -400x_0, \\ \frac{\partial^2 f}{\partial x_{N-1} \partial x_{N-2}} = \frac{\partial^2 f}{\partial x_{N-2} \partial x_{N-1}} &= -400x_{N-2}, \\ \frac{\partial^2 f}{\partial x_{N-1}^2} &= 200. \end{aligned}$$

For example, the Hessian when $N = 5$ is

$$\mathbf{H} = \begin{bmatrix} 1200x_0^2 + 2 & & & & \\ & 1200x_1^2 + 202 & & & \\ & & 1200x_1^2 + 202 & & \\ & & & 1200x_3^2 + 202 & \\ & & & & 200 \end{bmatrix} - 400 \begin{bmatrix} x_1 & x_0 & & & \\ x_0 & x_2 & x_1 & & \\ & x_1 & x_3 & x_2 & \\ & & x_2 & x_4 & x_3 \\ & & & x_3 & 0 \end{bmatrix}.$$

The code which computes this Hessian along with the code to minimize the function using Newton-CG method is shown in the following example:

```
>>> def rosen_hess(x):
...     x = np.asarray(x)
...     H = np.diag(-400*x[:-1],1) - np.diag(400*x[:-1],-1)
...     diagonal = np.zeros_like(x)
...     diagonal[0] = 1200*x[0]**2-400*x[1]+2
...     diagonal[-1] = 200
```

```
...     diagonal[1:-1] = 202 + 1200*x[1:-1]**2 - 400*x[2:]
...     H = H + np.diag(diagonal)
...     return H
```

```
>>> res = minimize(rosen, x0, method='Newton-CG',
...                jac=rosen_der, hess=rosen_hess,
...                options={'xtol': 1e-8, 'disp': True})
Optimization terminated successfully.
         Current function value: 0.000000
         Iterations: 19                       # may vary
         Function evaluations: 22
         Gradient evaluations: 19
         Hessian evaluations: 19
>>> res.x
array([1.,  1.,  1.,  1.,  1.])
```

Hessian product example

For larger minimization problems, storing the entire Hessian matrix can consume considerable time and memory. The Newton-CG algorithm only needs the product of the Hessian times an arbitrary vector. As a result, the user can supply code to compute this product rather than the full Hessian by giving a `hess` function which take the minimization vector as the first argument and the arbitrary vector as the second argument (along with extra arguments passed to the function to be minimized). If possible, using Newton-CG with the Hessian product option is probably the fastest way to minimize the function.

In this case, the product of the Rosenbrock Hessian with an arbitrary vector is not difficult to compute. If $\mathbf{p}$ is the arbitrary vector, then $\mathbf{H}\left(\mathbf{x}\right)\mathbf{p}$ has elements:

$$\mathbf{H}\left(\mathbf{x}\right)\mathbf{p} = \begin{bmatrix} \left(1200x_0^2 - 400x_1 + 2\right)p_0 - 400x_0p_1 \\ \vdots \\ -400x_{i-1}p_{i-1} + \left(202 + 1200x_i^2 - 400x_{i+1}\right)p_i - 400x_ip_{i+1} \\ \vdots \\ -400x_{N-2}p_{N-2} + 200p_{N-1} \end{bmatrix}.$$

Code which makes use of this Hessian product to minimize the Rosenbrock function using `minimize` follows:

```
>>> def rosen_hess_p(x, p):
...     x = np.asarray(x)
...     Hp = np.zeros_like(x)
...     Hp[0] = (1200*x[0]**2 - 400*x[1] + 2)*p[0] - 400*x[0]*p[1]
...     Hp[1:-1] = -400*x[:-2]*p[:-2]+(202+1200*x[1:-1]**2-400*x[2:])*p[1:-1] \
...                -400*x[1:-1]*p[2:]
...     Hp[-1] = -400*x[-2]*p[-2] + 200*p[-1]
...     return Hp
```

```
>>> res = minimize(rosen, x0, method='Newton-CG',
...                jac=rosen_der, hessp=rosen_hess_p,
...                options={'xtol': 1e-8, 'disp': True})
Optimization terminated successfully.
         Current function value: 0.000000
         Iterations: 20                    # may vary
         Function evaluations: 23
         Gradient evaluations: 20
         Hessian evaluations: 44
>>> res.x
array([1., 1., 1., 1., 1.])
```

According to [NW] p. 170 the `Newton-CG` algorithm can be inefficient when the Hessian is ill-conditioned because of the poor quality search directions provided by the method in those situations. The method `trust-ncg`, according to the authors, deals more effectively with this problematic situation and will be described next.

Trust-Region Newton-Conjugate-Gradient Algorithm (`method='trust-ncg'`)

The `Newton-CG` method is a line search method: it finds a direction of search minimizing a quadratic approximation of the function and then uses a line search algorithm to find the (nearly) optimal step size in that direction. An alternative approach is to, first, fix the step size limit Δ and then find the optimal step $\mathbf{p}$ inside the given trust-radius by solving the following quadratic subproblem:

$$\min_{\mathbf{p}} f(\mathbf{x}_k) + \nabla f(\mathbf{x}_k) \cdot \mathbf{p} + \frac{1}{2}\mathbf{p}^T \mathbf{H}(\mathbf{x}_k)\mathbf{p}; \quad \text{subject to: } \|\mathbf{p}\| \leq \Delta.$$

The solution is then updated $\mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{p}$ and the trust-radius Δ is adjusted according to the degree of agreement of the quadratic model with the real function. This family of methods is known as trust-region methods. The `trust-ncg` algorithm is a trust-region method that uses a conjugate gradient algorithm to solve the trust-region subproblem [NW].

Full Hessian example

```
>>> res = minimize(rosen, x0, method='trust-ncg',
...                jac=rosen_der, hess=rosen_hess,
...                options={'gtol': 1e-8, 'disp': True})
Optimization terminated successfully.
         Current function value: 0.000000
         Iterations: 20                    # may vary
         Function evaluations: 21
         Gradient evaluations: 20
         Hessian evaluations: 19
>>> res.x
array([1., 1., 1., 1., 1.])
```

Hessian product example

```
>>> res = minimize(rosen, x0, method='trust-ncg',
...                jac=rosen_der, hessp=rosen_hess_p,
...                options={'gtol': 1e-8, 'disp': True})
Optimization terminated successfully.
         Current function value: 0.000000
         Iterations: 20                    # may vary
         Function evaluations: 21
         Gradient evaluations: 20
         Hessian evaluations: 0
>>> res.x
array([1., 1., 1., 1., 1.])
```

Trust-Region Truncated Generalized Lanczos / Conjugate Gradient Algorithm (`method='trust-krylov'`)

Similar to the `trust-ncg` method, the `trust-krylov` method is a method suitable for large-scale problems as it uses the hessian only as linear operator by means of matrix-vector products. It solves the quadratic subproblem more accurately than the `trust-ncg` method.

$$\min_{\mathbf{p}} f(\mathbf{x}_k) + \nabla f(\mathbf{x}_k) \cdot \mathbf{p} + \frac{1}{2}\mathbf{p}^T \mathbf{H}(\mathbf{x}_k)\mathbf{p};$$
$$\text{subject to: } \|\mathbf{p}\| \leq \Delta.$$

This method wraps the [TRLIB] implementation of the [GLTR] method solving exactly a trust-region subproblem restricted to a truncated Krylov subspace. For indefinite problems it is usually better to use this method as it reduces the number of nonlinear iterations at the expense of few more matrix-vector products per subproblem solve in comparison to the `trust-ncg` method.

Full Hessian example

```
>>> res = minimize(rosen, x0, method='trust-krylov',
...                jac=rosen_der, hess=rosen_hess,
...                options={'gtol': 1e-8, 'disp': True})
Optimization terminated successfully.
         Current function value: 0.000000
         Iterations: 19                    # may vary
         Function evaluations: 20
         Gradient evaluations: 20
         Hessian evaluations: 18
>>> res.x
array([1., 1., 1., 1., 1.])
```

Hessian product example

```
>>> res = minimize(rosen, x0, method='trust-krylov',
...                jac=rosen_der, hessp=rosen_hess_p,
...                options={'gtol': 1e-8, 'disp': True})
Optimization terminated successfully.
         Current function value: 0.000000
         Iterations: 19                    # may vary
         Function evaluations: 20
         Gradient evaluations: 20
         Hessian evaluations: 0
>>> res.x
array([1., 1., 1., 1., 1.])
```

Trust-Region Nearly Exact Algorithm (`method='trust-exact'`)

All methods `Newton-CG`, `trust-ncg` and `trust-krylov` are suitable for dealing with large-scale problems (problems with thousands of variables). That is because the conjugate gradient algorithm approximately solve the trust-region subproblem (or invert the Hessian) by iterations without the explicit Hessian factorization. Since only the product of the Hessian with an arbitrary vector is needed, the algorithm is specially suited for dealing with sparse Hessians, allowing low storage requirements and significant time savings for those sparse problems.

For medium-size problems, for which the storage and factorization cost of the Hessian are not critical, it is possible to obtain a solution within fewer iteration by solving the trust-region subproblems almost exactly. To achieve that, a certain nonlinear equations is solved iteratively for each quadratic subproblem [CGT]. This solution requires usually 3 or 4 Cholesky factorizations of the Hessian matrix. As the result, the method converges in fewer number of iterations and takes fewer evaluations of the objective function than the other implemented trust-region methods. The Hessian product option is not supported by this algorithm. An example using the Rosenbrock function follows:

```
>>> res = minimize(rosen, x0, method='trust-exact',
...                jac=rosen_der, hess=rosen_hess,
...                options={'gtol': 1e-8, 'disp': True})
```

```
Optimization terminated successfully.
         Current function value: 0.000000
         Iterations: 13                    # may vary
         Function evaluations: 14
         Gradient evaluations: 13
         Hessian evaluations: 14
>>> res.x
array([1., 1., 1., 1., 1.])
```

Constrained minimization

The `minimize` function provides several algorithms for constrained minimization, namely `'trust-constr'` , `'SLSQP'`, `'COBYLA'`, and `'COBYQA'`. They require the constraints to be defined using slightly different structures. The methods `'trust-constr'`, `'COBYQA'`, and `'COBYLA'` require the constraints to be defined as a sequence of objects `LinearConstraint` and `NonlinearConstraint`. Method `'SLSQP'`, on the other hand, requires constraints to be defined as a sequence of dictionaries, with keys `type`, `fun` and `jac`.

As an example let us consider the constrained minimization of the Rosenbrock function:

$$\begin{aligned}\min_{x_0,x_1} \ & 100\left(x_1 - x_0^2\right)^2 + \left(1 - x_0\right)^2 \\ \text{subject to: } & x_0 + 2x_1 \leq 1 \\ & x_0^2 + x_1 \leq 1 \\ & x_0^2 - x_1 \leq 1 \\ & 2x_0 + x_1 = 1 \\ & 0 \leq x_0 \leq 1 \\ & -0.5 \leq x_1 \leq 2.0.\end{aligned}$$

This optimization problem has the unique solution $[x_0, x_1] = [0.4149,\ 0.1701]$, for which only the first and fourth constraints are active.

Trust-Region Constrained Algorithm (`method='trust-constr'`)

The trust-region constrained method deals with constrained minimization problems of the form:

$$\begin{aligned}\min_x & f(x) \\ \text{subject to: } & c^l \leq c(x) \leq c^u, \\ & x^l \leq x \leq x^u.\end{aligned}$$

When $c_j^l = c_j^u$ the method reads the j-th constraint as an equality constraint and deals with it accordingly. Besides that, one-sided constraint can be specified by setting the upper or lower bound to `np.inf` with the appropriate sign.

The implementation is based on [EQSQP] for equality-constraint problems and on [TRIP] for problems with inequality constraints. Both are trust-region type algorithms suitable for large-scale problems.

Defining Bounds Constraints

The bound constraints $0 \leq x_0 \leq 1$ and $-0.5 \leq x_1 \leq 2.0$ are defined using a `Bounds` object.

```
>>> from scipy.optimize import Bounds
>>> bounds = Bounds([0, -0.5], [1.0, 2.0])
```

Defining Linear Constraints

The constraints $x_0 + 2x_1 \leq 1$ and $2x_0 + x_1 = 1$ can be written in the linear constraint standard format:

$$\begin{bmatrix} -\infty \\ 1 \end{bmatrix} \leq \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \end{bmatrix} \leq \begin{bmatrix} 1 \\ 1 \end{bmatrix},$$

and defined using a `LinearConstraint` object.

```
>>> from scipy.optimize import LinearConstraint
>>> linear_constraint = LinearConstraint([[1, 2], [2, 1]], [-np.inf, 1], [1, 1])
```

Defining Nonlinear Constraints

The nonlinear constraint:

$$c(x) = \begin{bmatrix} x_0^2 + x_1 \\ x_0^2 - x_1 \end{bmatrix} \leq \begin{bmatrix} 1 \\ 1 \end{bmatrix},$$

with Jacobian matrix:

$$J(x) = \begin{bmatrix} 2x_0 & 1 \\ 2x_0 & -1 \end{bmatrix},$$

and linear combination of the Hessians:

$$H(x, v) = \sum_{i=0}^{1} v_i \nabla^2 c_i(x) = v_0 \begin{bmatrix} 2 & 0 \\ 0 & 0 \end{bmatrix} + v_1 \begin{bmatrix} 2 & 0 \\ 0 & 0 \end{bmatrix},$$

is defined using a `NonlinearConstraint` object.

```
>>> def cons_f(x):
...     return [x[0]**2 + x[1], x[0]**2 - x[1]]
>>> def cons_J(x):
...     return [[2*x[0], 1], [2*x[0], -1]]
>>> def cons_H(x, v):
...     return v[0]*np.array([[2, 0], [0, 0]]) + v[1]*np.array([[2, 0], [0, 0]])
>>> from scipy.optimize import NonlinearConstraint
>>> nonlinear_constraint = NonlinearConstraint(cons_f, -np.inf, 1, jac=cons_J, hess=cons_H)
```

Alternatively, it is also possible to define the Hessian $H(x, v)$ as a sparse matrix,

```
>>> from scipy.sparse import csc_array
>>> def cons_H_sparse(x, v):
...     return v[0]*csc_array([[2, 0], [0, 0]]) + v[1]*csc_array([[2, 0], [0, 0]])
>>> nonlinear_constraint = NonlinearConstraint(cons_f, -np.inf, 1,
...                                            jac=cons_J, hess=cons_H_sparse)
```

or as a `LinearOperator` object.

```
>>> from scipy.sparse.linalg import LinearOperator
>>> def cons_H_linear_operator(x, v):
...     def matvec(p):
...         return np.array([p[0]*2*(v[0]+v[1]), 0])
...     return LinearOperator((2, 2), matvec=matvec)
>>> nonlinear_constraint = NonlinearConstraint(cons_f, -np.inf, 1,
...                                            jac=cons_J, hess=cons_H_linear_operator)
```

When the evaluation of the Hessian $H(x, v)$ is difficult to implement or computationally infeasible, one may use `HessianUpdateStrategy`. Currently available strategies are `BFGS` and `SR1`.

```
>>> from scipy.optimize import BFGS
>>> nonlinear_constraint = NonlinearConstraint(cons_f, -np.inf, 1, jac=cons_J, hess=BFGS())
```

Alternatively, the Hessian may be approximated using finite differences.

```
>>> nonlinear_constraint = NonlinearConstraint(cons_f, -np.inf, 1, jac=cons_J, hess='2-point')
```

The Jacobian of the constraints can be approximated by finite differences as well. In this case, however, the Hessian cannot be computed with finite differences and needs to be provided by the user or defined using `HessianUpdateStrategy`.

```
>>> nonlinear_constraint = NonlinearConstraint(cons_f, -np.inf, 1, jac='2-point', hess=BFGS())
```

Solving the Optimization Problem The optimization problem is solved using:

```
>>> x0 = np.array([0.5, 0])
>>> res = minimize(rosen, x0, method='trust-constr', jac=rosen_der, hess=rosen_hess,
...                constraints=[linear_constraint, nonlinear_constraint],
...                options={'verbose': 1}, bounds=bounds)
# may vary
``gtol`` termination condition is satisfied.
Number of iterations: 12, function evaluations: 8, CG iterations: 7, optimality: 2.99e-09, constraint
 ↪  violation: 1.11e-16, execution time: 0.016 s.
>>> print(res.x)
[0.41494531 0.17010937]
```

When needed, the objective function Hessian can be defined using a `LinearOperator` object,

```
>>> def rosen_hess_linop(x):
...     def matvec(p):
...         return rosen_hess_p(x, p)
...     return LinearOperator((2, 2), matvec=matvec)
>>> res = minimize(rosen, x0, method='trust-constr', jac=rosen_der, hess=rosen_hess_linop,
...                constraints=[linear_constraint, nonlinear_constraint],
...                options={'verbose': 1}, bounds=bounds)
# may vary
``gtol`` termination condition is satisfied.
Number of iterations: 12, function evaluations: 8, CG iterations: 7, optimality: 2.99e-09, constraint
 ↪  violation: 1.11e-16, execution time: 0.018 s.
>>> print(res.x)
[0.41494531 0.17010937]
```

or a Hessian-vector product through the parameter `hessp`.

```
>>> res = minimize(rosen, x0, method='trust-constr', jac=rosen_der, hessp=rosen_hess_p,
...                constraints=[linear_constraint, nonlinear_constraint],
...                options={'verbose': 1}, bounds=bounds)
# may vary
``gtol`` termination condition is satisfied.
Number of iterations: 12, function evaluations: 8, CG iterations: 7, optimality: 2.99e-09, constraint
 ↪  violation: 1.11e-16, execution time: 0.018 s.
```

```
>>> print(res.x)
[0.41494531 0.17010937]
```

Alternatively, the first and second derivatives of the objective function can be approximated. For instance, the Hessian can be approximated with `SR1` quasi-Newton approximation and the gradient with finite differences.

```
>>> from scipy.optimize import SR1
>>> res = minimize(rosen, x0, method='trust-constr',  jac="2-point", hess=SR1(),
...                constraints=[linear_constraint, nonlinear_constraint],
...                options={'verbose': 1}, bounds=bounds)
# may vary
``gtol`` termination condition is satisfied.
Number of iterations: 12, function evaluations: 24, CG iterations: 7, optimality: 4.48e-09, constraint
↪  violation: 0.00e+00, execution time: 0.016 s.
>>> print(res.x)
[0.41494531 0.17010937]
```

Sequential Least SQuares Programming (SLSQP) Algorithm (`method='SLSQP'`)

The SLSQP method deals with constrained minimization problems of the form:

$$\begin{aligned} \min_x & f(x) \\ \text{subject to: } & c_j(x) = 0, && j \in \mathcal{E} \\ & c_j(x) \geq 0, && j \in \mathcal{J} \\ & \text{lb}_i \leq x_i \leq \text{ub}_i, && i = 1, ..., N. \end{aligned}$$

Where $\mathcal{E}$ or $\mathcal{J}$ are sets of indices containing equality and inequality constraints.

Both linear and nonlinear constraints are defined as dictionaries with keys `type`, `fun` and `jac`.

```
>>> ineq_cons = {'type': 'ineq',
...              'fun' : lambda x: np.array([1 - x[0] - 2*x[1],
...                                          1 - x[0]**2 - x[1],
...                                          1 - x[0]**2 + x[1]]),
...              'jac' : lambda x: np.array([[-1.0, -2.0],
...                                          [-2*x[0], -1.0],
...                                          [-2*x[0], 1.0]])}
>>> eq_cons = {'type': 'eq',
...            'fun' : lambda x: np.array([2*x[0] + x[1] - 1]),
...            'jac' : lambda x: np.array([2.0, 1.0])}
```

And the optimization problem is solved with:

```
>>> x0 = np.array([0.5, 0])
>>> res = minimize(rosen, x0, method='SLSQP', jac=rosen_der,
...                constraints=[eq_cons, ineq_cons], options={'ftol': 1e-9, 'disp': True},
...                bounds=bounds)
# may vary
Optimization terminated successfully.    (Exit mode 0)
            Current function value: 0.342717574857755
            Iterations: 5
            Function evaluations: 6
            Gradient evaluations: 5
>>> print(res.x)
[0.41494475 0.1701105 ]
```

Most of the options available for the method `'trust-constr'` are not available for `'SLSQP'`.

Local minimization solver comparison

Find a solver that meets your requirements using the table below. If there are multiple candidates, try several and see which ones best meet your needs (e.g. execution time, objective function value).

Solver	Bounds Constraints	Nonlinear Constraints	Uses Gradient	Uses Hessian	Utilizes Sparsity
CG			✓		
BFGS			✓		
dogleg			✓	✓	
trust-ncg			✓	✓	
trust-krylov			✓	✓	
trust-exact			✓	✓	
Newton-CG			✓	✓	✓
Nelder-Mead	✓				
Powell	✓				
L-BFGS-B	✓		✓		
TNC	✓		✓		
COBYLA	✓	✓			
COBYQA	✓	✓			
SLSQP	✓	✓	✓		
trust-constr	✓	✓	✓	✓	✓

Global optimization

Global optimization aims to find the global minimum of a function within given bounds, in the presence of potentially many local minima. Typically, global minimizers efficiently search the parameter space, while using a local minimizer (e.g., `minimize`) under the hood. SciPy contains a number of good global optimizers. Here, we'll use those on the same objective function, namely the (aptly named) `eggholder` function:

```
>>> def eggholder(x):
...     return (-(x[1] + 47) * np.sin(np.sqrt(abs(x[0]/2 + (x[1]  + 47))))
...             -x[0] * np.sin(np.sqrt(abs(x[0] - (x[1]  + 47)))))

>>> bounds = [(-512, 512), (-512, 512)]
```

This function looks like an egg carton:

```
>>> import matplotlib.pyplot as plt
>>> from mpl_toolkits.mplot3d import Axes3D

>>> x = np.arange(-512, 513)
>>> y = np.arange(-512, 513)
>>> xgrid, ygrid = np.meshgrid(x, y)
>>> xy = np.stack([xgrid, ygrid])
```

```
>>> fig = plt.figure()
>>> ax = fig.add_subplot(111, projection='3d')
>>> ax.view_init(45, -45)
>>> ax.plot_surface(xgrid, ygrid, eggholder(xy), cmap='terrain')
>>> ax.set_xlabel('x')
>>> ax.set_ylabel('y')
>>> ax.set_zlabel('eggholder(x, y)')
>>> plt.show()
```

We now use the global optimizers to obtain the minimum and the function value at the minimum. We'll store the results in a dictionary so we can compare different optimization results later.

```
>>> from scipy import optimize
>>> results = dict()
>>> results['shgo'] = optimize.shgo(eggholder, bounds)
>>> results['shgo']
     fun: -935.3379515604197  # may vary
    funl: array([-935.33795156])
 message: 'Optimization terminated successfully.'
    nfev: 42
     nit: 2
   nlfev: 37
   nlhev: 0
   nljev: 9
 success: True
       x: array([439.48096952, 453.97740589])
      xl: array([[439.48096952, 453.97740589]])
```

```
>>> results['DA'] = optimize.dual_annealing(eggholder, bounds)
>>> results['DA']
     fun: -956.9182316237413  # may vary
 message: ['Maximum number of iteration reached']
    nfev: 4091
    nhev: 0
     nit: 1000
    njev: 0
       x: array([482.35324114, 432.87892901])
```

All optimizers return an `OptimizeResult`, which in addition to the solution contains information on the number of function evaluations, whether the optimization was successful, and more. For brevity, we won't show the full output of the other optimizers:

```
>>> results['DE'] = optimize.differential_evolution(eggholder, bounds)
```

`shgo` has a second method, which returns all local minima rather than only what it thinks is the global minimum:

```
>>> results['shgo_sobol'] = optimize.shgo(eggholder, bounds, n=200, iters=5,
...                                       sampling_method='sobol')
```

We'll now plot all found minima on a heatmap of the function:

```
>>> fig = plt.figure()
>>> ax = fig.add_subplot(111)
>>> im = ax.imshow(eggholder(xy), interpolation='bilinear', origin='lower',
...                cmap='gray')
>>> ax.set_xlabel('x')
>>> ax.set_ylabel('y')
>>>
>>> def plot_point(res, marker='o', color=None):
...     ax.plot(512+res.x[0], 512+res.x[1], marker=marker, color=color, ms=10)
```

```
>>> plot_point(results['DE'], color='c')  # differential_evolution - cyan
>>> plot_point(results['DA'], color='w')  # dual_annealing.        - white

>>> # SHGO produces multiple minima, plot them all (with a smaller marker size)
>>> plot_point(results['shgo'], color='r', marker='+')
>>> plot_point(results['shgo_sobol'], color='r', marker='x')
>>> for i in range(results['shgo_sobol'].xl.shape[0]):
...     ax.plot(512 + results['shgo_sobol'].xl[i, 0],
...             512 + results['shgo_sobol'].xl[i, 1],
...             'ro', ms=2)

>>> ax.set_xlim([-4, 514*2])
>>> ax.set_ylim([-4, 514*2])
>>> plt.show()
```

Comparison of Global Optimizers

Find a solver that meets your requirements using the table below. If there are multiple candidates, try several and see which ones best meet your needs (e.g. execution time, objective function value).

Solver	Bounds Constraints	Nonlinear Constraints	Uses Gradient	Uses Hessian
basinhopping			(✓)	(✓)
direct	✓			
dual_annealing	✓		(✓)	(✓)
differential_evolution	✓	✓		
shgo	✓	✓	(✓)	(✓)

(✓) = Depending on the chosen local minimizer

Least-squares minimization (`least_squares`)

SciPy is capable of solving robustified bound-constrained nonlinear least-squares problems:

$$\min_{\mathbf{x}} \frac{1}{2} \sum_{i=1}^{m} \rho\left(f_i(\mathbf{x})^2\right)$$
$$\text{subject to } \mathbf{lb} \leq \mathbf{x} \leq \mathbf{ub}$$

Here $f_i(\mathbf{x})$ are smooth functions from $\mathbb{R}^n$ to $\mathbb{R}$, we refer to them as residuals. The purpose of a scalar-valued function $\rho(\cdot)$ is to reduce the influence of outlier residuals and contribute to robustness of the solution, we refer to it as a loss function. A linear loss function gives a standard least-squares problem. Additionally, constraints in a form of lower and upper bounds on some of x_j are allowed.

All methods specific to least-squares minimization utilize a $m \times n$ matrix of partial derivatives called Jacobian and defined as $J_{ij} = \partial f_i / \partial x_j$. It is highly recommended to compute this matrix analytically and pass it to `least_squares`, otherwise, it will be estimated by finite differences, which takes a lot of additional time and can be very inaccurate in hard cases.

Function `least_squares` can be used for fitting a function $\varphi(t; \mathbf{x})$ to empirical data $\{(t_i, y_i), i = 0, \ldots, m-1\}$. To do this, one should simply precompute residuals as $f_i(\mathbf{x}) = w_i(\varphi(t_i; \mathbf{x}) - y_i)$, where w_i are weights assigned to each observation.

Example of solving a fitting problem

Here we consider an enzymatic reaction[1]. There are 11 residuals defined as

$$f_i(x) = \frac{x_0(u_i^2 + u_i x_1)}{u_i^2 + u_i x_2 + x_3} - y_i, \quad i = 0, \ldots, 10,$$

where y_i are measurement values and u_i are values of the independent variable. The unknown vector of parameters is $\mathbf{x} = (x_0, x_1, x_2, x_3)^T$. As was said previously, it is recommended to compute Jacobian matrix in a closed form:

$$\begin{aligned}
J_{i0} &= \frac{\partial f_i}{\partial x_0} = \frac{u_i^2 + u_i x_1}{u_i^2 + u_i x_2 + x_3} \\
J_{i1} &= \frac{\partial f_i}{\partial x_1} = \frac{u_i x_0}{u_i^2 + u_i x_2 + x_3} \\
J_{i2} &= \frac{\partial f_i}{\partial x_2} = -\frac{x_0(u_i^2 + u_i x_1)u_i}{(u_i^2 + u_i x_2 + x_3)^2} \\
J_{i3} &= \frac{\partial f_i}{\partial x_3} = -\frac{x_0(u_i^2 + u_i x_1)}{(u_i^2 + u_i x_2 + x_3)^2}
\end{aligned}$$

We are going to use the "hard" starting point defined in[2]. To find a physically meaningful solution, avoid potential division by zero and assure convergence to the global minimum we impose constraints $0 \leq x_j \leq 100, j = 0, 1, 2, 3$.

The code below implements least-squares estimation of $\mathbf{x}$ and finally plots the original data and the fitted model function:

```
>>> from scipy.optimize import least_squares
```

```
>>> def model(x, u):
...     return x[0] * (u ** 2 + x[1] * u) / (u ** 2 + x[2] * u + x[3])
```

```
>>> def fun(x, u, y):
...     return model(x, u) - y
```

```
>>> def jac(x, u, y):
...     J = np.empty((u.size, x.size))
...     den = u ** 2 + x[2] * u + x[3]
...     num = u ** 2 + x[1] * u
...     J[:, 0] = num / den
...     J[:, 1] = x[0] * u / den
...     J[:, 2] = -x[0] * num * u / den ** 2
...     J[:, 3] = -x[0] * num / den ** 2
...     return J
```

[1] J. Kowalik and J. F. Morrison, "Analysis of kinetic data for allosteric enzyme reactions as a nonlinear regression problem", Math. Biosci., vol. 2, pp. 57-66, 1968.

[2] B. M. Averick, R. G. Carter, G.-L. Xue, and JJ Mor{'e}, "The MINPACK-2 Test Problem Collection", 1992. 10.2172/79972

```
>>> u = np.array([4.0, 2.0, 1.0, 5.0e-1, 2.5e-1, 1.67e-1, 1.25e-1, 1.0e-1,
...               8.33e-2, 7.14e-2, 6.25e-2])
>>> y = np.array([1.957e-1, 1.947e-1, 1.735e-1, 1.6e-1, 8.44e-2, 6.27e-2,
...               4.56e-2, 3.42e-2, 3.23e-2, 2.35e-2, 2.46e-2])
>>> x0 = np.array([2.5, 3.9, 4.15, 3.9])
>>> res = least_squares(fun, x0, jac=jac, bounds=(0, 100), args=(u, y), verbose=1)
# may vary
``ftol`` termination condition is satisfied.
Function evaluations 130, initial cost 4.4383e+00, final cost 1.5375e-04, first-order optimality 4.92e-08.
>>> res.x
array([ 0.19280596,  0.19130423,  0.12306063,  0.13607247])
```

```
>>> import matplotlib.pyplot as plt
>>> u_test = np.linspace(0, 5)
>>> y_test = model(res.x, u_test)
>>> plt.plot(u, y, 'o', markersize=4, label='data')
>>> plt.plot(u_test, y_test, label='fitted model')
>>> plt.xlabel("u")
>>> plt.ylabel("y")
>>> plt.legend(loc='lower right')
>>> plt.show()
```

References

Further examples

Three interactive examples below illustrate usage of `least_squares` in greater detail.

1. Large-scale bundle adjustment in scipy demonstrates large-scale capabilities of `least_squares` and how to efficiently compute finite difference approximation of sparse Jacobian.
2. Robust nonlinear regression in scipy shows how to handle outliers with a robust loss function in a nonlinear regression.
3. Solving a discrete boundary-value problem in scipy examines how to solve a large system of equations and use bounds to achieve desired properties of the solution.

For the details about mathematical algorithms behind the implementation refer to documentation of `least_squares`.

Univariate function minimizers (`minimize_scalar`)

Often only the minimum of a univariate function (i.e., a function that takes a scalar as input) is needed. In these circumstances, other optimization techniques have been developed that can work faster. These are accessible from the `minimize_scalar` function, which proposes several algorithms.

Unconstrained minimization (`method='brent'`)

There are, actually, two methods that can be used to minimize a univariate function: `brent` and `golden`, but `golden` is included only for academic purposes and should rarely be used. These can be respectively selected through the `method` parameter in `minimize_scalar`. The `brent` method uses Brent's algorithm for locating a minimum. Optimally, a bracket (the `bracket` parameter) should be given which contains the minimum desired. A bracket is a triple (a, b, c) such that $f(a) > f(b) < f(c)$ and $a < b < c$. If this is not given, then alternatively two starting points can be chosen and a bracket will be found from these points using a simple marching algorithm.

If these two starting points are not provided, 0 and 1 will be used (this may not be the right choice for your function and result in an unexpected minimum being returned).

Here is an example:

```
>>> from scipy.optimize import minimize_scalar
>>> f = lambda x: (x - 2) * (x + 1)**2
>>> res = minimize_scalar(f, method='brent')
>>> print(res.x)
1.0
```

Bounded minimization (`method='bounded'`)

Very often, there are constraints that can be placed on the solution space before minimization occurs. The `bounded` method in `minimize_scalar` is an example of a constrained minimization procedure that provides a rudimentary interval constraint for scalar functions. The interval constraint allows the minimization to occur only between two fixed endpoints, specified using the mandatory `bounds` parameter.

For example, to find the minimum of $J_1(x)$ near $x = 5$, `minimize_scalar` can be called using the interval $[4, 7]$ as a constraint. The result is $x_{\min} = 5.3314$:

```
>>> from scipy.special import j1
>>> res = minimize_scalar(j1, bounds=(4, 7), method='bounded')
>>> res.x
5.33144184241
```

Custom minimizers

Sometimes, it may be useful to use a custom method as a (multivariate or univariate) minimizer, for example, when using some library wrappers of `minimize` (e.g., `basinhopping`).

We can achieve that by, instead of passing a method name, passing a callable (either a function or an object implementing a `__call__` method) as the `method` parameter.

Let us consider an (admittedly rather virtual) need to use a trivial custom multivariate minimization method that will just search the neighborhood in each dimension independently with a fixed step size:

```
>>> from scipy.optimize import OptimizeResult
>>> def custmin(fun, x0, args=(), maxfev=None, stepsize=0.1,
...         maxiter=100, callback=None, **options):
...     bestx = x0
...     besty = fun(x0)
...     funcalls = 1
...     niter = 0
...     improved = True
...     stop = False
...
...     while improved and not stop and niter < maxiter:
...         improved = False
...         niter += 1
...         for dim in range(np.size(x0)):
...             for s in [bestx[dim] - stepsize, bestx[dim] + stepsize]:
...                 testx = np.copy(bestx)
...                 testx[dim] = s
...                 testy = fun(testx, *args)
```

```
...                 funcalls += 1
...                 if testy < besty:
...                     besty = testy
...                     bestx = testx
...                     improved = True
...             if callback is not None:
...                 callback(bestx)
...             if maxfev is not None and funcalls >= maxfev:
...                 stop = True
...                 break
...
...     return OptimizeResult(fun=besty, x=bestx, nit=niter,
...                           nfev=funcalls, success=(niter > 1))
>>> x0 = [1.35, 0.9, 0.8, 1.1, 1.2]
>>> res = minimize(rosen, x0, method=custmin, options=dict(stepsize=0.05))
>>> res.x
array([1., 1., 1., 1., 1.])
```

This will work just as well in case of univariate optimization:

```
>>> def custmin(fun, bracket, args=(), maxfev=None, stepsize=0.1,
...         maxiter=100, callback=None, **options):
...     bestx = (bracket[1] + bracket[0]) / 2.0
...     besty = fun(bestx)
...     funcalls = 1
...     niter = 0
...     improved = True
...     stop = False
...
...     while improved and not stop and niter < maxiter:
...         improved = False
...         niter += 1
...         for testx in [bestx - stepsize, bestx + stepsize]:
...             testy = fun(testx, *args)
...             funcalls += 1
...             if testy < besty:
...                 besty = testy
...                 bestx = testx
...                 improved = True
...         if callback is not None:
...             callback(bestx)
...         if maxfev is not None and funcalls >= maxfev:
...             stop = True
...             break
...
...     return OptimizeResult(fun=besty, x=bestx, nit=niter,
...                           nfev=funcalls, success=(niter > 1))
>>> def f(x):
...    return (x - 2)**2 * (x + 2)**2
>>> res = minimize_scalar(f, bracket=(-3.5, 0), method=custmin,
...                       options=dict(stepsize = 0.05))
>>> res.x
-2.0
```

Root finding

Scalar functions

If one has a single-variable equation, there are multiple different root finding algorithms that can be tried. Most of these algorithms require the endpoints of an interval in which a root is expected (because the function changes signs). In general, `brentq` is the best choice, but the other methods may be useful in certain circumstances or for academic purposes. When a bracket is not available, but one or more derivatives are available, then `newton` (or `halley`, `secant`) may be applicable. This is especially the case if the function is defined on a subset of the complex plane, and the bracketing methods cannot be used.

Fixed-point solving

A problem closely related to finding the zeros of a function is the problem of finding a fixed point of a function. A fixed point of a function is the point at which evaluation of the function returns the point: $g(x) = x$. Clearly, the fixed point of g is the root of $f(x) = g(x) - x$. Equivalently, the root of f is the fixed point of $g(x) = f(x) + x$. The routine `fixed_point` provides a simple iterative method using Aitkens sequence acceleration to estimate the fixed point of g given a starting point.

Sets of equations

Finding a root of a set of non-linear equations can be achieved using the `root` function. Several methods are available, amongst which `hybr` (the default) and `lm`, which, respectively, use the hybrid method of Powell and the Levenberg-Marquardt method from MINPACK.

The following example considers the single-variable transcendental equation

$$x + 2\cos(x) = 0,$$

a root of which can be found as follows:

```
>>> import numpy as np
>>> from scipy.optimize import root
>>> def func(x):
...     return x + 2 * np.cos(x)
>>> sol = root(func, 0.3)
>>> sol.x
array([-1.02986653])
>>> sol.fun
array([ -6.66133815e-16])
```

Consider now a set of non-linear equations

$$\begin{aligned} x_0 \cos(x_1) &= 4, \\ x_0 x_1 - x_1 &= 5. \end{aligned}$$

We define the objective function so that it also returns the Jacobian and indicate this by setting the `jac` parameter to `True`. Also, the Levenberg-Marquardt solver is used here.

```
>>> def func2(x):
...     f = [x[0] * np.cos(x[1]) - 4,
...          x[1]*x[0] - x[1] - 5]
...     df = np.array([[np.cos(x[1]), -x[0] * np.sin(x[1])],
```

```
...                     [x[1], x[0] - 1]])
...     return f, df
>>> sol = root(func2, [1, 1], jac=True, method='lm')
>>> sol.x
array([ 6.50409711,  0.90841421])
```

Root finding for large problems

Methods `hybr` and `lm` in `root` cannot deal with a very large number of variables (*N*), as they need to calculate and invert a dense *N* x *N* Jacobian matrix on every Newton step. This becomes rather inefficient when *N* grows.

Consider, for instance, the following problem: we need to solve the following integrodifferential equation on the square $[0,1] \times [0,1]$:

$$(\partial_x^2 + \partial_y^2) P + 5 \left(\int_0^1 \int_0^1 \cosh(P)\, dx\, dy \right)^2 = 0$$

with the boundary condition $P(x,1) = 1$ on the upper edge and $P = 0$ elsewhere on the boundary of the square. This can be done by approximating the continuous function P by its values on a grid, $P_{n,m} \approx P(nh, mh)$, with a small grid spacing h. The derivatives and integrals can then be approximated; for instance $\partial_x^2 P(x,y) \approx (P(x+h,y) - 2P(x,y) + P(x-h,y))/h^2$. The problem is then equivalent to finding the root of some function `residual(P)`, where `P` is a vector of length $N_x N_y$.

Now, because $N_x N_y$ can be large, methods `hybr` or `lm` in `root` will take a long time to solve this problem. The solution can, however, be found using one of the large-scale solvers, for example `krylov`, `broyden2`, or `anderson`. These use what is known as the inexact Newton method, which instead of computing the Jacobian matrix exactly, forms an approximation for it.

The problem we have can now be solved as follows:

```
import numpy as np
from scipy.optimize import root
from numpy import cosh, zeros_like, mgrid, zeros

# parameters
nx, ny = 75, 75
hx, hy = 1./(nx-1), 1./(ny-1)

P_left, P_right = 0, 0
P_top, P_bottom = 1, 0

def residual(P):
   d2x = zeros_like(P)
   d2y = zeros_like(P)

   d2x[1:-1] = (P[2:]   - 2*P[1:-1] + P[:-2]) / hx/hx
   d2x[0]    = (P[1]    - 2*P[0]    + P_left)/hx/hx
   d2x[-1]   = (P_right - 2*P[-1]   + P[-2])/hx/hx

   d2y[:,1:-1] = (P[:,2:] - 2*P[:,1:-1] + P[:,:-2])/hy/hy
   d2y[:,0]    = (P[:,1]  - 2*P[:,0]    + P_bottom)/hy/hy
   d2y[:,-1]   = (P_top   - 2*P[:,-1]   + P[:,-2])/hy/hy

   return d2x + d2y + 5*cosh(P).mean()**2
```

```
# solve
guess = zeros((nx, ny), float)
sol = root(residual, guess, method='krylov', options={'disp': True})
#sol = root(residual, guess, method='broyden2', options={'disp': True, 'max_rank': 50})
#sol = root(residual, guess, method='anderson', options={'disp': True, 'M': 10})
print('Residual: %g' % abs(residual(sol.x)).max())

# visualize
import matplotlib.pyplot as plt
x, y = mgrid[0:1:(nx*1j), 0:1:(ny*1j)]
plt.pcolormesh(x, y, sol.x, shading='gouraud')
plt.colorbar()
plt.show()
```

Still too slow? Preconditioning.

When looking for the zero of the functions $f_i(\mathbf{x}) = 0$, *i = 1, 2, ..., N*, the `krylov` solver spends most of the time inverting the Jacobian matrix,

$$J_{ij} = \frac{\partial f_i}{\partial x_j}.$$

If you have an approximation for the inverse matrix $M \approx J^{-1}$, you can use it for *preconditioning* the linear-inversion problem. The idea is that instead of solving $J\mathbf{s} = \mathbf{y}$ one solves $MJ\mathbf{s} = M\mathbf{y}$: since matrix MJ is "closer" to the identity matrix than J is, the equation should be easier for the Krylov method to deal with.

The matrix M can be passed to `root` with method `krylov` as an option `options['jac_options']['inner_M']`. It can be a (sparse) matrix or a `scipy.sparse.linalg.LinearOperator` instance.

For the problem in the previous section, we note that the function to solve consists of two parts: the first one is the application of the Laplace operator, $[\partial_x^2 + \partial_y^2]P$, and the second is the integral. We can actually easily compute the Jacobian corresponding to the Laplace operator part: we know that in 1-D

$$\partial_x^2 \approx \frac{1}{h_x^2} \begin{pmatrix} -2 & 1 & 0 & 0\cdots \\ 1 & -2 & 1 & 0\cdots \\ 0 & 1 & -2 & 1\cdots \\ \cdots & & & \end{pmatrix} = h_x^{-2} L$$

so that the whole 2-D operator is represented by

$$J_1 = \partial_x^2 + \partial_y^2 \simeq h_x^{-2} L \otimes I + h_y^{-2} I \otimes L$$

The matrix J_2 of the Jacobian corresponding to the integral is more difficult to calculate, and since *all* of it entries are nonzero, it will be difficult to invert. J_1 on the other hand is a relatively simple matrix, and can be inverted by `scipy.sparse.linalg.splu` (or the inverse can be approximated by `scipy.sparse.linalg.spilu`). So we are content to take $M \approx J_1^{-1}$ and hope for the best.

In the example below, we use the preconditioner $M = J_1^{-1}$.

Resulting run, first without preconditioning:

```
0:  |F(x)| = 803.614; step 1; tol 0.000257947
1:  |F(x)| = 345.912; step 1; tol 0.166755
2:  |F(x)| = 139.159; step 1; tol 0.145657
3:  |F(x)| = 27.3682; step 1; tol 0.0348109
4:  |F(x)| = 1.03303; step 1; tol 0.00128227
5:  |F(x)| = 0.0406634; step 1; tol 0.00139451
6:  |F(x)| = 0.00344341; step 1; tol 0.00645373
7:  |F(x)| = 0.000153671; step 1; tol 0.00179246
8:  |F(x)| = 6.7424e-06; step 1; tol 0.00173256
Residual 3.57078908664e-07
Evaluations 317
```

and then with preconditioning:

```
0:  |F(x)| = 136.993; step 1; tol 7.49599e-06
1:  |F(x)| = 4.80983; step 1; tol 0.00110945
2:  |F(x)| = 0.195942; step 1; tol 0.00149362
3:  |F(x)| = 0.000563597; step 1; tol 7.44604e-06
4:  |F(x)| = 1.00698e-09; step 1; tol 2.87308e-12
Residual 9.29603061195e-11
Evaluations 77
```

Using a preconditioner reduced the number of evaluations of the `residual` function by a factor of 4. For problems where the residual is expensive to compute, good preconditioning can be crucial --- it can even decide whether the problem is solvable in practice or not.

Preconditioning is an art, science, and industry. Here, we were lucky in making a simple choice that worked reasonably well, but there is a lot more depth to this topic than is shown here.

Linear programming (`linprog`)

The function `linprog` can minimize a linear objective function subject to linear equality and inequality constraints. This kind of problem is well known as linear programming. Linear programming solves problems of the following form:

$$\begin{aligned} \min_x \ & c^T x \\ \text{such that } & A_{ub} x \leq b_{ub}, \\ & A_{eq} x = b_{eq}, \\ & l \leq x \leq u, \end{aligned}$$

where x is a vector of decision variables; c, b_{ub}, b_{eq}, l, and u are vectors; and A_{ub} and A_{eq} are matrices.

In this tutorial, we will try to solve a typical linear programming problem using `linprog`.

Linear programming example

Consider the following simple linear programming problem:

$$\begin{aligned}
\max_{x_1,x_2,x_3,x_4} \quad & 29x_1 + 45x_2 \\
\text{such that} \quad & x_1 - x_2 - 3x_3 \leq 5 \\
& 2x_1 - 3x_2 - 7x_3 + 3x_4 \geq 10 \\
& 2x_1 + 8x_2 + x_3 = 60 \\
& 4x_1 + 4x_2 + x_4 = 60 \\
& 0 \leq x_0 \\
& 0 \leq x_1 \leq 5 \\
& x_2 \leq 0.5 \\
& -3 \leq x_3
\end{aligned}$$

We need some mathematical manipulations to convert the target problem to the form accepted by `linprog`.

First of all, let's consider the objective function. We want to maximize the objective function, but `linprog` can only accept a minimization problem. This is easily remedied by converting the maximize $29x_1 + 45x_2$ to minimizing $-29x_1 - 45x_2$. Also, x_3, x_4 are not shown in the objective function. That means the weights corresponding with x_3, x_4 are zero. So, the objective function can be converted to:

$$\min_{x_1,x_2,x_3,x_4} \quad -29x_1 - 45x_2 + 0x_3 + 0x_4$$

If we define the vector of decision variables $x = [x_1, x_2, x_3, x_4]^T$, the objective weights vector c of `linprog` in this problem should be

$$c = [-29, -45, 0, 0]^T$$

Next, let's consider the two inequality constraints. The first one is a "less than" inequality, so it is already in the form accepted by `linprog`. The second one is a "greater than" inequality, so we need to multiply both sides by -1 to convert it to a "less than" inequality. Explicitly showing zero coefficients, we have:

$$\begin{aligned}
x_1 - x_2 - 3x_3 + 0x_4 &\leq 5 \\
-2x_1 + 3x_2 + 7x_3 - 3x_4 &\leq -10
\end{aligned}$$

These equations can be converted to matrix form:

$$A_{ub}x \leq b_{ub}$$

where

$$A_{ub} = \begin{bmatrix} 1 & -1 & -3 & 0 \\ -2 & 3 & 7 & -3 \end{bmatrix}$$

$$b_{ub} = \begin{bmatrix} 5 \\ -10 \end{bmatrix}$$

Next, let's consider the two equality constraints. Showing zero weights explicitly, these are:

$$2x_1 + 8x_2 + 1x_3 + 0x_4 = 60$$
$$4x_1 + 4x_2 + 0x_3 + 1x_4 = 60$$

These equations can be converted to matrix form:

$$A_{eq}x = b_{eq}$$

where

$$A_{eq} = \begin{bmatrix} 2 & 8 & 1 & 0 \\ 4 & 4 & 0 & 1 \end{bmatrix}$$

$$b_{eq} = \begin{bmatrix} 60 \\ 60 \end{bmatrix}$$

Lastly, let's consider the separate inequality constraints on individual decision variables, which are known as "box constraints" or "simple bounds". These constraints can be applied using the bounds argument of `linprog`. As noted in the `linprog` documentation, the default value of bounds is `(0, None)`, meaning that the lower bound on each decision variable is 0, and the upper bound on each decision variable is infinity: all the decision variables are non-negative. Our bounds are different, so we will need to specify the lower and upper bound on each decision variable as a tuple and group these tuples into a list.

Finally, we can solve the transformed problem using `linprog`.

```
>>> import numpy as np
>>> from scipy.optimize import linprog
>>> c = np.array([-29.0, -45.0, 0.0, 0.0])
>>> A_ub = np.array([[1.0, -1.0, -3.0, 0.0],
...                  [-2.0, 3.0, 7.0, -3.0]])
>>> b_ub = np.array([5.0, -10.0])
>>> A_eq = np.array([[2.0, 8.0, 1.0, 0.0],
...                  [4.0, 4.0, 0.0, 1.0]])
>>> b_eq = np.array([60.0, 60.0])
>>> x0_bounds = (0, None)
>>> x1_bounds = (0, 5.0)
>>> x2_bounds = (-np.inf, 0.5)  # +/- np.inf can be used instead of None
>>> x3_bounds = (-3.0, None)
>>> bounds = [x0_bounds, x1_bounds, x2_bounds, x3_bounds]
>>> result = linprog(c, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq, bounds=bounds)
>>> print(result.message)
The problem is infeasible. (HiGHS Status 8: model_status is Infeasible; primal_status is None)
```

The result states that our problem is infeasible, meaning that there is no solution vector that satisfies all the constraints. That doesn't necessarily mean we did anything wrong; some problems truly are infeasible. Suppose, however, that we were to decide that our bound constraint on x_1 was too tight and that it could be loosened to $0 \leq x_1 \leq 6$. After adjusting our code `x1_bounds = (0, 6)` to reflect the change and executing it again:

```
>>> x1_bounds = (0, 6)
>>> bounds = [x0_bounds, x1_bounds, x2_bounds, x3_bounds]
>>> result = linprog(c, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq, bounds=bounds)
>>> print(result.message)
Optimization terminated successfully. (HiGHS Status 7: Optimal)
```

The result shows the optimization was successful. We can check the objective value (`result.fun`) is same as $c^T x$:

```
>>> x = np.array(result.x)
>>> obj = result.fun
>>> print(c @ x)
-505.97435889013434  # may vary
>>> print(obj)
-505.97435889013434  # may vary
```

We can also check that all constraints are satisfied within reasonable tolerances:

```
>>> print(b_ub - (A_ub @ x).flatten())  # this is equivalent to result.slack
[ 6.52747190e-10, -2.26730279e-09]  # may vary
>>> print(b_eq - (A_eq @ x).flatten())  # this is equivalent to result.con
[ 9.78840831e-09, 1.04662945e-08]]  # may vary
>>> print([0 <= result.x[0], 0 <= result.x[1] <= 6.0, result.x[2] <= 0.5, -3.0 <= result.x[3]])
[True, True, True, True]
```

Assignment problems

Linear sum assignment problem example

Consider the problem of selecting students for a swimming medley relay team. We have a table showing times for each swimming style of five students:

Student	backstroke	breaststroke	butterfly	freestyle
A	43.5	47.1	48.4	38.2
B	45.5	42.1	49.6	36.8
C	43.4	39.1	42.1	43.2
D	46.5	44.1	44.5	41.2
E	46.3	47.8	50.4	37.2

We need to choose a student for each of the four swimming styles such that the total relay time is minimized. This is a typical linear sum assignment problem. We can use `linear_sum_assignment` to solve it.

The linear sum assignment problem is one of the most famous combinatorial optimization problems. Given a "cost matrix" C, the problem is to choose

- exactly one element from each row
- without choosing more than one element from any column
- such that the sum of the chosen elements is minimized

In other words, we need to assign each row to one column such that the sum of the corresponding entries is minimized.

Formally, let X be a boolean matrix where $X[i, j] = 1$ iff row i is assigned to column j. Then the optimal assignment has cost

$$\min \sum_i \sum_j C_{i,j} X_{i,j}$$

The first step is to define the cost matrix. In this example, we want to assign each swimming style to a student. `linear_sum_assignment` is able to assign each row of a cost matrix to a column.

Therefore, to form the cost matrix, the table above needs to be transposed so that the rows correspond with swimming styles and the columns correspond with students:

```
>>> import numpy as np
>>> cost = np.array([[43.5, 45.5, 43.4, 46.5, 46.3],
...                  [47.1, 42.1, 39.1, 44.1, 47.8],
...                  [48.4, 49.6, 42.1, 44.5, 50.4],
...                  [38.2, 36.8, 43.2, 41.2, 37.2]])
```

We can solve the assignment problem with `linear_sum_assignment`:

```
>>> from scipy.optimize import linear_sum_assignment
>>> row_ind, col_ind = linear_sum_assignment(cost)
```

The `row_ind` and `col_ind` are optimal assigned matrix indexes of the cost matrix:

```
>>> row_ind
array([0, 1, 2, 3])
>>> col_ind
array([0, 2, 3, 1])
```

The optimal assignment is:

```
>>> styles = np.array(["backstroke", "breaststroke", "butterfly", "freestyle"])[row_ind]
>>> students = np.array(["A", "B", "C", "D", "E"])[col_ind]
>>> dict(zip(styles, students))
{'backstroke': 'A', 'breaststroke': 'C', 'butterfly': 'D', 'freestyle': 'B'}
```

The optimal total medley time is:

```
>>> cost[row_ind, col_ind].sum()
163.89999999999998
```

Note that this result is not the same as the sum of the minimum times for each swimming style:

```
>>> np.min(cost, axis=1).sum()
161.39999999999998
```

because student ”C” is the best swimmer in both ”breaststroke” and ”butterfly” style. We cannot assign student ”C” to both styles, so we assigned student C to the ”breaststroke” style and D to the ”butterfly” style to minimize the total time.

References

Some further reading and related software, such as Newton-Krylov [KK], PETSc [PP], and PyAMG [AMG]:

Mixed integer linear programming

Knapsack problem example

The knapsack problem is a well known combinatorial optimization problem. Given a set of items, each with a size and a value, the problem is to choose the items that maximize the total value under the condition that the total size is below a certain threshold.

Formally, let

- x_i be a boolean variable that indicates whether item i is included in the knapsack,
- n be the total number of items,
- v_i be the value of item i,
- s_i be the size of item i, and
- C be the capacity of the knapsack.

Then the problem is:

$$\max \sum_i^n v_i x_i$$

$$\text{subject to} \sum_i^n s_i x_i \leq C, x_i \in \{0, 1\}$$

Although the objective function and inequality constraints are linear in the *decision variables* x_i, this differs from a typical linear programming problem in that the decision variables can only assume integer values. Specifically, our decision variables can only be 0 or 1, so this is known as a *binary integer linear program* (BILP). Such a problem falls within the larger class of *mixed integer linear programs* (MILPs), which we we can solve with `milp`.

In our example, there are 8 items to choose from, and the size and value of each is specified as follows.

```
>>> import numpy as np
>>> from scipy import optimize
>>> sizes = np.array([21, 11, 15, 9, 34, 25, 41, 52])
>>> values = np.array([22, 12, 16, 10, 35, 26, 42, 53])
```

We need to constrain our eight decision variables to be binary. We do so by adding a `Bounds`: constraint to ensure that they lie between 0 and 1, and we apply "integrality" constraints to ensure that they are *either* 0 *or* 1.

```
>>> bounds = optimize.Bounds(0, 1)  # 0 <= x_i <= 1
>>> integrality = np.full_like(values, True)  # x_i are integers
```

The knapsack capacity constraint is specified using `LinearConstraint`.

```
>>> capacity = 100
>>> constraints = optimize.LinearConstraint(A=sizes, lb=0, ub=capacity)
```

If we are following the usual rules of linear algebra, the input `A` should be a two-dimensional matrix, and the lower and upper bounds `lb` and `ub` should be one-dimensional vectors, but `LinearConstraint` is forgiving as long as the inputs can be broadcast to consistent shapes.

Using the variables defined above, we can solve the knapsack problem using `milp`. Note that `milp` minimizes the objective function, but we want to maximize the total value, so we set `c` to be negative of the values.

```
>>> from scipy.optimize import milp
>>> res = milp(c=-values, constraints=constraints,
...            integrality=integrality, bounds=bounds)
```

Let's check the result:

```
>>> res.success
True
>>> res.x
array([1., 1., 0., 1., 1., 1., 0., 0.])
```

This means that we should select the items 1, 2, 4, 5, 6 to optimize the total value under the size constraint. Note that this is different from we would have obtained had we solved the *linear programming relaxation* (without integrality constraints) and attempted to round the decision variables.

```
>>> from scipy.optimize import milp
>>> res = milp(c=-values, constraints=constraints,
...            integrality=False, bounds=bounds)
>>> res.x
array([1.        , 1.        , 1.        , 1.        ,
       0.55882353, 1.        , 0.        , 0.        ])
```

If we were to round this solution up to `array([1., 1., 1., 1., 1., 1., 0., 0.])`, our knapsack would be over the capacity constraint, whereas if we were to round down to `array([1., 1., 1., 1., 0., 1., 0., 0.])`, we would have a sub-optimal solution.

For more MILP tutorials, see the Jupyter notebooks on SciPy Cookbooks:

- Compressed Sensing l1 program
- Compressed Sensing l0 program

Parallel execution support

Some SciPy optimization methods, such as `differential_evolution`, offer parallelization through the use of a `workers` keyword.

For `differential_evolution` there are two loops (iteration) levels in the algorithm. The outer loop represents successive generations of a population. This loop can't be parallelized. For a given generation candidate solutions are generated that have to be compared against existing population members. The fitness of the candidate solution can be done in a loop, but it's also possible to parallelize the calculation.

Parallelization is also possible in other optimization algorithms. For example in various `minimize` methods numerical differentiation is used to estimate derivatives. For a simple gradient calculation using two-point forward differences a total of `N + 1` objective function calculations have to be done, where `N` is the number of parameters. These are just small perturbations around a given location (the +1). Those `N + 1` calculations are also parallelizable. The calculation of numerical derivatives are used by the minimization algorithm to generate new steps.

Each optimization algorithm is quite different in how they work, but they often have locations where multiple objective function calculations are required before the algorithm does something else. Those locations are what can be parallelized. There are therefore common characteristics in how `workers` is used. These commonalities are described below.

If an int is supplied then a `multiprocessing.Pool` is created, with the object's `map` method being used to evaluate solutions in parallel. With this approach it is mandatory that the objective function is pickleable. Lambda functions do not meet that requirement.

```
>>> import numpy as np
>>> from scipy.optimize import rosen, differential_evolution, Bounds
>>> bnds = Bounds([0., 0., 0.], [10., 10., 10.])
>>> res = differential_evolution(rosen, bnds, workers=2, updating='deferred')
```

It is also possible to use a map-like callable as a worker. Here the map-like function is provided with a series of vectors that the optimization algorithm provides. The map-like function needs to evaluate each vector against the objective function. In the following example we use `multiprocessing.Pool` as the map-like. As before, the objective function still needs to be pickleable. This example is semantically identical to the previous example.

```
>>> from multiprocessing import Pool
>>> with Pool(2) as pwl:
...     res = differential_evolution(rosen, bnds, workers=pwl.map, updating='deferred')
```

It can be an advantage to use this pattern because the Pool can be re-used for further calculations - there is a significant amount of overhead in creating those objects. Alternatives to `multiprocessing.Pool` include the mpi4py package, which enables parallel processing on clusters.

In Scipy 1.16.0 the `workers` keyword was introduced to selected `minimize` methods. Here parallelization is typically applied during numerical differentiation. Either of the two approaches outlined above can be used, although it's strongly advised to supply the map-like callable due to the overhead of creating new processes. Performance gains will only be made if the objective function is expensive to calculate. Let's compare how much parallelization can help compared to the serial version. To simulate a slow function we use the `time` package.

```
>>> import time
>>> def slow_func(x):
...     time.sleep(0.0002)
...     return rosen(x)
```

Examine the serial minimization first:

```
In [1]: rng = np.random.default_rng()

In [2]: x0 = rng.uniform(low=0.0, high=10.0, size=(20,))

In [3]: %timeit minimize(slow_func, x0, method='L-BFGS-B')  # serial approach
365 ms ± 6.17 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)  # may vary
```

Now the parallel version:

```
In [4]: with Pool() as pwl:  # parallel approach
...         %timeit minimize(slow_func, x0, method='L-BFGS-B', options={'workers':pwl.map})
70.5 ms ± 146 µs per loop (mean ± std. dev. of 7 runs, 1 loop each)  # may vary
```

If the objective function can be vectorized, then a map-like can be used to take advantage of vectorization during function evaluation. Vectorization means that the objective function can carry out the required calculations in a single (rather than multiple) call, which is typically very efficient:

```
In [5]: def vectorized_maplike(fun, iterable):
...         arr = np.array([i for i in iter(iterable)])   # arr.shape = (S, N)
...         arr_t = arr.T                                 # arr_t.shape = (N, S)
...         r = slow_func(arr_t)                          # calculation vectorized over S
...         return r

In [6]: %timeit minimize(slow_func, x0, method='L-BFGS-B', options={'workers':vectorized_maplike})
38.9 ms ± 734 µs per loop (mean ± std. dev. of 7 runs, 10 loops each)  # may vary
```

There are several important points to note about this example:

- The iterable represents the series of parameter vectors that the algorithm wishes to be evaluated.
- The iterable is first converted to an iterator, before being made into an array via a list comprehension. This allows the iterable to be a generator, list, array, etc.
- Within the map-like the calculation is done using `slow_func` instead of using `fun`. The map-like is actually supplied with a wrapped version of the objective function. The wrapping is used to detect various types of common user errors, including checking whether the objective function returns a scalar. If `fun` is used then a `RuntimeError` will result, because `fun(arr_t)` will be a 1-D array and not a scalar. We therefore use `slow_func` directly.
- `arr.T` is sent to the objective function. This is because `arr.shape == (S, N)`, where `S` is the number of parameter vectors to evaluate and `N` is the number of variables. For `slow_func` vectorization occurs on `(N, S)` shaped arrays.

- This approach is not needed for `differential_evolution` as that minimizer already has a keyword for vectorization.

AMG PyAMG (algebraic multigrid preconditioners/solvers) https://github.com/pyamg/pyamg/issues

CGT Conn, A. R., Gould, N. I., & Toint, P. L. "Trust region methods". Siam. (2000). pp. 169-200.

EQSQP Lalee, Marucha, Jorge Nocedal, and Todd Plantega. 1998. On the implementation of an algorithm for large-scale equality constrained optimization. SIAM Journal on Optimization 8.3: 682-706.

GLTR N. Gould, S. Lucidi, M. Roma, P. Toint: "Solving the Trust-Region Subproblem using the Lanczos Method", SIAM J. Optim., 9(2), 504--525, (1999). `10.1137/S1052623497322735`

KK D.A. Knoll and D.E. Keyes, "Jacobian-free Newton-Krylov methods", J. Comp. Phys. 193, 357 (2004). `10.1016/j.jcp.2003.08.010`

NW J. Nocedal, S.J. Wright "Numerical optimization." 2nd edition. Springer Science (2006).

PP PETSc https://www.mcs.anl.gov/petsc/ and its Python bindings https://bitbucket.org/petsc/petsc4py/

TRIP Byrd, Richard H., Mary E. Hribar, and Jorge Nocedal. 1999. An interior point algorithm for large-scale nonlinear programming. SIAM Journal on Optimization 9.4: 877-900.

TRLIB F. Lenders, C. Kirches, A. Potschka: "trlib: A vector-free implementation of the GLTR method for iterative solution of the trust region problem", `1611.04718`

Statistics (scipy.stats): overview

Source: https://docs.scipy.org/doc/scipy/tutorial/stats.html

Statistics (`scipy.stats`) ==========

In this tutorial, we discuss many, but certainly not all, features of `scipy.stats`. The intention here is to provide a user with a working knowledge of this package. We refer to the reference manual for further details.

Note: This documentation is work in progress.

Probability distributions

Source: https://docs.scipy.org/doc/scipy/tutorial/stats/probability_distributions.html

SciPy has two infrastructures for working with probability distributions. This tutorial is for the older one, which has many pre-defined distributions; however, the new infrastructure can be used with most of these and has many advantages. For the new infrastructure, see rv_infrastructure.

There are two general distribution classes that have been implemented for encapsulating `continuous random variables <continuous-random-variables>` and `discrete random variables <discrete-random-variables>`. Over 100 continuous random variables (RVs) and 20 discrete random variables have been implemented using these classes. For mathematical reference information about individual distributions, please see continuous and discrete.

All of the statistics functions are located in the sub-package `scipy.stats` and a fairly complete listing of these functions and random variables available can also be obtained from the docstring for the stats sub-package.

In the discussion below, we mostly focus on continuous RVs. Nearly everything also applies to discrete variables, but we point out some differences here: discrete_points_label.

In the code samples below, we assume that the `scipy.stats` package is imported as

```
>>> from scipy import stats
```

and in some cases we assume that individual objects are imported as

```
>>> from scipy.stats import norm
```

Getting help

First of all, all distributions are accompanied with help functions. To obtain just some basic information, we print the relevant docstring: `print(stats.norm.__doc__)`.

To find the support, i.e., upper and lower bounds of the distribution, call:

```
>>> print('bounds of distribution lower: %s, upper: %s' % norm.support())
bounds of distribution lower: -inf, upper: inf
```

We can list all methods and properties of the distribution with `dir(norm)`. As it turns out, some of the methods are private, although they are not named as such (their names do not start with a leading underscore), for example `veccdf`, are only available for internal calculation (those methods will give warnings when one tries to use them, and will be removed at some point).

To obtain the *real* main methods, we list the methods of the frozen distribution. (We explain the meaning of a `frozen` distribution below).

```
>>> rv = norm()
>>> dir(rv)  # reformatted
['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__',
 '__format__', '__ge__', '__getattribute__', '__gt__', '__hash__',
 '__init__', '__le__', '__lt__', '__module__', '__ne__', '__new__',
 '__reduce__', '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__',
 '__str__', '__subclasshook__', '__weakref__', 'a', 'args', 'b', 'cdf',
 'dist', 'entropy', 'expect', 'interval', 'isf', 'kwds', 'logcdf',
```

```
 'logpdf', 'logpmf', 'logsf', 'mean', 'median', 'moment', 'pdf', 'pmf',
 'ppf', 'random_state', 'rvs', 'sf', 'stats', 'std', 'var']
```

Finally, we can obtain the list of available distribution through introspection:

```
>>> dist_continu = [d for d in dir(stats) if
...                 isinstance(getattr(stats, d), stats.rv_continuous)]
>>> dist_discrete = [d for d in dir(stats) if
...                  isinstance(getattr(stats, d), stats.rv_discrete)]
>>> print('number of continuous distributions: %d' % len(dist_continu))
number of continuous distributions: 109
>>> print('number of discrete distributions:   %d' % len(dist_discrete))
number of discrete distributions:   21
```

Common methods

The main public methods for continuous RVs are:

- rvs: Random Variates
- pdf: Probability Density Function
- cdf: Cumulative Distribution Function
- sf: Survival Function (1-CDF)
- ppf: Percent Point Function (Inverse of CDF)
- isf: Inverse Survival Function (Inverse of SF)
- stats: Return mean, variance, (Fisher's) skew, or (Fisher's) kurtosis
- moment: non-central moments of the distribution

Let's take a normal RV as an example.

```
>>> norm.cdf(0)
0.5
```

To compute the `cdf` at a number of points, we can pass a list or a numpy array.

```
>>> norm.cdf([-1., 0, 1])
array([ 0.15865525,  0.5,  0.84134475])
>>> import numpy as np
>>> norm.cdf(np.array([-1., 0, 1]))
array([ 0.15865525,  0.5,  0.84134475])
```

Thus, the basic methods, such as `pdf`, `cdf`, and so on, are vectorized.

Other generally useful methods are supported too:

```
>>> norm.mean(), norm.std(), norm.var()
(0.0, 1.0, 1.0)
>>> norm.stats(moments="mv")
(array(0.0), array(1.0))
```

To find the median of a distribution, we can use the percent point function `ppf`, which is the inverse of the `cdf`:

```
>>> norm.ppf(0.5)
0.0
```

To generate a sequence of random variates, use the `size` keyword argument:

```
>>> norm.rvs(size=3)
array([-0.35687759,  1.34347647, -0.11710531])   # random
```

Don't think that `norm.rvs(5)` generates 5 variates:

```
>>> norm.rvs(5)
5.471435163732493  # random
```

Here, `5` with no keyword is being interpreted as the first possible keyword argument, `loc`, which is the first of a pair of keyword arguments taken by all continuous distributions. This brings us to the topic of the next subsection.

Random number generation

Drawing random numbers relies on generators from `numpy.random` package. In the examples above, the specific stream of random numbers is not reproducible across runs. To achieve reproducibility, you can explicitly *seed* a random number generator. In NumPy, a generator is an instance of `numpy.random.Generator`. Here is the canonical way to create a generator:

```
>>> from numpy.random import default_rng
>>> rng = default_rng()
```

And fixing the seed can be done like this:

```
>>> # do NOT copy this value
>>> rng = default_rng(301439351238479871608357552876690613766)
```

> Warning
> Do not use this number or common values such as 0. Using just a small set of seeds to instantiate larger state spaces means that there are some initial states that are impossible to reach. This creates some biases if everyone uses such values. A good way to get a seed is to use a `numpy.random.SeedSequence`:
>
> ```
> >>> from numpy.random import SeedSequence
> >>> print(SeedSequence().entropy)
> 301439351238479871608357552876690613766  # random
> ```

The `random_state` parameter in distributions accepts an instance of `numpy.random.Generator` class, or an integer, which is then used to seed an internal `Generator` object:

```
>>> norm.rvs(size=5, random_state=rng)
array([ 0.47143516, -1.19097569,  1.43270697, -0.3126519 , -0.72058873])  # random
```

For further info, see NumPy's documentation.

To learn more about the random number samplers implemented in SciPy, see `non-uniform random number sampling tutorial <non-uniform-random-number-sampling>` and `quasi monte carlo tutorial <quasi-monte-carlo>`

Shifting and scaling

All continuous distributions take `loc` and `scale` as keyword parameters to adjust the location and scale of the distribution, e.g., for the standard normal distribution, the location is the mean and the scale is the standard deviation.

```
>>> norm.stats(loc=3, scale=4, moments="mv")
(array(3.0), array(16.0))
```

In many cases, the standardized distribution for a random variable `X` is obtained through the transformation `(X - loc) / scale`. The default values are `loc = 0` and `scale = 1`.

Smart use of `loc` and `scale` can help modify the standard distributions in many ways. To illustrate the scaling further, the `cdf` of an exponentially distributed RV with mean $1/\lambda$ is given by

$$F(x) = 1 - \exp(-\lambda x)$$

By applying the scaling rule above, it can be seen that by taking `scale = 1./lambda` we get the proper scale.

```
>>> from scipy.stats import expon
>>> expon.mean(scale=3.)
3.0
```

> Note
> Distributions that take shape parameters may require more than simple application of `loc` and/or `scale` to achieve the desired form. For example, the distribution of 2-D vector lengths given a constant vector of length R perturbed by independent N(0, σ^2) deviations in each component is rice(R/σ, scale= σ). The first argument is a shape parameter that needs to be scaled along with x.

The uniform distribution is also interesting:

```
>>> from scipy.stats import uniform
>>> uniform.cdf([0, 1, 2, 3, 4, 5], loc=1, scale=4)
array([ 0.  ,  0.  ,  0.25,  0.5 ,  0.75,  1.  ])
```

Finally, recall from the previous paragraph that we are left with the problem of the meaning of `norm.rvs(5)`. As it turns out, calling a distribution like this, the first argument, i.e., the 5, gets passed to set the `loc` parameter. Let's see:

```
>>> np.mean(norm.rvs(5, size=500))
5.0098355106969992  # random
```

Thus, to explain the output of the example of the last section: `norm.rvs(5)` generates a single normally distributed random variate with mean `loc=5`, because of the default `size=1`.

We recommend that you set `loc` and `scale` parameters explicitly, by passing the values as keywords rather than as arguments. Repetition can be minimized when calling more than one method of a given RV by using the technique of Freezing a Distribution, as explained below.

Shape parameters

While a general continuous random variable can be shifted and scaled with the `loc` and `scale` parameters, some distributions require additional shape parameters. For instance, the gamma distribution with density

$$\gamma(x, a) = \frac{\lambda(\lambda x)^{a-1}}{\Gamma(a)} e^{-\lambda x} ,$$

requires the shape parameter a. Observe that setting λ can be obtained by setting the `scale` keyword to $1/\lambda$.

Let's check the number and name of the shape parameters of the gamma distribution. (We know from the above that this should be 1.)

```
>>> from scipy.stats import gamma
>>> gamma.numargs
1
>>> gamma.shapes
'a'
```

Now, we set the value of the shape variable to 1 to obtain the exponential distribution, so that we compare easily whether we get the results we expect.

```
>>> gamma(1, scale=2.).stats(moments="mv")
(array(2.0), array(4.0))
```

Notice that we can also specify shape parameters as keywords:

```
>>> gamma(a=1, scale=2.).stats(moments="mv")
(array(2.0), array(4.0))
```

Freezing a distribution

Passing the `loc` and `scale` keywords time and again can become quite bothersome. The concept of `freezing` a RV is used to solve such problems.

```
>>> rv = gamma(1, scale=2.)
```

By using `rv` we no longer have to include the scale or the shape parameters anymore. Thus, distributions can be used in one of two ways, either by passing all distribution parameters to each method call (such as we did earlier) or by freezing the parameters for the instance of the distribution. Let us check this:

```
>>> rv.mean(), rv.std()
(2.0, 2.0)
```

This is, indeed, what we should get.

Broadcasting

The basic methods `pdf`, and so on, satisfy the usual numpy broadcasting rules. For example, we can calculate the critical values for the upper tail of the t distribution for different probabilities and degrees of freedom.

```
>>> stats.t.isf([0.1, 0.05, 0.01], [[10], [11]])
array([[ 1.37218364,  1.81246112,  2.76376946],
       [ 1.36343032,  1.79588482,  2.71807918]])
```

Here, the first row contains the critical values for 10 degrees of freedom and the second row for 11 degrees of freedom (d.o.f.). Thus, the broadcasting rules give the same result of calling `isf` twice:

```
>>> stats.t.isf([0.1, 0.05, 0.01], 10)
array([ 1.37218364,  1.81246112,  2.76376946])
>>> stats.t.isf([0.1, 0.05, 0.01], 11)
array([ 1.36343032,  1.79588482,  2.71807918])
```

If the array with probabilities, i.e., `[0.1, 0.05, 0.01]` and the array of degrees of freedom i.e., `[10, 11, 12]`, have the same array shape, then element-wise matching is used. As an example,

we can obtain the 10% tail for 10 d.o.f., the 5% tail for 11 d.o.f. and the 1% tail for 12 d.o.f. by calling

```
>>> stats.t.isf([0.1, 0.05, 0.01], [10, 11, 12])
array([ 1.37218364,  1.79588482,  2.68099799])
```

Specific points for discrete distributions

Discrete distributions have mostly the same basic methods as the continuous distributions. However, `pdf` is replaced by the probability mass function `pmf`, no estimation methods, such as fit, are available, and `scale` is not a valid keyword parameter. The location parameter, keyword `loc`, can still be used to shift the distribution.

The computation of the cdf requires some extra attention. In the case of a continuous distribution, the cumulative distribution function is, in most standard cases, strictly monotonic increasing in the bounds (a,b) and has, therefore, a unique inverse. The cdf of a discrete distribution, however, is a step function, hence the inverse cdf, i.e., the percent point function, requires a different definition:

```
ppf(q) = min{x : cdf(x) >= q, x integer}
```

For further info, see the docs here.

We can look at the hypergeometric distribution as an example

```
>>> from scipy.stats import hypergeom
>>> [M, n, N] = [20, 7, 12]
```

If we use the cdf at some integer points and then evaluate the ppf at those cdf values, we get the initial integers back, for example

```
>>> x = np.arange(4) * 2
>>> x
array([0, 2, 4, 6])
>>> prb = hypergeom.cdf(x, M, n, N)
>>> prb
array([  1.03199174e-04,   5.21155831e-02,   6.08359133e-01,
         9.89783282e-01])
>>> hypergeom.ppf(prb, M, n, N)
array([ 0.,  2.,  4.,  6.])
```

If we use values that are not at the kinks of the cdf step function, we get the next higher integer back:

```
>>> hypergeom.ppf(prb + 1e-8, M, n, N)
array([ 1.,  3.,  5.,  7.])
>>> hypergeom.ppf(prb - 1e-8, M, n, N)
array([ 0.,  2.,  4.,  6.])
```

Fitting distributions

The main additional methods of the not frozen distribution are related to the estimation of distribution parameters:

- **fit: maximum likelihood estimation of distribution parameters, including location and scale**
- fit_loc_scale: estimation of location and scale when shape parameters are given

- nnlf: negative log likelihood function
- expect: calculate the expectation of a function against the pdf or pmf

Performance issues and cautionary remarks

The performance of the individual methods, in terms of speed, varies widely by distribution and method. The results of a method are obtained in one of two ways: either by explicit calculation, or by a generic algorithm that is independent of the specific distribution.

Explicit calculation, on the one hand, requires that the method is directly specified for the given distribution, either through analytic formulas or through special functions in `scipy.special` or `numpy.random` for `rvs`. These are usually relatively fast calculations.

The generic methods, on the other hand, are used if the distribution does not specify any explicit calculation. To define a distribution, only one of pdf or cdf is necessary; all other methods can be derived using numeric integration and root finding. However, these indirect methods can be `very` slow. As an example, `rgh = stats.gausshyper.rvs(0.5, 2, 2, 2, size=100)` creates random variables in a very indirect way and takes about 19 seconds for 100 random variables on my computer, while one million random variables from the standard normal or from the t distribution take just above one second.

Remaining issues

The distributions in `scipy.stats` have recently been corrected and improved and gained a considerable test suite; however, a few issues remain:

- The distributions have been tested over some range of parameters; however, in some corner ranges, a few incorrect results may remain.
- The maximum likelihood estimation in `fit` does not work with default starting parameters for all distributions and the user needs to supply good starting parameters. Also, for some distribution using a maximum likelihood estimator might inherently not be the best choice.

Building specific distributions

The next example shows how to build your own distributions. Further examples show the usage of the distributions and some statistical tests.

Making a continuous distribution, i.e., subclassing `rv_continuous`

Making continuous distributions is fairly simple.

```
>>> from scipy import stats
>>> class deterministic_gen(stats.rv_continuous):
...     def _cdf(self, x):
...         return np.where(x < 0, 0., 1.)
...     def _stats(self):
...         return 0., 0., 0., 0.
```

```
>>> deterministic = deterministic_gen(name="deterministic")
>>> deterministic.cdf(np.arange(-3, 3, 0.5))
array([ 0.,  0.,  0.,  0.,  0.,  0.,  1.,  1.,  1.,  1.,  1.,  1.])
```

Interestingly, the `pdf` is now computed automatically:

```
>>> deterministic.pdf(np.arange(-3, 3, 0.5))
array([  0.00000000e+00,   0.00000000e+00,   0.00000000e+00,
         0.00000000e+00,   0.00000000e+00,   0.00000000e+00,
         5.83333333e+04,   4.16333634e-12,   4.16333634e-12,
         4.16333634e-12,   4.16333634e-12,   4.16333634e-12])
```

Be aware of the performance issues mentioned in performance_issues_label. The computation of unspecified common methods can become very slow, since only general methods are called, which, by their very nature, cannot use any specific information about the distribution. Thus, as a cautionary example:

```
>>> from scipy.integrate import quad
>>> quad(deterministic.pdf, -1e-1, 1e-1)
(4.163336342344337e-13, 0.0)
```

But this is not correct: the integral over this pdf should be 1. Let's make the integration interval smaller:

```
>>> quad(deterministic.pdf, -1e-3, 1e-3)  # warning removed
(1.000076872229173, 0.0010625571718182458)
```

This looks better. However, the problem originates from the fact that the pdf is not specified in the class definition of the deterministic distribution.

Subclassing `rv_discrete`

In the following, we use `stats.rv_discrete` to generate a discrete distribution that has the probabilities of the truncated normal for the intervals centered around the integers.

General info

From the docstring of rv_discrete, `help(stats.rv_discrete)`,

> "You can construct an arbitrary discrete rv where P{X=xk} = pk by passing to the rv_discrete initialization method (through the values= keyword) a tuple of sequences (xk, pk) which describes only those values of X (xk) that occur with nonzero probability (pk)."

Additionally, there are some further requirements for this approach to work:

- The keyword `name` is required.
- The support points of the distribution xk have to be integers.
- The number of significant digits (decimals) needs to be specified.

In fact, if the last two requirements are not satisfied, an exception may be raised or the resulting numbers may be incorrect.

An example

Let's do the work. First:

```
>>> npoints = 20   # number of integer support points of the distribution minus 1
>>> npointsh = npoints // 2
>>> npointsf = float(npoints)
>>> nbound = 4   # bounds for the truncated normal
>>> normbound = (1+1/npointsf) * nbound   # actual bounds of truncated normal
>>> grid = np.arange(-npointsh, npointsh+2, 1)   # integer grid
>>> gridlimitsnorm = (grid-0.5) / npointsh * nbound   # bin limits for the truncnorm
>>> gridlimits = grid - 0.5   # used later in the analysis
>>> grid = grid[:-1]
```

```
>>> probs = np.diff(stats.truncnorm.cdf(gridlimitsnorm, -normbound, normbound))
>>> gridint = grid
```

And, finally, we can subclass `rv_discrete`:

```
>>> normdiscrete = stats.rv_discrete(values=(gridint,
...              np.round(probs, decimals=7)), name='normdiscrete')
```

Now that we have defined the distribution, we have access to all common methods of discrete distributions.

```
>>> print('mean = %6.4f, variance = %6.4f, skew = %6.4f, kurtosis = %6.4f' %
...       normdiscrete.stats(moments='mvsk'))
mean = -0.0000, variance = 6.3302, skew = 0.0000, kurtosis = -0.0076
```

```
>>> nd_std = np.sqrt(normdiscrete.stats(moments='v'))
```

Testing the implementation

Let's generate a random sample and compare observed frequencies with the probabilities.

```
>>> n_sample = 500
>>> rvs = normdiscrete.rvs(size=n_sample)
>>> f, l = np.histogram(rvs, bins=gridlimits)
>>> sfreq = np.vstack([gridint, f, probs*n_sample]).T
>>> print(sfreq)
[[-1.00000000e+01  0.00000000e+00  2.95019349e-02]  # random
 [-9.00000000e+00  0.00000000e+00  1.32294142e-01]
 [-8.00000000e+00  0.00000000e+00  5.06497902e-01]
 [-7.00000000e+00  2.00000000e+00  1.65568919e+00]
 [-6.00000000e+00  1.00000000e+00  4.62125309e+00]
 [-5.00000000e+00  9.00000000e+00  1.10137298e+01]
 [-4.00000000e+00  2.60000000e+01  2.24137683e+01]
 [-3.00000000e+00  3.70000000e+01  3.89503370e+01]
 [-2.00000000e+00  5.10000000e+01  5.78004747e+01]
 [-1.00000000e+00  7.10000000e+01  7.32455414e+01]
 [ 0.00000000e+00  7.40000000e+01  7.92618251e+01]
 [ 1.00000000e+00  8.90000000e+01  7.32455414e+01]
 [ 2.00000000e+00  5.50000000e+01  5.78004747e+01]
 [ 3.00000000e+00  5.00000000e+01  3.89503370e+01]
 [ 4.00000000e+00  1.70000000e+01  2.24137683e+01]
 [ 5.00000000e+00  1.10000000e+01  1.10137298e+01]
 [ 6.00000000e+00  4.00000000e+00  4.62125309e+00]
 [ 7.00000000e+00  3.00000000e+00  1.65568919e+00]
 [ 8.00000000e+00  0.00000000e+00  5.06497902e-01]
 [ 9.00000000e+00  0.00000000e+00  1.32294142e-01]
 [ 1.00000000e+01  0.00000000e+00  2.95019349e-02]]
```

Next, we can use a chi-squared test, `scipy.stats.chisquare`, to test the null hypothesis that the sample is distributed according to our norm-discrete distribution.

The test requires that there are a minimum number of observations in each bin. We combine the tail bins into larger bins so that they contain enough observations.

```
>>> f2 = np.hstack([f[:5].sum(), f[5:-5], f[-5:].sum()])
>>> p2 = np.hstack([probs[:5].sum(), probs[5:-5], probs[-5:].sum()])
>>> ch2, pval = stats.chisquare(f2, p2*n_sample)
```

```
>>> print('chisquare for normdiscrete: chi2 = %6.3f pvalue = %6.4f' % (ch2, pval))
chisquare for normdiscrete: chi2 = 12.466 pvalue = 0.4090  # random
```

Conceptually, the test statistic `chi2` is sensitive to deviations between the frequencies of observations and their expected frequencies under the null hypothesis. The p-value is the probability of drawing samples from the hypothesized distribution that would produce a statistic value more extreme than the one we observed. Our statistic value is not very high; in fact, there is a 40.9% chance that the statistic would be higher than 12.466 if we were to draw a sample of the same size from the discrete distribution defined by `p2`. Therefore, the test provides little evidence against the null hypothesis that the sample was drawn from our norm-discrete distribution.

Analysing one sample

Source: https://docs.scipy.org/doc/scipy/tutorial/stats/analysing_one_sample.html

First, we create some random variables. We set a seed so that in each run we get identical results to look at. As an example we take a sample from the Student t distribution:

```
>>> import numpy as np
>>> import scipy.stats as stats
>>> x = stats.t.rvs(10, size=1000)
```

Here, we set the required shape parameter of the t distribution, which in statistics corresponds to the degrees of freedom, to 10. Using size=1000 means that our sample consists of 1000 independently drawn (pseudo) random numbers. Since we did not specify the keyword arguments `loc` and `scale`, those are set to their default values zero and one.

Descriptive statistics

`x` is a numpy array, and we have direct access to all array methods, e.g.,

```
>>> print(x.min())   # equivalent to np.min(x)
-3.78975572422  # random
>>> print(x.max())   # equivalent to np.max(x)
5.26327732981  # random
>>> print(x.mean())  # equivalent to np.mean(x)
0.0140610663985  # random
>>> print(x.var())   # equivalent to np.var(x)
1.28899386208  # random
```

How do the sample properties compare to their theoretical counterparts?

```
>>> m, v, s, k = stats.t.stats(10, moments='mvsk')
>>> n, (smin, smax), sm, sv, ss, sk = stats.describe(x)
```

```
>>> sstr = '%-14s mean = %6.4f, variance = %6.4f, skew = %6.4f, kurtosis = %6.4f'
>>> print(sstr % ('distribution:', m, v, s ,k))
distribution:  mean = 0.0000, variance = 1.2500, skew = 0.0000, kurtosis = 1.0000  # random
>>> print(sstr % ('sample:', sm, sv, ss, sk))
sample:        mean = 0.0141, variance = 1.2903, skew = 0.2165, kurtosis = 1.0556  # random
```

Note: `stats.describe` uses the unbiased estimator for the variance, while np.var is the biased estimator.

For our sample, the sample statistics differ by a small amount from their theoretical counterparts.

T-test and KS-test

We can use the t-test to test whether the mean of our sample differs in a statistically significant way from the theoretical expectation.

```
>>> print('t-statistic = %6.3f pvalue = %6.4f' %  stats.ttest_1samp(x, m))
t-statistic =  0.391 pvalue = 0.6955  # random
```

The pvalue is 0.7, this means that with an alpha error of, for example, 10%, we cannot reject the hypothesis that the sample mean is equal to zero, the expectation of the standard t-distribution.

As an exercise, we can calculate our ttest also directly without using the provided function, which should give us the same answer, and so it does:

```
>>> tt = (sm-m)/np.sqrt(sv/float(n))  # t-statistic for mean
>>> pval = stats.t.sf(np.abs(tt), n-1)*2  # two-sided pvalue = Prob(abs(t)>tt)
>>> print('t-statistic = %6.3f pvalue = %6.4f' % (tt, pval))
t-statistic =  0.391 pvalue = 0.6955  # random
```

The Kolmogorov-Smirnov test can be used to test the hypothesis that the sample comes from the standard t-distribution

```
>>> print('KS-statistic D = %6.3f pvalue = %6.4f' % stats.kstest(x, 't', (10,)))
KS-statistic D =  0.016 pvalue = 0.9571  # random
```

Again, the p-value is high enough that we cannot reject the hypothesis that the random sample really is distributed according to the t-distribution. In real applications, we don't know what the underlying distribution is. If we perform the Kolmogorov-Smirnov test of our sample against the standard normal distribution, then we also cannot reject the hypothesis that our sample was generated by the normal distribution given that, in this example, the p-value is almost 40%.

```
>>> print('KS-statistic D = %6.3f pvalue = %6.4f' % stats.kstest(x, 'norm'))
KS-statistic D =  0.028 pvalue = 0.3918  # random
```

However, the standard normal distribution has a variance of 1, while our sample has a variance of 1.29. If we standardize our sample and test it against the normal distribution, then the p-value is again large enough that we cannot reject the hypothesis that the sample came from the normal distribution.

```
>>> d, pval = stats.kstest((x-x.mean())/x.std(), 'norm')
>>> print('KS-statistic D = %6.3f pvalue = %6.4f' % (d, pval))
KS-statistic D =  0.032 pvalue = 0.2397  # random
```

Note: The Kolmogorov-Smirnov test assumes that we test against a distribution with given parameters, since, in the last case, we estimated mean and variance, this assumption is violated and the distribution of the test statistic, on which the p-value is based, is not correct.

Tails of the distribution

Finally, we can check the upper tail of the distribution. We can use the percent point function ppf, which is the inverse of the cdf function, to obtain the critical values, or, more directly, we can use the inverse of the survival function

```
>>> crit01, crit05, crit10 = stats.t.ppf([1-0.01, 1-0.05, 1-0.10], 10)
>>> print('critical values from ppf at 1%%, 5%% and 10%% %8.4f %8.4f %8.4f' % (crit01, crit05, crit10))
critical values from ppf at 1%, 5% and 10%   2.7638   1.8125   1.3722
>>> print('critical values from isf at 1%%, 5%% and 10%% %8.4f %8.4f %8.4f' %
 ↪  tuple(stats.t.isf([0.01,0.05,0.10],10)))
critical values from isf at 1%, 5% and 10%   2.7638   1.8125   1.3722
```

```
>>> freq01 = np.sum(x>crit01) / float(n) * 100
>>> freq05 = np.sum(x>crit05) / float(n) * 100
>>> freq10 = np.sum(x>crit10) / float(n) * 100
>>> print('sample %%-frequency at 1%%, 5%% and 10%% tail %8.4f %8.4f %8.4f' % (freq01, freq05, freq10))
sample %-frequency at 1%, 5% and 10% tail   1.4000   5.8000  10.5000  # random
```

In all three cases, our sample has more weight in the top tail than the underlying distribution.

We can briefly check a larger sample to see if we get a closer match. In this case, the empirical frequency is quite close to the theoretical probability, but if we repeat this several times, the fluctuations are still pretty large.

```
>>> freq05l = np.sum(stats.t.rvs(10, size=10000) > crit05) / 10000.0 * 100
>>> print('larger sample %%-frequency at 5%% tail %8.4f' % freq05l)
larger sample %-frequency at 5% tail   4.8000  # random
```

We can also compare it with the tail of the normal distribution, which has less weight in the tails:

```
>>> print('tail prob. of normal at 1%%, 5%% and 10%% %8.4f %8.4f %8.4f' %
...       tuple(stats.norm.sf([crit01, crit05, crit10])*100))
tail prob. of normal at 1%, 5% and 10%   0.2857   3.4957   8.5003
```

The chisquare test can be used to test whether for a finite number of bins, the observed frequencies differ significantly from the probabilities of the hypothesized distribution.

```
>>> quantiles = [0.0, 0.01, 0.05, 0.1, 1-0.10, 1-0.05, 1-0.01, 1.0]
>>> crit = stats.t.ppf(quantiles, 10)
>>> crit
array([       -inf, -2.76376946, -1.81246112, -1.37218364,  1.37218364,
        1.81246112,  2.76376946,         inf])
>>> n_sample = x.size
>>> freqcount = np.histogram(x, bins=crit)[0]
>>> tprob = np.diff(quantiles)
>>> nprob = np.diff(stats.norm.cdf(crit))
>>> tch, tpval = stats.chisquare(freqcount, tprob*n_sample)
>>> nch, npval = stats.chisquare(freqcount, nprob*n_sample)
>>> print('chisquare for t:      chi2 = %6.2f pvalue = %6.4f' % (tch, tpval))
chisquare for t:      chi2 =  2.30 pvalue = 0.8901  # random
>>> print('chisquare for normal: chi2 = %6.2f pvalue = %6.4f' % (nch, npval))
chisquare for normal: chi2 = 64.60 pvalue = 0.0000  # random
```

We see that the standard normal distribution is clearly rejected, while the standard t-distribution cannot be rejected. Since the variance of our sample differs from both standard distributions, we can again redo the test taking the estimate for scale and location into account.

The fit method of the distributions can be used to estimate the parameters of the distribution, and the test is repeated using probabilities of the estimated distribution.

```
>>> tdof, tloc, tscale = stats.t.fit(x)
>>> nloc, nscale = stats.norm.fit(x)
>>> tprob = np.diff(stats.t.cdf(crit, tdof, loc=tloc, scale=tscale))
>>> nprob = np.diff(stats.norm.cdf(crit, loc=nloc, scale=nscale))
>>> tch, tpval = stats.chisquare(freqcount, tprob*n_sample)
>>> nch, npval = stats.chisquare(freqcount, nprob*n_sample)
>>> print('chisquare for t:      chi2 = %6.2f pvalue = %6.4f' % (tch, tpval))
chisquare for t:      chi2 =  1.58 pvalue = 0.9542  # random
>>> print('chisquare for normal: chi2 = %6.2f pvalue = %6.4f' % (nch, npval))
chisquare for normal: chi2 = 11.08 pvalue = 0.0858  # random
```

Taking account of the estimated parameters, we can still reject the hypothesis that our sample came from a normal distribution (at the 5% level), but again, with a p-value of 0.95, we cannot reject the t-distribution.

Special tests for normal distributions

Since the normal distribution is the most common distribution in statistics, there are several additional functions available to test whether a sample could have been drawn from a normal

distribution.

First, we can test if skew and kurtosis of our sample differ significantly from those of a normal distribution:

```
>>> print('normal skewtest teststat = %6.3f pvalue = %6.4f' % stats.skewtest(x))
normal skewtest teststat =  2.785 pvalue = 0.0054  # random
>>> print('normal kurtosistest teststat = %6.3f pvalue = %6.4f' % stats.kurtosistest(x))
normal kurtosistest teststat =  4.757 pvalue = 0.0000  # random
```

These two tests are combined in the normality test

```
>>> print('normaltest teststat = %6.3f pvalue = %6.4f' % stats.normaltest(x))
normaltest teststat = 30.379 pvalue = 0.0000  # random
```

In all three tests, the p-values are very low and we can reject the hypothesis that our sample has skew and kurtosis of the normal distribution.

Since skew and kurtosis of our sample are based on central moments, we get exactly the same results if we test the standardized sample:

```
>>> print('normaltest teststat = %6.3f pvalue = %6.4f' %
...       stats.normaltest((x-x.mean())/x.std()))
normaltest teststat = 30.379 pvalue = 0.0000  # random
```

Because normality is rejected so strongly, we can check whether the normaltest gives reasonable results for other cases:

```
>>> print('normaltest teststat = %6.3f pvalue = %6.4f' %
...       stats.normaltest(stats.t.rvs(10, size=100)))
normaltest teststat =  4.698 pvalue = 0.0955  # random
>>> print('normaltest teststat = %6.3f pvalue = %6.4f' %
...              stats.normaltest(stats.norm.rvs(size=1000)))
normaltest teststat =  0.613 pvalue = 0.7361  # random
```

When testing for normality of a small sample of t-distributed observations and a large sample of normal-distributed observations, then in neither case can we reject the null hypothesis that the sample comes from a normal distribution. In the first case, this is because the test is not powerful enough to distinguish a t-distributed and a normally distributed random variable in a small sample.

Kernel density estimation

Source: https://docs.scipy.org/doc/scipy/tutorial/stats/kernel_density_estimation.html

A common task in statistics is to estimate the probability density function (PDF) of a random variable from a set of data samples. This task is called density estimation. The most well-known tool to do this is the histogram. A histogram is a useful tool for visualization (mainly because everyone understands it), but doesn't use the available data very efficiently. Kernel density estimation (KDE) is a more efficient tool for the same task. The `scipy.stats.gaussian_kde` estimator can be used to estimate the PDF of univariate as well as multivariate data. It works best if the data is unimodal.

Univariate estimation

We start with a minimal amount of data in order to see how `scipy.stats.gaussian_kde` works and what the different options for bandwidth selection do. The data sampled from the PDF are shown as blue dashes at the bottom of the figure (this is called a rug plot):

```
>>> import numpy as np
>>> from scipy import stats
>>> import matplotlib.pyplot as plt

>>> x1 = np.array([-7, -5, 1, 4, 5], dtype=np.float64)
>>> kde1 = stats.gaussian_kde(x1)
>>> kde2 = stats.gaussian_kde(x1, bw_method='silverman')

>>> fig = plt.figure()
>>> ax = fig.add_subplot(111)

>>> ax.plot(x1, np.zeros(x1.shape), 'b+', ms=20)  # rug plot
>>> x_eval = np.linspace(-10, 10, num=200)
>>> ax.plot(x_eval, kde1(x_eval), 'k-', label="Scott's Rule")
>>> ax.plot(x_eval, kde2(x_eval), 'r-', label="Silverman's Rule")

>>> plt.show()
```

We see that there is very little difference between Scott's Rule and Silverman's Rule, and that the bandwidth selection with a limited amount of data is probably a bit too wide. We can define our own bandwidth function to get a less smoothed-out result.

```
>>> def my_kde_bandwidth(obj, fac=1./5):
...     """We use Scott's Rule, multiplied by a constant factor."""
...     return np.power(obj.n, -1./(obj.d+4)) * fac
```

```
>>> fig = plt.figure()
>>> ax = fig.add_subplot(111)
```

```
>>> ax.plot(x1, np.zeros(x1.shape), 'b+', ms=20)  # rug plot
>>> kde3 = stats.gaussian_kde(x1, bw_method=my_kde_bandwidth)
>>> ax.plot(x_eval, kde3(x_eval), 'g-', label="With smaller BW")
```

```
>>> plt.show()
```

We see that if we set the bandwidth to be very narrow, the obtained estimate for the probability

density function (PDF) is simply the sum of Gaussians around each data point.

We now take a more realistic example and look at the difference between the two available bandwidth selection rules. Those rules are known to work well for (close to) normal distributions, but even for unimodal distributions that are quite strongly non-normal they work reasonably well. As a non-normal distribution we take a Student's T distribution with 5 degrees of freedom.

We now take a look at a bimodal distribution with one wider and one narrower Gaussian feature. We expect that this will be a more difficult density to approximate, due to the different bandwidths required to accurately resolve each feature.

```
>>> from functools import partial
```

```
>>> loc1, scale1, size1 = (-2, 1, 175)
>>> loc2, scale2, size2 = (2, 0.2, 50)
>>> x2 = np.concatenate([np.random.normal(loc=loc1, scale=scale1, size=size1),
...                      np.random.normal(loc=loc2, scale=scale2, size=size2)])
```

```
>>> x_eval = np.linspace(x2.min() - 1, x2.max() + 1, 500)
```

```
>>> kde = stats.gaussian_kde(x2)
>>> kde2 = stats.gaussian_kde(x2, bw_method='silverman')
>>> kde3 = stats.gaussian_kde(x2, bw_method=partial(my_kde_bandwidth, fac=0.2))
>>> kde4 = stats.gaussian_kde(x2, bw_method=partial(my_kde_bandwidth, fac=0.5))
```

```
>>> pdf = stats.norm.pdf
>>> bimodal_pdf = pdf(x_eval, loc=loc1, scale=scale1) * float(size1) / x2.size + \
...               pdf(x_eval, loc=loc2, scale=scale2) * float(size2) / x2.size
```

```
>>> fig = plt.figure(figsize=(8, 6))
>>> ax = fig.add_subplot(111)
```

```
>>> ax.plot(x2, np.zeros(x2.shape), 'b+', ms=12)
>>> ax.plot(x_eval, kde(x_eval), 'k-', label="Scott's Rule")
>>> ax.plot(x_eval, kde2(x_eval), 'b-', label="Silverman's Rule")
>>> ax.plot(x_eval, kde3(x_eval), 'g-', label="Scott * 0.2")
>>> ax.plot(x_eval, kde4(x_eval), 'c-', label="Scott * 0.5")
>>> ax.plot(x_eval, bimodal_pdf, 'r--', label="Actual PDF")
```

```
>>> ax.set_xlim([x_eval.min(), x_eval.max()])
>>> ax.legend(loc=2)
>>> ax.set_xlabel('x')
>>> ax.set_ylabel('Density')
>>> plt.show()
```

As expected, the KDE is not as close to the true PDF as we would like due to the different characteristic sizes of the two features of the bimodal distribution. By halving the default bandwidth (`Scott * 0.5`), we can do somewhat better, while using a bandwidth one-fifth that of the default does not smooth enough. What we really need, though, in this case, is a non-uniform (adaptive) bandwidth.

Multivariate estimation

With `scipy.stats.gaussian_kde` we can perform multivariate, as well as univariate estimation. We demonstrate the bivariate case. First, we generate some random data with a model in which the two variates are correlated.

```
>>> def measure(n):
...     """Measurement model, return two coupled measurements."""
...     m1 = np.random.normal(size=n)
...     m2 = np.random.normal(scale=0.5, size=n)
...     return m1+m2, m1-m2
```

```
>>> m1, m2 = measure(2000)
>>> xmin = m1.min()
>>> xmax = m1.max()
>>> ymin = m2.min()
>>> ymax = m2.max()
```

Then we apply the KDE to the data:

```
>>> X, Y = np.mgrid[xmin:xmax:100j, ymin:ymax:100j]
>>> positions = np.vstack([X.ravel(), Y.ravel()])
>>> values = np.vstack([m1, m2])
>>> kernel = stats.gaussian_kde(values)
>>> Z = np.reshape(kernel.evaluate(positions).T, X.shape)
```

Finally, we plot the estimated bivariate distribution as a colormap and plot the individual data points on top.

```
>>> fig = plt.figure(figsize=(8, 6))
>>> ax = fig.add_subplot(111)
```

```
>>> ax.imshow(np.rot90(Z), cmap=plt.cm.gist_earth_r,
...           extent=[xmin, xmax, ymin, ymax])
>>> ax.plot(m1, m2, 'k.', markersize=2)
```

```
>>> ax.set_xlim([xmin, xmax])
>>> ax.set_ylim([ymin, ymax])
```

```
>>> plt.show()
```

Integration (scipy.integrate)

Source: *https://docs.scipy.org/doc/scipy/tutorial/integrate.html*

The `scipy.integrate` sub-package provides several integration techniques including an ordinary differential equation integrator.

General integration (quad)

The function `quad` is provided to integrate a function of one variable between two points. The points can be $\pm\infty$ ($\pm$ `inf`) to indicate infinite limits. For example, suppose you wish to integrate a bessel function `jv(2.5, x)` along the interval $[0, 4.5]$.

$$I = \int_0^{4.5} J_{2.5}(x)\, dx.$$

This could be computed using `quad`:

```
>>> import scipy.integrate as integrate
>>> import scipy.special as special
>>> result = integrate.quad(lambda x: special.jv(2.5,x), 0, 4.5)
>>> result
(1.1178179380783249, 7.8663172481899801e-09)
```

```
>>> from numpy import sqrt, sin, cos, pi
>>> I = sqrt(2/pi)*(18.0/27*sqrt(2)*cos(4.5) - 4.0/27*sqrt(2)*sin(4.5) +
...                 sqrt(2*pi) * special.fresnel(3/sqrt(pi))[0])
>>> I
1.117817938088701
```

```
>>> print(abs(result[0]-I))
1.03761443881e-11
```

The first argument to quad is a "callable" Python object (i.e., a function, method, or class instance). Notice the use of a lambda-function in this case as the argument. The next two arguments are the limits of integration. The return value is a tuple, with the first element holding the estimated value of the integral and the second element holding an estimate of the absolute integration error. Notice, that in this case, the true value of this integral is

$$I = \sqrt{\frac{2}{\pi}}\left(\frac{18}{27}\sqrt{2}\cos(4.5) - \frac{4}{27}\sqrt{2}\sin(4.5) + \sqrt{2\pi}\,\mathrm{Si}\left(\frac{3}{\sqrt{\pi}}\right)\right),$$

where

$$\mathrm{Si}(x) = \int_0^x \sin\left(\frac{\pi}{2}t^2\right)\, dt.$$

is the Fresnel sine integral. Note that the numerically-computed integral is within 1.04×10^{-11} of the exact result --- well below the reported error estimate.

If the function to integrate takes additional parameters, they can be provided in the `args` argument. Suppose that the following integral shall be calculated:

$$I(a,b) = \int_0^1 ax^2 + b\,dx.$$

This integral can be evaluated by using the following code:

```
>>> from scipy.integrate import quad
>>> def integrand(x, a, b):
...     return a*x**2 + b
...
>>> a = 2
>>> b = 1
>>> I = quad(integrand, 0, 1, args=(a,b))
>>> I
(1.6666666666666667, 1.8503717077085944e-14)
```

Infinite inputs are also allowed in `quad` by using $\pm$ `inf` as one of the arguments. For example, suppose that a numerical value for the exponential integral:

$$E_n(x) = \int_1^\infty \frac{e^{-xt}}{t^n}\,dt.$$

is desired (and the fact that this integral can be computed as `special.expn(n,x)` is forgotten). The functionality of the function `special.expn` can be replicated by defining a new function `vec_expint` based on the routine `quad`:

```
>>> from scipy.integrate import quad
>>> import numpy as np
>>> def integrand(t, n, x):
...     return np.exp(-x*t) / t**n
...
```

```
>>> def expint(n, x):
...     return quad(integrand, 1, np.inf, args=(n, x))[0]
...
```

```
>>> vec_expint = np.vectorize(expint)
```

```
>>> vec_expint(3, np.arange(1.0, 4.0, 0.5))
array([ 0.1097,  0.0567,  0.0301,  0.0163,  0.0089,  0.0049])
>>> import scipy.special as special
>>> special.expn(3, np.arange(1.0,4.0,0.5))
array([ 0.1097,  0.0567,  0.0301,  0.0163,  0.0089,  0.0049])
```

The function which is integrated can even use the quad argument (though the error bound may underestimate the error due to possible numerical error in the integrand from the use of `quad`). The integral in this case is

$$I_n = \int_0^\infty \int_1^\infty \frac{e^{-xt}}{t^n}\,dt\,dx = \frac{1}{n}.$$

```
>>> result = quad(lambda x: expint(3, x), 0, np.inf)
>>> print(result)
(0.33333333324560266, 2.8548934485373678e-09)
```

```
>>> I3 = 1.0/3.0
>>> print(I3)
0.333333333333
```

```
>>> print(I3 - result[0])
8.77306560731e-11
```

This last example shows that multiple integration can be handled using repeated calls to `quad`.

Warning
Numerical integration algorithms sample the integrand at a finite number of points. Consequently, they cannot guarantee accurate results (or accuracy estimates) for arbitrary integrands and limits of integration. Consider the Gaussian integral, for example:

```
>>> def gaussian(x):
...     return np.exp(-x**2)
>>> res = integrate.quad(gaussian, -np.inf, np.inf)
>>> res
(1.7724538509055159, 1.4202636756659625e-08)
>>> np.allclose(res[0], np.sqrt(np.pi))  # compare against theoretical result
True
```

Since the integrand is nearly zero except near the origin, we would expect large but finite limits of integration to yield the same result. However:

```
>>> integrate.quad(gaussian, -10000, 10000)
(1.975190562208035e-203, 0.0)
```

This happens because the adaptive quadrature routine implemented in `quad`, while working as designed, does not notice the small, important part of the function within such a large, finite interval. For best results, consider using integration limits that tightly surround the important part of the integrand.

```
>>> integrate.quad(gaussian, -15, 15)
(1.772453850905516, 8.476526631214648e-11)
```

Integrands with several important regions can be broken into pieces as necessary.

General multiple integration (`dblquad`, `tplquad`, `nquad`)

The mechanics for double and triple integration have been wrapped up into the functions `dblquad` and `tplquad`. These functions take the function to integrate and four, or six arguments, respectively. The limits of all inner integrals need to be defined as functions.

An example of using double integration to compute several values of I_n is shown below:

```
>>> from scipy.integrate import quad, dblquad
>>> def I(n):
...     return dblquad(lambda t, x: np.exp(-x*t)/t**n, 0, np.inf, lambda x: 1, lambda x: np.inf)
...
```

```
>>> print(I(4))
(0.2500000000043577, 1.29830334693681e-08)
>>> print(I(3))
(0.33333333325010883, 1.3888461883425516e-08)
>>> print(I(2))
(0.4999999999985751, 1.3894083651858995e-08)
```

As example for non-constant limits consider the integral

$$I = \int_{y=0}^{1/2} \int_{x=0}^{1-2y} xy \, dx \, dy = \frac{1}{96}.$$

This integral can be evaluated using the expression below (Note the use of the non-constant lambda functions for the upper limit of the inner integral):

```
>>> from scipy.integrate import dblquad
>>> area = dblquad(lambda x, y: x*y, 0, 0.5, lambda x: 0, lambda x: 1-2*x)
>>> area
(0.010416666666666668, 1.1564823173178715e-16)
```

For n-fold integration, scipy provides the function `nquad`. The integration bounds are an iterable object: either a list of constant bounds, or a list of functions for the non-constant integration bounds. The order of integration (and therefore the bounds) is from the innermost integral to the outermost one.

The integral from above

$$I_n = \int_0^{\infty} \int_1^{\infty} \frac{e^{-xt}}{t^n} \, dt \, dx = \frac{1}{n}$$

can be calculated as

```
>>> from scipy import integrate
>>> N = 5
>>> def f(t, x):
...     return np.exp(-x*t) / t**N
...
>>> integrate.nquad(f, [[1, np.inf],[0, np.inf]])
(0.20000000000002294, 1.2239614263187945e-08)
```

Note that the order of arguments for `f` must match the order of the integration bounds; i.e., the inner integral with respect to t is on the interval $[1, \infty]$ and the outer integral with respect to x is on the interval $[0, \infty]$.

Non-constant integration bounds can be treated in a similar manner; the example from above

$$I = \int_{y=0}^{1/2} \int_{x=0}^{1-2y} xy \, dx \, dy = \frac{1}{96}.$$

can be evaluated by means of

```
>>> from scipy import integrate
>>> def f(x, y):
...     return x*y
...
>>> def bounds_y():
...     return [0, 0.5]
...
>>> def bounds_x(y):
...     return [0, 1-2*y]
...
>>> integrate.nquad(f, [bounds_x, bounds_y])
(0.010416666666666668, 4.101620128472366e-16)
```

which is the same result as before.

Gaussian quadrature

`fixed_quad` performs fixed-order Gaussian quadrature over a fixed interval. This function uses the collection of orthogonal polynomials provided by `scipy.special`, which can calculate the roots and quadrature weights of a large variety of orthogonal polynomials (the polynomials themselves are available as special functions returning instances of the polynomial class --- e.g., `special.legendre`).

Integrating using Samples

If the samples are equally-spaced and the number of samples available is $2^k + 1$ for some integer k, then Romberg `romb` integration can be used to obtain high-precision estimates of the integral using the available samples. Romberg integration uses the trapezoid rule at step-sizes related by a power of two and then performs Richardson extrapolation on these estimates to approximate the integral with a higher degree of accuracy.

In case of arbitrary spaced samples, the two functions `trapezoid` and `simpson` are available. They are using Newton-Coates formulas of order 1 and 2 respectively to perform integration. The trapezoidal rule approximates the function as a straight line between adjacent points, while Simpson's rule approximates the function between three adjacent points as a parabola.

For an odd number of samples that are equally spaced Simpson's rule is exact if the function is a polynomial of order 3 or less. If the samples are not equally spaced, then the result is exact only if the function is a polynomial of order 2 or less.

```
>>> import numpy as np
>>> def f1(x):
...     return x**2
...
>>> def f2(x):
...     return x**3
...
>>> x = np.array([1,3,4])
>>> y1 = f1(x)
>>> from scipy import integrate
>>> I1 = integrate.simpson(y1, x=x)
>>> print(I1)
21.0
```

This corresponds exactly to

$$\int_1^4 x^2 \, dx = 21,$$

whereas integrating the second function

```
>>> y2 = f2(x)
>>> I2 = integrate.simpson(y2, x=x)
>>> print(I2)
61.5
```

does not correspond to

$$\int_1^4 x^3 \, dx = 63.75$$

because the order of the polynomial in f2 is larger than two.

Faster integration using low-level callback functions

A user desiring reduced integration times may pass a C function pointer through `scipy.LowLevelCallable` to `quad`, `dblquad`, `tplquad` or `nquad` and it will be integrated and return a result in Python. The performance increase here arises from two factors. The primary improvement is faster function evaluation, which is provided by compilation of the function itself. Additionally we have a speedup provided by the removal of function calls between C and Python in `quad`. This method may provide a speed improvements of ~2x for trivial functions such as sine but can produce a much more noticeable improvements (10x+) for more complex functions. This feature then, is geared towards a user with numerically intensive integrations willing to write a little C to reduce computation time significantly.

The approach can be used, for example, via `ctypes` in a few simple steps:

1.) Write an integrand function in C with the function signature `double f(int n, double *x, void *user_data)`, where `x` is an array containing the point the function f is evaluated at, and `user_data` to arbitrary additional data you want to provide.

```
/* testlib.c */
double f(int n, double *x, void *user_data) {
    double c = *(double *)user_data;
    return c + x[0] - x[1] * x[2]; /* corresponds to c + x - y * z */
}
```

2.) Now compile this file to a shared/dynamic library (a quick search will help with this as it is OS-dependent). The user must link any math libraries, etc., used. On linux this looks like:

```
$ gcc -shared -fPIC -o testlib.so testlib.c
```

The output library will be referred to as `testlib.so`, but it may have a different file extension. A library has now been created that can be loaded into Python with `ctypes`.

3.) Load shared library into Python using `ctypes` and set `restypes` and `argtypes` - this allows SciPy to interpret the function correctly:

```
import os, ctypes
from scipy import integrate, LowLevelCallable

lib = ctypes.CDLL(os.path.abspath('testlib.so'))
lib.f.restype = ctypes.c_double
lib.f.argtypes = (ctypes.c_int, ctypes.POINTER(ctypes.c_double), ctypes.c_void_p)

c = ctypes.c_double(1.0)
user_data = ctypes.cast(ctypes.pointer(c), ctypes.c_void_p)

func = LowLevelCallable(lib.f, user_data)
```

The last `void *user_data` in the function is optional and can be omitted (both in the C function and ctypes argtypes) if not needed. Note that the coordinates are passed in as an array of doubles rather than a separate argument.

4.) Now integrate the library function as normally, here using `nquad`:

```
>>> integrate.nquad(func, [[0, 10], [-10, 0], [-1, 1]])
(1200.0, 1.1102230246251565e-11)
```

The Python tuple is returned as expected in a reduced amount of time. All optional parameters can be used with this method including specifying singularities, infinite bounds, etc.

Ordinary differential equations (`solve_ivp`)

Integrating a set of ordinary differential equations (ODEs) given initial conditions is another useful example. The function `solve_ivp` is available in SciPy for integrating a first-order vector differential equation:

$$\frac{d\mathbf{y}}{dt} = \mathbf{f}\left(\mathbf{y}, t\right),$$

given initial conditions $\mathbf{y}\left(0\right) = \mathbf{y}_0$, where $\mathbf{y}$ is a length N vector and $\mathbf{f}$ is a mapping from $\mathbb{R}^N$ to $\mathbb{R}^N$. A higher-order ordinary differential equation can always be reduced to a differential equation of this type by introducing intermediate derivatives into the $\mathbf{y}$ vector.

For example, suppose it is desired to find the solution to the following second-order differential equation:

$$\frac{d^2w}{dz^2} - zw(z) = 0$$

with initial conditions $w\left(0\right) = \frac{1}{\sqrt[3]{3^2}\Gamma\left(\frac{2}{3}\right)}$ and $\frac{dw}{dz}\Big|_{z=0} = -\frac{1}{\sqrt[3]{3}\Gamma\left(\frac{1}{3}\right)}$. It is known that the solution to this differential equation with these boundary conditions is the Airy function

$$w = \mathrm{Ai}\left(z\right),$$

which gives a means to check the integrator using `special.airy`.

First, convert this ODE into standard form by setting $\mathbf{y} = \left[\frac{dw}{dz}, w\right]$ and $t = z$. Thus, the differential equation becomes

$$\frac{d\mathbf{y}}{dt} = \begin{bmatrix} ty_1 \\ y_0 \end{bmatrix} = \begin{bmatrix} 0 & t \\ 1 & 0 \end{bmatrix} \begin{bmatrix} y_0 \\ y_1 \end{bmatrix} = \begin{bmatrix} 0 & t \\ 1 & 0 \end{bmatrix} \mathbf{y}.$$

In other words,

$$\mathbf{f}\left(\mathbf{y}, t\right) = \mathbf{A}\left(t\right)\mathbf{y}.$$

As an interesting reminder, if $\mathbf{A}\left(t\right)$ commutes with $\int_0^t \mathbf{A}\left(\tau\right)\,d\tau$ under matrix multiplication, then this linear differential equation has an exact solution using the matrix exponential:

$$\mathbf{y}\left(t\right) = \exp\left(\int_0^t \mathbf{A}\left(\tau\right)d\tau\right)\mathbf{y}\left(0\right),$$

However, in this case, $\mathbf{A}\left(t\right)$ and its integral do not commute.

This differential equation can be solved using the function `solve_ivp`. It requires the derivative, *fprime*, the time span `[t_start, t_end]` and the initial conditions vector, *y0*, as input arguments and returns an object whose *y* field is an array with consecutive solution values as columns. The initial conditions are therefore given in the first output column.

```
>>> from scipy.integrate import solve_ivp
>>> from scipy.special import gamma, airy
>>> y1_0 = +1 / 3**(2/3) / gamma(2/3)
>>> y0_0 = -1 / 3**(1/3) / gamma(1/3)
>>> y0 = [y0_0, y1_0]
>>> def func(t, y):
...     return [t*y[1],y[0]]
...
>>> t_span = [0, 4]
>>> sol1 = solve_ivp(func, t_span, y0)
>>> print("sol1.t: {}".format(sol1.t))
sol1.t:    [0.         0.10097672 1.04643602 1.91060117 2.49872472 3.08684827
 3.62692846 4.        ]
```

As it can be seen `solve_ivp` determines its time steps automatically if not specified otherwise. To compare the solution of `solve_ivp` with the `airy` function the time vector created by `solve_ivp` is passed to the `airy` function.

```
>>> print("sol1.y[1]: {}".format(sol1.y[1]))
sol1.y[1]: [0.35502805 0.328952   0.12801343 0.04008508 0.01601291 0.00623879
 0.00356316 0.00405982]
>>> print("airy(sol.t)[0]:  {}".format(airy(sol1.t)[0]))
airy(sol.t)[0]: [0.35502805 0.328952   0.12804768 0.03995804 0.01575943 0.00562799
 0.00201689 0.00095156]
```

The solution of `solve_ivp` with its standard parameters shows a big deviation to the airy function. To minimize this deviation, relative and absolute tolerances can be used.

```
>>> rtol, atol = (1e-8, 1e-8)
>>> sol2 = solve_ivp(func, t_span, y0, rtol=rtol, atol=atol)
>>> print("sol2.y[1][::6]: {}".format(sol2.y[1][0::6]))
sol2.y[1][::6]: [0.35502805 0.19145234 0.06368989 0.0205917  0.00554734 0.00106409]
>>> print("airy(sol2.t)[0][::6]: {}".format(airy(sol2.t)[0][::6]))
airy(sol2.t)[0][::6]: [0.35502805 0.19145234 0.06368989 0.0205917  0.00554733 0.00106406]
```

To specify user defined time points for the solution of `solve_ivp`, `solve_ivp` offers two possibilities that can also be used complementarily. By passing the `t_eval` option to the function call `solve_ivp` returns the solutions of these time points of `t_eval` in its output.

```
>>> import numpy as np
>>> t = np.linspace(0, 4, 100)
>>> sol3 = solve_ivp(func, t_span, y0, t_eval=t)
```

If the jacobian matrix of function is known, it can be passed to the `solve_ivp` to achieve better results. Please be aware however that the default integration method `RK45` does not support jacobian matrices and thereby another integration method has to be chosen. One of the integration methods that support a jacobian matrix is the for example the `Radau` method of following example.

```
>>> def gradient(t, y):
...     return [[0,t], [1,0]]
>>> sol4 = solve_ivp(func, t_span, y0, method='Radau', jac=gradient)
```

Solving a system with a banded Jacobian matrix

`odeint` can be told that the Jacobian is *banded*. For a large system of differential equations that are known to be stiff, this can improve performance significantly.

As an example, we'll solve the 1-D Gray-Scott partial differential equations using the method

of lines [MOL]. The Gray-Scott equations for the functions $u(x,t)$ and $v(x,t)$ on the interval $x \in [0, L]$ are

$$\frac{\partial u}{\partial t} = D_u \frac{\partial^2 u}{\partial x^2} - uv^2 + f(1-u)$$
$$\frac{\partial v}{\partial t} = D_v \frac{\partial^2 v}{\partial x^2} + uv^2 - (f+k)v$$

where D_u and D_v are the diffusion coefficients of the components u and v, respectively, and f and k are constants. (For more information about the system, see http://groups.csail.mit.edu/mac/projects/amorphous/GrayScott/)

We'll assume Neumann (i.e., "no flux") boundary conditions:

$$\frac{\partial u}{\partial x}(0,t) = 0, \quad \frac{\partial v}{\partial x}(0,t) = 0, \quad \frac{\partial u}{\partial x}(L,t) = 0, \quad \frac{\partial v}{\partial x}(L,t) = 0$$

To apply the method of lines, we discretize the x variable by defining the uniformly spaced grid of N points $\{x_0, x_1, \ldots, x_{N-1}\}$, with $x_0 = 0$ and $x_{N-1} = L$. We define $u_j(t) \equiv u(x_k, t)$ and $v_j(t) \equiv v(x_k, t)$, and replace the x derivatives with finite differences. That is,

$$\frac{\partial^2 u}{\partial x^2}(x_j, t) \to \frac{u_{j-1}(t) - 2u_j(t) + u_{j+1}(t)}{(\Delta x)^2}$$

We then have a system of $2N$ ordinary differential equations:

$$\frac{du_j}{dt} = \frac{D_u}{(\Delta x)^2}\left(u_{j-1} - 2u_j + u_{j+1}\right) - u_j v_j^2 + f(1-u_j)$$
$$\frac{dv_j}{dt} = \frac{D_v}{(\Delta x)^2}\left(v_{j-1} - 2v_j + v_{j+1}\right) + u_j v_j^2 - (f+k)v_j$$

For convenience, the (t) arguments have been dropped.

To enforce the boundary conditions, we introduce "ghost" points x_{-1} and x_N, and define $u_{-1}(t) \equiv u_1(t)$, $u_N(t) \equiv u_{N-2}(t)$; $v_{-1}(t)$ and $v_N(t)$ are defined analogously.

Then

$$\frac{du_0}{dt} = \frac{D_u}{(\Delta x)^2}\left(2u_1 - 2u_0\right) - u_0 v_0^2 + f(1-u_0)$$
$$\frac{dv_0}{dt} = \frac{D_v}{(\Delta x)^2}\left(2v_1 - 2v_0\right) + u_0 v_0^2 - (f+k)v_0$$

and

$$\frac{du_{N-1}}{dt} = \frac{D_u}{(\Delta x)^2}\left(2u_{N-2} - 2u_{N-1}\right) - u_{N-1} v_{N-1}^2 + f(1-u_{N-1})$$
$$\frac{dv_{N-1}}{dt} = \frac{D_v}{(\Delta x)^2}\left(2v_{N-2} - 2v_{N-1}\right) + u_{N-1} v_{N-1}^2 - (f+k)v_{N-1}$$

Our complete system of $2N$ ordinary differential equations is `interior` for $k = 1, 2, ..., N-2$, along with `boundary0` and `boundaryL`.

We can now starting implementing this system in code. We must combine $\{u_k\}$ and $\{v_k\}$ into a single vector of length $2N$. The two obvious choices are $\{u_0, u_1, ..., u_{N-1}, v_0, v_1, ..., v_{N-1}\}$ and $\{u_0, v_0, u_1, v_1, ..., u_{N-1}, v_{N-1}\}$. Mathematically, it does not matter, but the choice affects how efficiently `odeint` can solve the system. The reason is in how the order affects the pattern of the nonzero elements of the Jacobian matrix.

When the variables are ordered as $\{u_0, u_1, ..., u_{N-1}, v_0, v_1, ..., v_{N-1}\}$, the pattern of nonzero elements of the Jacobian matrix is

$$
\begin{array}{cccccccccccccccc}
* & * & 0 & 0 & 0 & 0 & 0 & * & 0 & 0 & 0 & 0 & 0 & 0 \\
* & * & * & 0 & 0 & 0 & 0 & 0 & * & 0 & 0 & 0 & 0 & 0 \\
0 & * & * & * & 0 & 0 & 0 & 0 & 0 & * & 0 & 0 & 0 & 0 \\
0 & 0 & * & * & * & 0 & 0 & 0 & 0 & 0 & * & 0 & 0 & 0 \\
0 & 0 & 0 & * & * & * & 0 & 0 & 0 & 0 & 0 & * & 0 & 0 \\
0 & 0 & 0 & 0 & * & * & * & 0 & 0 & 0 & 0 & 0 & * & 0 \\
0 & 0 & 0 & 0 & 0 & * & * & 0 & 0 & 0 & 0 & 0 & 0 & * \\
* & 0 & 0 & 0 & 0 & 0 & 0 & * & * & 0 & 0 & 0 & 0 & 0 \\
0 & * & 0 & 0 & 0 & 0 & 0 & * & * & * & 0 & 0 & 0 & 0 \\
0 & 0 & * & 0 & 0 & 0 & 0 & 0 & * & * & * & 0 & 0 & 0 \\
0 & 0 & 0 & * & 0 & 0 & 0 & 0 & 0 & * & * & * & 0 & 0 \\
0 & 0 & 0 & 0 & * & 0 & 0 & 0 & 0 & 0 & * & * & * & 0 \\
0 & 0 & 0 & 0 & 0 & * & 0 & 0 & 0 & 0 & 0 & * & * & * \\
0 & 0 & 0 & 0 & 0 & 0 & * & 0 & 0 & 0 & 0 &) & * & *
\end{array}
$$

The Jacobian pattern with variables interleaved as $\{u_0, v_0, u_1, v_1, ..., u_{N-1}, v_{N-1}\}$ is

$$
\begin{array}{cccccccccccccccc}
* & * & * & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
* & * & 0 & * & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
* & 0 & * & * & * & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & * & * & * & 0 & * & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & * & 0 & * & * & * & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & * & * & * & 0 & * & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & * & 0 & * & * & * & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & * & * & * & 0 & * & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & * & 0 & * & * & * & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & * & * & * & 0 & * & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & * & 0 & * & * & * & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & * & * & * & 0 & * \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & * & 0 & * & * \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & * & * & *
\end{array}
$$

In both cases, there are just five nontrivial diagonals, but when the variables are interleaved, the bandwidth is much smaller. That is, the main diagonal and the two diagonals immediately above and the two immediately below the main diagonal are the nonzero diagonals. This is important, because the inputs `mu` and `ml` of `odeint` are the upper and lower bandwidths of the Jacobian matrix. When the variables are interleaved, `mu` and `ml` are 2. When the variables are stacked with $\{v_k\}$ following $\{u_k\}$, the upper and lower bandwidths are N.

With that decision made, we can write the function that implements the system of differential equations.

First, we define the functions for the source and reaction terms of the system:

```
def G(u, v, f, k):
    return f * (1 - u) - u*v**2

def H(u, v, f, k):
    return -(f + k) * v + u*v**2
```

Next, we define the function that computes the right-hand side of the system of differential equations:

```
def grayscott1d(y, t, f, k, Du, Dv, dx):
    """
    Differential equations for the 1-D Gray-Scott equations.
```

```
    The ODEs are derived using the method of lines.
    """
    # The vectors u and v are interleaved in y.  We define
    # views of u and v by slicing y.
    u = y[::2]
    v = y[1::2]

    # dydt is the return value of this function.
    dydt = np.empty_like(y)

    # Just like u and v are views of the interleaved vectors
    # in y, dudt and dvdt are views of the interleaved output
    # vectors in dydt.
    dudt = dydt[::2]
    dvdt = dydt[1::2]

    # Compute du/dt and dv/dt.  The end points and the interior points
    # are handled separately.
    dudt[0]    = G(u[0],    v[0],    f, k) + Du * (-2.0*u[0] + 2.0*u[1]) / dx**2
    dudt[1:-1] = G(u[1:-1], v[1:-1], f, k) + Du * np.diff(u,2) / dx**2
    dudt[-1]   = G(u[-1],   v[-1],   f, k) + Du * (- 2.0*u[-1] + 2.0*u[-2]) / dx**2
    dvdt[0]    = H(u[0],    v[0],    f, k) + Dv * (-2.0*v[0] + 2.0*v[1]) / dx**2
    dvdt[1:-1] = H(u[1:-1], v[1:-1], f, k) + Dv * np.diff(v,2) / dx**2
    dvdt[-1]   = H(u[-1],   v[-1],   f, k) + Dv * (-2.0*v[-1] + 2.0*v[-2]) / dx**2

    return dydt
```

We won't implement a function to compute the Jacobian, but we will tell `odeint` that the Jacobian matrix is banded. This allows the underlying solver (LSODA) to avoid computing values that it knows are zero. For a large system, this improves the performance significantly, as demonstrated in the following ipython session.

First, we define the required inputs:

```
In [30]: rng = np.random.default_rng()

In [31]: y0 = rng.standard_normal(5000)

In [32]: t = np.linspace(0, 50, 11)

In [33]: f = 0.024

In [34]: k = 0.055

In [35]: Du = 0.01

In [36]: Dv = 0.005

In [37]: dx = 0.025
```

Time the computation without taking advantage of the banded structure of the Jacobian matrix:

```
In [38]: %timeit sola = odeint(grayscott1d, y0, t, args=(f, k, Du, Dv, dx))
1 loop, best of 3: 25.2 s per loop
```

Now set `ml=2` and `mu=2`, so `odeint` knows that the Jacobian matrix is banded:

```
In [39]: %timeit solb = odeint(grayscott1d, y0, t, args=(f, k, Du, Dv, dx), ml=2, mu=2)
10 loops, best of 3: 191 ms per loop
```

That is quite a bit faster!

Let's ensure that they have computed the same result:

```
In [41]: np.allclose(sola, solb)
Out[41]: True
```

References

MOL https://en.wikipedia.org/wiki/Method_of_lines

Interpolation (scipy.interpolate): overview

Source: https://docs.scipy.org/doc/scipy/tutorial/interpolate.html

There are several general facilities available in SciPy for interpolation and smoothing for data in 1, 2, and higher dimensions. The choice of a specific interpolation routine depends on the data: whether it is one-dimensional, is given on a structured grid, or is unstructured. One other factor is the desired smoothness of the interpolator. In short, routines recommended *for interpolation* can be summarized as follows:

	kind	**routine**	**continuity**	**comment**
1D	linear	numpy.interp	piecewise continuous	Alternatively, `make_interp_spline(..., k=1)`
	cubic spline	CubicSpline	2nd derivative	
	monotone cubic spline	PchipInterpolator	1st derivative	non-overshooting
	non-cubic spline	make_interp_spline	(k-1)th derivative	`k=3` is equivalent to CubicSpline
	nearest	interp1d		kind='nearest', 'previous', 'next'
N-D curve	nearest, linear, spline	make_interp_spline	(k-1)th derivative	use N-dim y array
N-D regular (*rectilinear*) grid	nearest	RegularGridInterpolator		method='nearest'
	linear			method='linear'
	splines		2nd derivatives	method='cubic', 'quintic'
	monotone splines		1st derivatives	method='pchip'
N-D scattered	nearest	NearestNDInterpolator		alias: griddata
	linear	LinearNDInterpolator		
	cubic (2D only)	CloughTocher2DInterpolator	1st derivatives	
	radial basis function	RBFInterpolator		

Smoothing and approximation of data

1D spline functions	make_smoothing_spline	classic smoothing splines, GCV penalty
	make_splrep	automated/semi-automated knot selection
spine curves in N-D	make_splprep	

unconstrained least squares spline fit	make_lsq_spline	
2D smoothing surfaces	bisplrep	scattered data
	RectBivariateSpline	gridded data
Radial basis functions in N-D	RBFInterpolator	

Further details are given in the links below

1-D interpolation

Source: https://docs.scipy.org/doc/scipy/tutorial/interpolate/1D.html

Piecewise linear interpolation

If all you need is a linear (a.k.a. broken line) interpolation, you can use the `numpy.interp` routine. It takes two arrays of data to interpolate, `x`, and `y`, and a third array, `xnew`, of points to evaluate the interpolation on:

```
>>> import numpy as np
>>> x = np.linspace(0, 10, num=11)
>>> y = np.cos(-x**2 / 9.0)
```

Construct the interpolation

```
>>> xnew = np.linspace(0, 10, num=1001)
>>> ynew = np.interp(xnew, x, y)
```

And plot it

```
>>> import matplotlib.pyplot as plt
>>> plt.plot(xnew, ynew, '-', label='linear interp')
>>> plt.plot(x, y, 'o', label='data')
>>> plt.legend(loc='best')
>>> plt.show()
```

One limitation of `numpy.interp` is that it does not allow controlling the extrapolation. See the interpolation with B-Splines section section for alternative routines which provide this kind of functionality.

Cubic splines

Of course, piecewise linear interpolation produces corners at data points, where the linear pieces join. To produce a smoother curve, you can use cubic splines, where the interpolating curve is made of cubic pieces with matching first and second derivatives. In code, these objects are represented by instances of the `CubicSpline` class. An instance is constructed with the `x` and `y` arrays of data, and then it can be evaluated using the target `xnew` values:

```
>>> from scipy.interpolate import CubicSpline
>>> spl = CubicSpline([1, 2, 3, 4, 5, 6], [1, 4, 8, 16, 25, 36])
>>> spl(2.5)
5.57
```

A `CubicSpline` object's `__call__` method accepts both scalar values and arrays. It also accepts a second argument, `nu`, to evaluate the derivative of order `nu`. As an example, we plot the derivatives of a spline:

```
>>> from scipy.interpolate import CubicSpline
>>> x = np.linspace(0, 10, num=11)
>>> y = np.cos(-x**2 / 9.)
>>> spl = CubicSpline(x, y)
```

```
>>> import matplotlib.pyplot as plt
>>> fig, ax = plt.subplots(4, 1, figsize=(5, 7))
>>> xnew = np.linspace(0, 10, num=1001)
>>> ax[0].plot(xnew, spl(xnew))
>>> ax[0].plot(x, y, 'o', label='data')
>>> ax[1].plot(xnew, spl(xnew, nu=1), '--', label='1st derivative')
>>> ax[2].plot(xnew, spl(xnew, nu=2), '--', label='2nd derivative')
>>> ax[3].plot(xnew, spl(xnew, nu=3), '--', label='3rd derivative')
>>> for j in range(4):
...     ax[j].legend(loc='best')
>>> plt.tight_layout()
>>> plt.show()
```

Note that the first and second derivatives are continuous by construction, and the third derivative jumps at data points.

Monotone interpolants

Cubic splines are by construction twice continuously differentiable. This may lead to the spline function oscillating and ''overshooting'' in between the data points. In these situations, an alternative is to use the so-called *monotone* cubic interpolants: these are constructed to be only once continuously differentiable, and attempt to preserve the local shape implied by the data. `scipy.interpolate` provides two objects of this kind: `PchipInterpolator` and `Akima1DInterpolator`. To illustrate, let's consider data with an outlier:

```
>>> from scipy.interpolate import CubicSpline, PchipInterpolator, Akima1DInterpolator
>>> x = np.array([1., 2., 3., 4., 4.5, 5., 6., 7., 8])
>>> y = x**2
>>> y[4] += 101

>>> import matplotlib.pyplot as plt
>>> xx = np.linspace(1, 8, 51)
>>> plt.plot(xx, CubicSpline(x, y)(xx), '--', label='spline')
>>> plt.plot(xx, Akima1DInterpolator(x, y)(xx), '-', label='Akima1D')
>>> plt.plot(xx, PchipInterpolator(x, y)(xx), '-', label='pchip')
>>> plt.plot(x, y, 'o')
>>> plt.legend()
>>> plt.show()
```

Interpolation with B-splines

B-splines form an alternative (though formally equivalent) representation of piecewise polynomials. This basis is generally more computationally stable than the power basis and is useful for a variety of applications which include interpolation, regression and curve representation. Details are given in the piecewise polynomials section, and here we illustrate their usage by constructing the interpolation of a sine function:

```
>>> x = np.linspace(0, 3/2, 7)
>>> y = np.sin(np.pi*x)
```

To construct the interpolating objects given data arrays, `x` and `y`, we use the `make_interp_spline` function:

```
>>> from scipy.interpolate import make_interp_spline
>>> bspl = make_interp_spline(x, y, k=3)
```

This function returns an object which has an interface similar to that of the `CubicSpline` objects. In particular, it can be evaluated at a data point and differentiated:

```
>>> der = bspl.derivative()      # a BSpline representing the derivative
>>> import matplotlib.pyplot as plt
>>> xx = np.linspace(0, 3/2, 51)
>>> plt.plot(xx, bspl(xx), '--', label=r'$\sin(\pi x)$ approx')
>>> plt.plot(x, y, 'o', label='data')
>>> plt.plot(xx, der(xx)/np.pi, '--', label=r'$d \sin(\pi x)/dx / \pi$ approx')
>>> plt.legend()
>>> plt.show()
```

Note that by specifying `k=3` in the `make_interp_spline` call, we requested a cubic spline (this is the default, so `k=3` could have been omitted); the derivative of a cubic is a quadratic:

```
>>> bspl.k, der.k
(3, 2)
```

By default, the result of `make_interp_spline(x, y)` is equivalent to `CubicSpline(x, y)`. The difference is that the former allows several optional capabilities: it can construct splines of various degrees (via the optional argument `k`) and predefined knots (via the optional argument `t`).

Boundary conditions for the spline interpolation can be controlled by the `bc_type` argument to `make_interp_spline` function and `CubicSpline` constructor. By default, both use the 'not-a-knot' boundary condition.

Non-cubic splines

One use of `make_interp_spline` is constructing a linear interpolant with linear extrapolation since `make_interp_spline` extrapolates by default. Consider

```
>>> from scipy.interpolate import make_interp_spline
>>> x = np.linspace(0, 5, 11)
>>> y = 2*x
>>> spl = make_interp_spline(x, y, k=1)  # k=1: linear
>>> spl([-1, 6])
[-2., 12.]
>>> np.interp([-1, 6], x, y)
[0., 10.]
```

See the extrapolation section for more details and discussion.

Batches of `y`

Univariate interpolators accept not only one-dimensional `y` arrays, but also `y.ndim > 1`. The interpretation is that `y` is a *batch* of 1D data arrays: by default, the zeroth dimension of `y` is the interpolation axis, and the trailing dimensions are batch dimensions. Consider a collection (a *batch*) of functions f_j sampled at the points x_i. We can instantiate a single interpolator for all of these functions by providing a two-dimensional array `y` such that `y[i, j]` records $f_j(x_i)$.

To illustrate:

```
>>> import numpy as np
>>> import matplotlib.pyplot as plt
>>> from scipy.interpolate import make_interp_spline
>>> n = 11
>>> x = 2 * np.pi * np.arange(n) / n
>>> x.shape
```

```
(11,)
>>> y = np.stack((np.sin(x)**2, np.cos(x)), axis=1)
>>> y.shape
(11, 2)
>>> spl = make_interp_spline(x, y)
>>> xv = np.linspace(0, 2*np.pi, 51)
>>> plt.plot(x, y, 'o')
>>> plt.plot(xv, spl(xv), '-')
>>> plt.show()
```

Several notes are in order. First and foremost, the behavior here looks similar to NumPy's broadcasting, but differs in two respects:

1. The `x` array is expected to be 1D even if the `y` array is not: `x.ndim == 1` while `y.ndim >= 1`. There is no broadcasting of `x` vs `y`.
2. By default, the *trailing* dimensions are used as batch dimensions, in contrast to the NumPy convention of using the *leading* dimensions as batch dimensions.

Second, the interpolation axis can be controlled by an optional `axis` argument. The example above uses the default value of `axis=0`. For a non-default values, the following is true:

- `y.shape[axis] == x.size` (otherwise an error is raised)
- the shape of `spl(xv)` is `y.shape[axis:] + xv.shape + y.shape[:axis]`

While we demonstrated the batching behavior with `make_interp_spline`, in fact the majority of univariate interpolators support this functionality: `PchipInterpolator` and `Akima1DInterpolator`, `CubicSpline`; low-level polynomial representation classes, `PPoly`, `BPoly` and `BSpline`; as well as `least-squares fit and spline smoothing functions<tutorial-interpolate_fitpack>`, `make_lsq_spline` and `make_smoothing_spline`.

Parametric spline curves

So far we considered spline *functions*, where the data, `y`, is expected to depend explicitly on the independent variable `x`---so that the interpolating function satisfies $f(x_j) = y_j$. Spline *curves* treat the `x` and `y` arrays as coordinates of points, $\mathbf{p}_j$ on a plane, and an interpolating curve which passes through these points is parameterized by some additional parameter (typically called `u`). Note that this construction readily generalizes to higher dimensions where $\mathbf{p}_j$ are points in an N-dimensional space.

Spline curves can be easily constructed using the fact that interpolation functions handle multidimensional data arrays, as discussed in the previous section. The values of the parameter, `u`, corresponding to the data points, need to be separately supplied by the user.

The choice of parametrization is problem-dependent and different parametrizations may produce vastly different curves. As an example, we consider three parametrizations of (a somewhat difficult) dataset, which we take from Chapter 6 of Ref [1] listed in the `BSpline` docstring:

```
>>> x = [0, 1, 2, 3, 4, 5, 6]
>>> y = [0, 0, 0, 9, 0, 0, 0]
>>> p = np.stack((x, y))
>>> p
array([[0, 1, 2, 3, 4, 5, 6],
       [0, 0, 0, 9, 0, 0, 0]])
```

We take elements of the `p` array as coordinates of seven points on the plane, where `p[:, j]` gives the coordinates of the point

First, consider the *uniform* parametrization, $u_j = j$:

```
>>> u_unif = x
```

Second, we consider the so-called *chord length* parametrization, which is nothing but a cumulative length of straight line segments connecting the data points:

$$u_j = u_{j-1} + |\mathbf{p}_j - \mathbf{p}_{j-1}|$$

for $j = 1, 2, \ldots$ and $u_0 = 0$. Here $|\cdots|$ is the length between the consecutive points p_j on the plane.

```
>>> dp = p[:, 1:] - p[:, :-1]      # 2-vector distances between points
>>> l = (dp**2).sum(axis=0)        # squares of lengths of 2-vectors between points
>>> u_chord = np.sqrt(l).cumsum()   # cumulative sums of 2-norms
>>> u_chord = np.r_[0, u_chord]      # the first point is parameterized at zero
```

Finally, we consider what is sometimes called the *centripetal* parametrization: $u_j = u_{j-1} + |\mathbf{p}_j - \mathbf{p}_{j-1}|^{1/2}$. Due to the extra square root, the difference between consecutive values

```
>>> u_c = np.r_[0, np.cumsum((dp**2).sum(axis=0)**0.25)]
```

Now plot the resulting curves:

```
>>> from scipy.interpolate import make_interp_spline
>>> import matplotlib.pyplot as plt
>>> fig, ax = plt.subplots(1, 3, figsize=(8, 3))
>>> parametrizations = ['uniform', 'chord length', 'centripetal']
>>>
>>> for j, u in enumerate([u_unif, u_chord, u_c]):
...     spl = make_interp_spline(u, p, axis=1)    # note p is a 2D array
...
...     uu = np.linspace(u[0], u[-1], 51)
...     xx, yy = spl(uu)
...
...     ax[j].plot(xx, yy, '--')
...     ax[j].plot(p[0, :], p[1, :], 'o')
...     ax[j].set_title(parametrizations[j])
>>> plt.show()
```

Missing data

We note that `scipy.interpolate` does *not* support interpolation with missing data. Two popular ways of representing missing data are using masked arrays of the `numpy.ma` library, and encoding missing values as not-a-number, `NaN`.

Neither of these two approaches is directly supported in `scipy.interpolate`. Individual routines may offer partial support, and/or workarounds, but in general, the library firmly adheres to the IEEE 754 semantics where a `NaN` means *not-a-number*, i.e. a result of an illegal mathematical operation (e.g., division by zero), not *missing*.

Out-of-bounds behavior

Several interpolators in `scipy.interpolate` share a common way to control the out-of-bounds behavior, including `CubicSpline`, `CubicHermiteSpline`, `Akima1DInterpolator`, `PchipInterpolator`, `PPoly`, `BPoly`, and `BSpline`.

There are two principal ingredients which affect the out-of-bounds behavior: `bc_type` and `extrapolate` arguments. The former controls how the object is constructed, and the latter controls how it is evaluated.

The `bc_type` parameter

For `CubicSpline` and `make_interp_spline`, the `bc_type` construction-time argument controls the shape of the spline at its boundaries and therefore also affects extrapolation. When `bc_type="periodic"`, periodic boundary conditions are enforced and the interpolant is smooth at the join point. For all other values of `bc_type`, out-of-bounds behavior is controlled separately by the `extrapolate` argument described below.

To illustrate, we compare several values of the `extrapolate` constructor argument for `CubicSpline`, including `bc_type="periodic"` which produces a smooth periodic interpolant:

```
import matplotlib.pyplot as plt
import numpy as np
from scipy.interpolate import CubicSpline

# data
x = np.array([0, 1, 2.5, 3, 4])
y = np.array([0.0, 1.0, 0.8, 1.2, 0.0])
# evaluation points
x_eval = np.linspace(-4, 8, 300)

fig, ax = plt.subplots(figsize=(8, 6))
for extrapolate, linestyle in zip(
    [True, False, "periodic"], ["-", "--", ":"], strict=True
):
    y_interp = CubicSpline(x, y, extrapolate=extrapolate)(x_eval)
    ax.plot(x_eval, y_interp, linestyle, label=f"extrapolate={extrapolate}")

ax.plot(
    x_eval,
    CubicSpline(x, y, bc_type="periodic")(x_eval),
    "-.",
    label="bc_type='periodic'",
)
ax.plot(x, y, "ko", label="data")
ax.axvline(x[0] - 4, color="k", alpha=0.2)
ax.axvline(x[0], color="k", alpha=0.2)
ax.axvline(x[-1], color="k", alpha=0.2)
ax.axvline(x[-1] + 4, color="k", alpha=0.2)
ax.set_ylim(-0.5, 1.5)
ax.legend(ncol=2, loc="lower left")
ax.set_xlabel("x")
ax.set_ylabel("f(x)")
ax.set_title("CubicSpline")
plt.show()
```

Note that using `bc_type="periodic"` (red curve) enforces equal derivatives at the boundaries of the interpolation interval (`x=0` and `x=4`), so that the resulting interpolant extends smoothly as a periodic function.

On the contrary, using `extrapolate="periodic"` together with the default value for `bc_type` (the green curve) does not enforce any boundary conditions, and simply extends the interpolant + periodically outside the data domain. See tutorial-extrapolation-cubicspline-extend for an additional discussion of how different `bc_type` values affect the interpolant.

The `extrapolate` constructor argument

All the interpolators listed above accept an `extrapolate` keyword at construction time, which sets the default out-of-bounds behavior for all subsequent evaluations:

- `extrapolate=True` (default for most): evaluate the boundary polynomial pieces;
- `extrapolate=False`: return `NaN`;
- `extrapolate="periodic"` (if supported): wrap the evaluation periodically.

The default is `True` for most interpolators, with two exceptions: `Akima1DInterpolator` defaults to `False`, and `bc_type="periodic"` forces the default to `extrapolate="periodic"`.

To illustrate, we consider `PchipInterpolator` with several values of the `extrapolate` constructor argument:

```
import matplotlib.pyplot as plt
import numpy as np
from scipy.interpolate import PchipInterpolator

# data
x = np.array([0, 1, 2.5, 3, 4])
y = np.array([0.0, 1.0, 0.8, 1.2, 0.0])
# evaluation points
x_eval = np.linspace(-4, 8, 300)

fig, ax = plt.subplots(figsize=(8, 6))
for extrapolate, linestyle in zip(
    [True, False, "periodic"], ["-", "--", ":"], strict=True
):
    y_interp = PchipInterpolator(x, y, extrapolate=extrapolate)(x_eval)
    ax.plot(x_eval, y_interp, linestyle, label=f"extrapolate={extrapolate}")
ax.plot(x, y, "k.", markersize=8, label="data")
ax.axvline(x[0] - 4, color="k", alpha=0.2)
ax.axvline(x[0], color="k", alpha=0.2)
ax.axvline(x[-1], color="k", alpha=0.2)
ax.axvline(x[-1] + 4, color="k", alpha=0.2)
ax.set_ylim(-2.0, 2.5)
ax.legend(ncol=2, loc="lower left")
ax.set_xlabel("x")
ax.set_ylabel("f(x)")
ax.set_title("PchipInterpolator")
plt.show()
```

Note that `PchipInterpolator` has no `bc_type` argument, so there is no way to construct a truly periodic PCHIP interpolant. Using `extrapolate="periodic"` only wraps evaluation at the domain boundaries and makes no guarantees about smoothness at the join point, as visible in the plot above.

Overriding `extrapolate` per call

The `extrapolate` argument can also be passed at evaluation time (i.e. when calling the interpolator object). Passing `extrapolate=None` (the default) means "use the value that was set at construction time." Passing any other value overrides the construction-time setting for that call only:

```
>>> from scipy.interpolate import CubicSpline
>>> import numpy as np
>>> spl = CubicSpline([0, 1, 2], [0, 1, 0], extrapolate=False)
>>> x_eval = np.array([-1, 0.5, 3])
>>> spl(x_eval)           # uses construction-time extrapolate=False
array([nan, 0.75, nan])
```

```
>>> spl(x_eval, extrapolate=True)   # override for this call
array([-3.,  0.75,  -3.])
```

See Extrapolation tips and tricks for further details.

Legacy interface for 1-D interpolation (`interp1d`)

> Note
> `interp1d` is considered legacy API and is not recommended for use in new code. Consider using more specific interpolators instead.

The `interp1d` class in `scipy.interpolate` is a convenient method to create a function based on fixed data points, which can be evaluated anywhere within the domain defined by the given data using linear interpolation. An instance of this class is created by passing the 1-D vectors comprising the data. The instance of this class defines a `__call__` method and can therefore be treated like a function which interpolates between known data values to obtain unknown values. Behavior at the boundary can be specified at instantiation time. The following example demonstrates its use, for linear and cubic spline interpolation:

```
>>> from scipy.interpolate import interp1d

>>> x = np.linspace(0, 10, num=11, endpoint=True)
>>> y = np.cos(-x**2/9.0)
>>> f = interp1d(x, y)
>>> f2 = interp1d(x, y, kind='cubic')

>>> xnew = np.linspace(0, 10, num=41, endpoint=True)
>>> import matplotlib.pyplot as plt
>>> plt.plot(x, y, 'o', xnew, f(xnew), '-', xnew, f2(xnew), '--')
>>> plt.legend(['data', 'linear', 'cubic'], loc='best')
>>> plt.show()
```

The 'cubic' kind of `interp1d` is equivalent to `make_interp_spline`, and the 'linear' kind is equivalent to `numpy.interp` while also allowing N-dimensional `y` arrays.

Another set of interpolations in `interp1d` is `nearest`, `previous`, and `next`, where they return the nearest, previous, or next point along the x-axis. Nearest and next can be thought of as a special case of a causal interpolating filter. The following example demonstrates their use, using the same data as in the previous example:

```
>>> from scipy.interpolate import interp1d

>>> x = np.linspace(0, 10, num=11, endpoint=True)
>>> y = np.cos(-x**2/9.0)
>>> f1 = interp1d(x, y, kind='nearest')
>>> f2 = interp1d(x, y, kind='previous')
>>> f3 = interp1d(x, y, kind='next')

>>> xnew = np.linspace(0, 10, num=1001, endpoint=True)
>>> import matplotlib.pyplot as plt
>>> plt.plot(x, y, 'o')
>>> plt.plot(xnew, f1(xnew), '-', xnew, f2(xnew), '--', xnew, f3(xnew), ':')
>>> plt.legend(['data', 'nearest', 'previous', 'next'], loc='best')
>>> plt.show()
```

Recommended replacements for `interp1d` modes ----------

As mentioned, `interp1d` class is *legacy*: we have no plans to remove it; we are going to keep supporting its existing usages; however we believe there are better alternatives which we recommend using in new code.

Here we list specific recommendations, depending on the interpolation `kind`.

Linear interpolation, `kind="linear"`

The default recommendation is to use `numpy.interp` function. Alternatively, you can use linear splines, `make_interp_spline(x, y, k=1)`, see this section for a discussion.

Spline interpolators, `kind="quadratic"` or `"cubic"`

Under the hood, `interp1d` delegates to `make_interp_spline`, so we recommend using the latter directly.

Piecewise constant modes, `kind="nearest", "previous", "next"`

First, we note that `interp1d(x, y, kind='previous')` is equivalent to `make_interp_spline(x, y, k=0)`.

More generally however, all these piecewise constant interpolation modes are based on `numpy.searchsorted`. For example, the `"nearest"` mode is nothing but

```
>>> x = np.arange(8)
>>> y = x**2
>>> x_new = np.linspace(0, 7, 101)    # input points
>>> x_bds = x[:-1] / 2.0 + x[1:] / 2.0   # halfway points
>>> idx = np.searchsorted(x_bds, x_new, side='left')
>>> idx = np.clip(idx, 0, len(x) - 1)  # clip the indices so that they are within the range of x indices.
>>> import matplotlib.pyplot as plt
>>> plt.plot(x, y, 'o')
>>> plt.plot(x_new, y[idx], '--')
>>> plt.show()
```

Other variants are similar, see the `interp1d` source code for details.

Linear Algebra (`scipy.linalg`) ==========

When SciPy is built using the optimized ATLAS LAPACK and BLAS libraries, it has very fast linear algebra capabilities. If you dig deep enough, all of the raw LAPACK and BLAS libraries are available for your use for even more speed. In this section, some easier-to-use interfaces to these routines are described.

All of these linear algebra routines expect an object that can be converted into a 2-D array. The output of these routines is also a 2-D array.

Linear algebra (scipy.linalg)

Source: https://docs.scipy.org/doc/scipy/tutorial/linalg.html

`scipy.linalg` contains all the functions in numpy.linalg. plus some other more advanced ones not contained in `numpy.linalg`.

Another advantage of using `scipy.linalg` over `numpy.linalg` is that it is always compiled with BLAS/LAPACK support, while for NumPy this is optional. Therefore, the SciPy version might be faster depending on how NumPy was installed.

Therefore, unless you don't want to add `scipy` as a dependency to your `numpy` program, use `scipy.linalg` instead of `numpy.linalg`.

numpy.matrix vs 2-D numpy.ndarray

The classes that represent matrices, and basic operations, such as matrix multiplications and transpose are a part of `numpy`. For convenience, we summarize the differences between `numpy.matrix` and `numpy.ndarray` here.

`numpy.matrix` is matrix class that has a more convenient interface than `numpy.ndarray` for matrix operations. This class supports, for example, MATLAB-like creation syntax via the semicolon, has matrix multiplication as default for the `*` operator, and contains `I` and `T` members that serve as shortcuts for inverse and transpose:

```
>>> import numpy as np
>>> A = np.asmatrix('[1 2;3 4]')
>>> A
matrix([[1, 2],
        [3, 4]])
>>> A.I
matrix([[-2. ,  1. ],
        [ 1.5, -0.5]])
>>> b = np.asmatrix('[5 6]')
>>> b
matrix([[5, 6]])
>>> b.T
matrix([[5],
        [6]])
>>> A*b.T
matrix([[17],
        [39]])
```

Despite its convenience, the use of the `numpy.matrix` class is discouraged, since it adds nothing that cannot be accomplished with 2-D `numpy.ndarray` objects, and may lead to a confusion of which class is being used. For example, the above code can be rewritten as:

```
>>> import numpy as np
>>> from scipy import linalg
>>> A = np.array([[1,2],[3,4]])
>>> A
array([[1, 2],
      [3, 4]])
>>> linalg.inv(A)
array([[-2. ,  1. ],
```

```
      [ 1.5, -0.5]])
>>> b = np.array([[5,6]]) #2D array
>>> b
array([[5, 6]])
>>> b.T
array([[5],
      [6]])
>>> A*b #not matrix multiplication!
array([[ 5, 12],
      [15, 24]])
>>> A.dot(b.T) #matrix multiplication
array([[17],
      [39]])
>>> b = np.array([5,6]) #1D array
>>> b
array([5, 6])
>>> b.T  #not matrix transpose!
array([5, 6])
>>> A.dot(b)  #does not matter for multiplication
array([17, 39])
```

`scipy.linalg` operations can be applied equally to `numpy.matrix` or to 2D `numpy.ndarray` objects.

Basic routines

Finding the inverse

The inverse of a matrix $\mathbf{A}$ is the matrix $\mathbf{B}$, such that $\mathbf{AB} = \mathbf{I}$, where $\mathbf{I}$ is the identity matrix consisting of ones down the main diagonal. Usually, $\mathbf{B}$ is denoted $\mathbf{B} = \mathbf{A}^{-1}$. In SciPy, the matrix inverse of the NumPy array, A, is obtained using `linalg.inv (A)`, or using `A.I` if `A` is a Matrix. For example, let

$$\mathbf{A} = \begin{bmatrix} 1 & 3 & 5 \\ 2 & 5 & 1 \\ 2 & 3 & 8 \end{bmatrix},$$

then

$$\mathbf{A^{-1}} = \frac{1}{25} \begin{bmatrix} -37 & 9 & 22 \\ 14 & 2 & -9 \\ 4 & -3 & 1 \end{bmatrix} = \begin{bmatrix} -1.48 & 0.36 & 0.88 \\ 0.56 & 0.08 & -0.36 \\ 0.16 & -0.12 & 0.04 \end{bmatrix}.$$

The following example demonstrates this computation in SciPy

```
>>> import numpy as np
>>> from scipy import linalg
>>> A = np.array([[1,3,5],[2,5,1],[2,3,8]])
>>> A
array([[1, 3, 5],
      [2, 5, 1],
      [2, 3, 8]])
>>> linalg.inv(A)
array([[-1.48,  0.36,  0.88],
      [ 0.56,  0.08, -0.36],
      [ 0.16, -0.12,  0.04]])
>>> A.dot(linalg.inv(A)) #double check
```

```
array([[  1.00000000e+00,  -1.11022302e-16,  -5.55111512e-17],
       [  3.05311332e-16,   1.00000000e+00,   1.87350135e-16],
       [  2.22044605e-16,  -1.11022302e-16,   1.00000000e+00]])
```

Solving a linear system

Solving linear systems of equations is straightforward using the scipy command `linalg.solve`. This command expects an input matrix and a right-hand side vector. The solution vector is then computed. An option for entering a symmetric matrix is offered, which can speed up the processing when applicable. As an example, suppose it is desired to solve the following simultaneous equations:

$$\begin{aligned} x + 3y + 5z &= 10 \\ 2x + 5y + z &= 8 \\ 2x + 3y + 8z &= 3 \end{aligned}$$

We could find the solution vector using a matrix inverse:

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 1 & 3 & 5 \\ 2 & 5 & 1 \\ 2 & 3 & 8 \end{bmatrix}^{-1} \begin{bmatrix} 10 \\ 8 \\ 3 \end{bmatrix} = \frac{1}{25} \begin{bmatrix} -232 \\ 129 \\ 19 \end{bmatrix} = \begin{bmatrix} -9.28 \\ 5.16 \\ 0.76 \end{bmatrix}.$$

However, it is better to use the linalg.solve command, which can be faster and more numerically stable. In this case, it, however, gives the same answer as shown in the following example:

```
>>> import numpy as np
>>> from scipy import linalg
>>> A = np.array([[1, 2], [3, 4]])
>>> A
array([[1, 2],
       [3, 4]])
>>> b = np.array([[5], [6]])
>>> b
array([[5],
       [6]])
>>> linalg.inv(A).dot(b)  # slow
array([[-4. ],
       [ 4.5]])
>>> A.dot(linalg.inv(A).dot(b)) - b  # check
array([[  8.88178420e-16],
       [  2.66453526e-15]])
>>> np.linalg.solve(A, b)  # fast
array([[-4. ],
       [ 4.5]])
>>> A.dot(np.linalg.solve(A, b)) - b  # check
array([[ 0.],
       [ 0.]])
```

Finding the determinant

The determinant of a square matrix $\mathbf{A}$ is often denoted $|\mathbf{A}|$ and is a quantity often used in linear algebra. Suppose a_{ij} are the elements of the matrix $\mathbf{A}$ and let $M_{ij} = |\mathbf{A}_{ij}|$ be the determinant of the matrix left by removing the ith row and j^{th} column from $\mathbf{A}$. Then, for any row i,

$$|\mathbf{A}| = \sum_j (-1)^{i+j} a_{ij} M_{ij}.$$

This is a recursive way to define the determinant, where the base case is defined by accepting that the determinant of a 1×1 matrix is the only matrix element. In SciPy the determinant can be calculated with `linalg.det`. For example, the determinant of

$$\mathbf{A} = \begin{bmatrix} 1 & 3 & 5 \\ 2 & 5 & 1 \\ 2 & 3 & 8 \end{bmatrix}$$

is

$$\begin{aligned} |\mathbf{A}| &= 1\begin{vmatrix} 5 & 1 \\ 3 & 8 \end{vmatrix} - 3\begin{vmatrix} 2 & 1 \\ 2 & 8 \end{vmatrix} + 5\begin{vmatrix} 2 & 5 \\ 2 & 3 \end{vmatrix}. \\ &= 1\,(5 \cdot 8 - 3 \cdot 1) - 3\,(2 \cdot 8 - 2 \cdot 1) + 5\,(2 \cdot 3 - 2 \cdot 5) = -25. \end{aligned}$$

In SciPy, this is computed as shown in this example:

```
>>> import numpy as np
>>> from scipy import linalg
>>> A = np.array([[1,2],[3,4]])
>>> A
array([[1, 2],
       [3, 4]])
>>> linalg.det(A)
-2.0
```

Computing norms

Matrix and vector norms can also be computed with SciPy. A wide range of norm definitions are available using different parameters to the order argument of `linalg.norm`. This function takes a rank-1 (vectors) or a rank-2 (matrices) array and an optional order argument (default is 2). Based on these inputs, a vector or matrix norm of the requested order is computed.

For vector *x*, the order parameter can be any real number including `inf` or `-inf`. The computed norm is

$$\|\mathbf{x}\| = \begin{cases} \max |x_i| & \text{ord} = \text{inf} \\ \min |x_i| & \text{ord} = -\text{inf} \\ \left(\sum_i |x_i|^{\text{ord}}\right)^{1/\text{ord}} & |\text{ord}| < \infty. \end{cases}$$

For matrix **A**, the only valid values for norm are $\pm 2, \pm 1, \pm$ inf, and 'fro' (or 'f') Thus,

$$\|\mathbf{A}\| = \begin{cases} \max_i \sum_j |a_{ij}| & \text{ord} = \text{inf} \\ \min_i \sum_j |a_{ij}| & \text{ord} = -\text{inf} \\ \max_j \sum_i |a_{ij}| & \text{ord} = 1 \\ \min_j \sum_i |a_{ij}| & \text{ord} = -1 \\ \max \sigma_i & \text{ord} = 2 \\ \min \sigma_i & \text{ord} = -2 \\ \sqrt{\text{trace}\,(\mathbf{A}^H \mathbf{A})} & \text{ord} = \text{'fro'} \end{cases}$$

where σ_i are the singular values of $\mathbf{A}$.

Examples:

```
>>> import numpy as np
>>> from scipy import linalg
>>> A=np.array([[1, 2], [3, 4]])
>>> A
array([[1, 2],
       [3, 4]])
>>> linalg.norm(A)
5.4772255750516612
>>> linalg.norm(A, 'fro') # frobenius norm is the default
5.4772255750516612
>>> linalg.norm(A, 1) # L1 norm (max column sum)
6.0
>>> linalg.norm(A, -1)
4.0
>>> linalg.norm(A, np.inf) # L inf norm (max row sum)
7.0
```

Solving linear least-squares problems and pseudo-inverses

Linear least-squares problems occur in many branches of applied mathematics. In this problem, a set of linear scaling coefficients is sought that allows a model to fit the data. In particular, it is assumed that data y_i is related to data $\mathbf{x}_i$ through a set of coefficients c_j and model functions $f_j(\mathbf{x}_i)$ via the model

$$y_i = \sum_j c_j f_j(\mathbf{x}_i) + \epsilon_i,$$

where ϵ_i represents uncertainty in the data. The strategy of least squares is to pick the coefficients c_j to minimize

$$J(\mathbf{c}) = \sum_i \left| y_i - \sum_j c_j f_j(x_i) \right|^2.$$

Theoretically, a global minimum will occur when

$$\frac{\partial J}{\partial c_n^*} = 0 = \sum_i \left(y_i - \sum_j c_j f_j(x_i) \right) \left(-f_n^*(x_i) \right)$$

or

$$\begin{aligned} \sum_j c_j \sum_i f_j(x_i) f_n^*(x_i) &= \sum_i y_i f_n^*(x_i) \\ \mathbf{A}^H \mathbf{A} \mathbf{c} &= \mathbf{A}^H \mathbf{y} \end{aligned},$$

where

$$\{\mathbf{A}\}_{ij} = f_j(x_i).$$

When $\mathbf{A^H A}$ is invertible, then

$$\mathbf{c} = \left(\mathbf{A}^H\mathbf{A}\right)^{-1}\mathbf{A}^H\mathbf{y} = \mathbf{A}^\dagger\mathbf{y},$$

where $\mathbf{A}^\dagger$ is called the pseudo-inverse of $\mathbf{A}$. Notice that using this definition of $\mathbf{A}$ the model can be written

$$\mathbf{y} = \mathbf{A}\mathbf{c} + \epsilon.$$

The command `linalg.lstsq` will solve the linear least-squares problem for $\mathbf{c}$ given $\mathbf{A}$ and $\mathbf{y}$. In addition, `linalg.pinv` will find $\mathbf{A}^\dagger$ given $\mathbf{A}$.

The following example and figure demonstrate the use of `linalg.lstsq` and `linalg.pinv` for solving a data-fitting problem. The data shown below were generated using the model:

$$y_i = c_1 e^{-x_i} + c_2 x_i,$$

where $x_i = 0.1i$ for $i = 1, \ldots, 10$, $c_1 = 5$, and $c_2 = 4$. Noise is added to y_i and the coefficients c_1 and c_2 are estimated using linear least squares.

```
>>> import numpy as np
>>> from scipy import linalg
>>> import matplotlib.pyplot as plt
>>> rng = np.random.default_rng()

>>> c1, c2 = 5.0, 2.0
>>> i = np.r_[1:11]
>>> xi = 0.1*i
>>> yi = c1*np.exp(-xi) + c2*xi
>>> zi = yi + 0.05 * np.max(yi) * rng.standard_normal(len(yi))

>>> A = np.c_[np.exp(-xi)[:, np.newaxis], xi[:, np.newaxis]]
>>> c, resid, rank, sigma = linalg.lstsq(A, zi)

>>> xi2 = np.r_[0.1:1.0:100j]
>>> yi2 = c[0]*np.exp(-xi2) + c[1]*xi2

>>> plt.plot(xi,zi,'x',xi2,yi2)
>>> plt.axis([0,1.1,3.0,5.5])
>>> plt.xlabel('$x_i$')
>>> plt.title('Data fitting with linalg.lstsq')
>>> plt.show()
```

Generalized inverse

The generalized inverse is calculated using the command `linalg.pinv`. Let $\mathbf{A}$ be an $M \times N$ matrix, then if $M > N$, the generalized inverse is

$$\mathbf{A}^\dagger = \left(\mathbf{A}^H\mathbf{A}\right)^{-1}\mathbf{A}^H,$$

while if $M < N$ matrix, the generalized inverse is

$$\mathbf{A}^\# = \mathbf{A}^H\left(\mathbf{A}\mathbf{A}^H\right)^{-1}.$$

In the case that $M = N$, then

$$\mathbf{A}^{\dagger} = \mathbf{A}^{\#} = \mathbf{A}^{-1},$$

as long as $\mathbf{A}$ is invertible.

Decompositions

In many applications, it is useful to decompose a matrix using other representations. There are several decompositions supported by SciPy.

Eigenvalues and eigenvectors

The eigenvalue-eigenvector problem is one of the most commonly employed linear algebra operations. In one popular form, the eigenvalue-eigenvector problem is to find for some square matrix $\mathbf{A}$ scalars λ and corresponding vectors $\mathbf{v}$, such that

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v}.$$

For an $N \times N$ matrix, there are N (not necessarily distinct) eigenvalues --- roots of the (characteristic) polynomial

$$|\mathbf{A} - \lambda\mathbf{I}| = 0.$$

The eigenvectors, $\mathbf{v}$, are also sometimes called right eigenvectors to distinguish them from another set of left eigenvectors that satisfy

$$\mathbf{v}_L^H\mathbf{A} = \lambda\mathbf{v}_L^H$$

or

$$\mathbf{A}^H\mathbf{v}_L = \lambda^*\mathbf{v}_L.$$

With its default optional arguments, the command `linalg.eig` returns λ and $\mathbf{v}$. However, it can also return $\mathbf{v}_L$ and just λ by itself (`linalg.eigvals` returns just λ as well).

In addition, `linalg.eig` can also solve the more general eigenvalue problem

$$\begin{aligned}\mathbf{A}\mathbf{v} &= \lambda\mathbf{B}\mathbf{v} \\ \mathbf{A}^H\mathbf{v}_L &= \lambda^*\mathbf{B}^H\mathbf{v}_L\end{aligned}$$

for square matrices $\mathbf{A}$ and $\mathbf{B}$. The standard eigenvalue problem is an example of the general eigenvalue problem for $\mathbf{B} = \mathbf{I}$. When a generalized eigenvalue problem can be solved, it provides a decomposition of $\mathbf{A}$ as

$$\mathbf{A} = \mathbf{B}\mathbf{V}\Lambda\mathbf{V}^{-1},$$

where $\mathbf{V}$ is the collection of eigenvectors into columns and Λ is a diagonal matrix of eigenvalues.

By definition, eigenvectors are only defined up to a constant scale factor. In SciPy, the scaling factor for the eigenvectors is chosen so that $\|\mathbf{v}\|^2 = \sum_i v_i^2 = 1$.

As an example, consider finding the eigenvalues and eigenvectors of the matrix

$$\mathbf{A} = \begin{bmatrix} 1 & 5 & 2 \\ 2 & 4 & 1 \\ 3 & 6 & 2 \end{bmatrix}.$$

The characteristic polynomial is

$$\begin{aligned} |\mathbf{A} - \lambda\mathbf{I}| = & (1-\lambda)\left[(4-\lambda)(2-\lambda) - 6\right] - \\ & 5\left[2(2-\lambda) - 3\right] + 2\left[12 - 3(4-\lambda)\right] \\ = & -\lambda^3 + 7\lambda^2 + 8\lambda - 3. \end{aligned}$$

The roots of this polynomial are the eigenvalues of $\mathbf{A}$:

$$\begin{aligned} \lambda_1 = & \ 7.9579 \\ \lambda_2 = & -1.2577 \\ \lambda_3 = & \ 0.2997. \end{aligned}$$

The eigenvectors corresponding to each eigenvalue can be found using the original equation. The eigenvectors associated with these eigenvalues can then be found.

```
>>> import numpy as np
>>> from scipy import linalg
>>> A = np.array([[1, 2], [3, 4]])
>>> la, v = linalg.eig(A)
>>> l1, l2 = la
>>> print(l1, l2)   # eigenvalues
(-0.3722813232690143+0j) (5.372281323269014+0j)
>>> print(v[:, 0])   # first eigenvector
[-0.82456484  0.56576746]
>>> print(v[:, 1])   # second eigenvector
[-0.41597356 -0.90937671]
>>> print(np.sum(abs(v**2), axis=0))  # eigenvectors are unitary
[1. 1.]
>>> v1 = np.array(v[:, 0]).T
>>> print(linalg.norm(A.dot(v1) - l1*v1))  # check the computation
3.23682852457e-16
```

Singular value decomposition

Singular value decomposition (SVD) can be thought of as an extension of the eigenvalue problem to matrices that are not square. Let $\mathbf{A}$ be an $M \times N$ matrix with M and N arbitrary. The matrices $\mathbf{A}^H\mathbf{A}$ and $\mathbf{A}\mathbf{A}^H$ are square hermitian matrices[3] of size $N \times N$ and $M \times M$, respectively. It is known that the eigenvalues of square hermitian matrices are real and non-negative. In addition, there are at most $\min(M, N)$ identical non-zero eigenvalues of $\mathbf{A}^H\mathbf{A}$ and $\mathbf{A}\mathbf{A}^H$. Define these positive eigenvalues as σ_i^2. The square-root of these are called singular values of $\mathbf{A}$. The eigenvectors of $\mathbf{A}^H\mathbf{A}$ are collected by columns into an $N \times N$ unitary[4] matrix $\mathbf{V}$, while the eigenvectors of $\mathbf{A}\mathbf{A}^H$ are collected by columns in the unitary matrix $\mathbf{U}$, the singular values are collected in an $M \times N$ zero matrix Σ with main diagonal entries set to the singular values. Then

$$\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^H$$

[3] A hermitian matrix $\mathbf{D}$ satisfies $\mathbf{D}^H = \mathbf{D}$.
[4] A unitary matrix $\mathbf{D}$ satisfies $\mathbf{D}^H\mathbf{D} = \mathbf{I} = \mathbf{D}\mathbf{D}^H$ so that $\mathbf{D}^{-1} = \mathbf{D}^H$.

is the singular value decomposition of $\mathbf{A}$. Every matrix has a singular value decomposition. Sometimes, the singular values are called the spectrum of $\mathbf{A}$. The command `linalg.svd` will return $\mathbf{U}$, $\mathbf{V}^H$, and σ_i as an array of the singular values. To obtain the matrix Σ, use `linalg.diagsvd`. The following example illustrates the use of `linalg.svd`:

```
>>> import numpy as np
>>> from scipy import linalg
>>> A = np.array([[1,2,3],[4,5,6]])
>>> A
array([[1, 2, 3],
       [4, 5, 6]])
>>> M,N = A.shape
>>> U,s,Vh = linalg.svd(A)
>>> Sig = linalg.diagsvd(s,M,N)
>>> U, Vh = U, Vh
>>> U
array([[-0.3863177 , -0.92236578],
       [-0.92236578,  0.3863177 ]])
>>> Sig
array([[ 9.508032  ,  0.        ,  0.        ],
       [ 0.        ,  0.77286964,  0.        ]])
>>> Vh
array([[-0.42866713, -0.56630692, -0.7039467 ],
       [ 0.80596391,  0.11238241, -0.58119908],
       [ 0.40824829, -0.81649658,  0.40824829]])
>>> U.dot(Sig.dot(Vh)) #check computation
array([[ 1.,  2.,  3.],
       [ 4.,  5.,  6.]])
```

LU decomposition

The LU decomposition finds a representation for the $M \times N$ matrix $\mathbf{A}$ as

$$\mathbf{A} = \mathbf{P}\,\mathbf{L}\,\mathbf{U},$$

where $\mathbf{P}$ is an $M \times M$ permutation matrix (a permutation of the rows of the identity matrix), $\mathbf{L}$ is in $M \times K$ lower triangular or trapezoidal matrix ($K = \min(M, N)$) with unit-diagonal, and $\mathbf{U}$ is an upper triangular or trapezoidal matrix. The SciPy command for this decomposition is `linalg.lu`.

Such a decomposition is often useful for solving many simultaneous equations where the left-hand side does not change but the right-hand side does. For example, suppose we are going to solve

$$\mathbf{A}\mathbf{x}_i = \mathbf{b}_i$$

for many different $\mathbf{b}_i$. The LU decomposition allows this to be written as

$$\mathbf{PLU}\mathbf{x}_i = \mathbf{b}_i.$$

Because $\mathbf{L}$ is lower-triangular, the equation can be solved for $\mathbf{U}\mathbf{x}_i$ and, finally, $\mathbf{x}_i$ very rapidly using forward- and back-substitution. An initial time spent factoring $\mathbf{A}$ allows for very rapid solution of similar systems of equations in the future. If the intent for performing LU decomposition is for solving linear systems, then the command `linalg.lu_factor` should be used followed by repeated applications of the command `linalg.lu_solve` to solve the system for each new right-hand side.

Cholesky decomposition

Cholesky decomposition is a special case of LU decomposition applicable to Hermitian positive definite matrices. When $\mathbf{A} = \mathbf{A}^H$ and $\mathbf{x}^H\mathbf{A}\mathbf{x} \geq 0$ for all $\mathbf{x}$, then decompositions of $\mathbf{A}$ can be found so that

$$\begin{aligned}\mathbf{A} &= \mathbf{U}^H\mathbf{U} \\ \mathbf{A} &= \mathbf{L}\mathbf{L}^H,\end{aligned}$$

where $\mathbf{L}$ is lower triangular and $\mathbf{U}$ is upper triangular. Notice that $\mathbf{L} = \mathbf{U}^H$. The command `linalg.cholesky` computes the Cholesky factorization. For using the Cholesky factorization to solve systems of equations, there are also `linalg.cho_factor` and `linalg.cho_solve` routines that work similarly to their LU decomposition counterparts.

QR decomposition

The QR decomposition (sometimes called a polar decomposition) works for any $M \times N$ array and finds an $M \times M$ unitary matrix $\mathbf{Q}$ and an $M \times N$ upper-trapezoidal matrix $\mathbf{R}$, such that

$$\mathbf{A} = \mathbf{Q}\mathbf{R}.$$

Notice that if the SVD of $\mathbf{A}$ is known, then the QR decomposition can be found.

$$\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^H = \mathbf{Q}\mathbf{R}$$

implies that $\mathbf{Q} = \mathbf{U}$ and $\mathbf{R} = \Sigma\mathbf{V}^H$. Note, however, that in SciPy independent algorithms are used to find QR and SVD decompositions. The command for QR decomposition is `linalg.qr`.

Schur decomposition

For a square $N \times N$ matrix, $\mathbf{A}$, the Schur decomposition finds (not necessarily unique) matrices $\mathbf{T}$ and $\mathbf{Z}$, such that

$$\mathbf{A} = \mathbf{Z}\mathbf{T}\mathbf{Z}^H,$$

where $\mathbf{Z}$ is a unitary matrix and $\mathbf{T}$ is either upper triangular or quasi upper triangular, depending on whether or not a real Schur form or complex Schur form is requested. For a real Schur form both $\mathbf{T}$ and $\mathbf{Z}$ are real-valued when $\mathbf{A}$ is real-valued. When $\mathbf{A}$ is a real-valued matrix, the real Schur form is only quasi upper triangular because 2×2 blocks extrude from the main diagonal corresponding to any complex-valued eigenvalues. The command `linalg.schur` finds the Schur decomposition, while the command `linalg.rsf2csf` converts $\mathbf{T}$ and $\mathbf{Z}$ from a real Schur form to a complex Schur form. The Schur form is especially useful in calculating functions of matrices.

The following example illustrates the Schur decomposition:

```
>>> from scipy import linalg
>>> A = np.asmatrix('[1 3 2; 1 4 5; 2 3 6]')
>>> T, Z = linalg.schur(A)
>>> T1, Z1 = linalg.schur(A, 'complex')
>>> T2, Z2 = linalg.rsf2csf(T, Z)
>>> T
array([[ 9.90012467,  1.78947961, -0.65498528],
       [ 0.        ,  0.54993766, -1.57754789],
       [ 0.        ,  0.51260928,  0.54993766]])
```

```
>>> T2
array([[ 9.90012467+0.00000000e+00j, -0.32436598+1.55463542e+00j,
        -0.88619748+5.69027615e-01j],
       [ 0.        +0.00000000e+00j,  0.54993766+8.99258408e-01j,
         1.06493862+3.05311332e-16j],
       [ 0.        +0.00000000e+00j,  0.        +0.00000000e+00j,
         0.54993766-8.99258408e-01j]])
>>> abs(T1 - T2) # different
array([[  1.06604538e-14,   2.06969555e+00,   1.69375747e+00],  # may vary
       [  0.00000000e+00,   1.33688556e-15,   4.74146496e-01],
       [  0.00000000e+00,   0.00000000e+00,   1.13220977e-15]])
>>> abs(Z1 - Z2) # different
array([[ 0.06833781,  0.88091091,  0.79568503],    # may vary
       [ 0.11857169,  0.44491892,  0.99594171],
       [ 0.12624999,  0.60264117,  0.77257633]])
>>> T, Z, T1, Z1, T2, Z2 = map(np.asmatrix,(T,Z,T1,Z1,T2,Z2))
>>> abs(A - Z*T*Z.H)  # same
matrix([[  5.55111512e-16,   1.77635684e-15,   2.22044605e-15],
        [  0.00000000e+00,   3.99680289e-15,   8.88178420e-16],
        [  1.11022302e-15,   4.44089210e-16,   3.55271368e-15]])
>>> abs(A - Z1*T1*Z1.H)  # same
matrix([[  4.26993904e-15,   6.21793362e-15,   8.00007092e-15],
        [  5.77945386e-15,   6.21798014e-15,   1.06653681e-14],
        [  7.16681444e-15,   8.90271058e-15,   1.77635764e-14]])
>>> abs(A - Z2*T2*Z2.H)  # same
matrix([[  6.02594127e-16,   1.77648931e-15,   2.22506907e-15],
        [  2.46275555e-16,   3.99684548e-15,   8.91642616e-16],
        [  8.88225111e-16,   8.88312432e-16,   4.44104848e-15]])
```

Interpolative decomposition

`scipy.linalg.interpolative` contains routines for computing the interpolative decomposition (ID) of a matrix. For a matrix $A \in \mathbb{C}^{m\times n}$ of rank $k \leq \min\{m, n\}$ this is a factorization

$$A\Pi = [A\Pi_1 \quad A\Pi_2] = A\Pi_1 \begin{bmatrix} I & T \end{bmatrix},$$

where $\Pi = [\Pi_1, \Pi_2]$ is a permutation matrix with $\Pi_1 \in \{0, 1\}^{n\times k}$, i.e., $A\Pi_2 = A\Pi_1 T$. This can equivalently be written as $A = BP$, where $B = A\Pi_1$ and $P = [I, T]\Pi^{\mathsf{T}}$ are the *skeleton* and *interpolation matrices*, respectively.

Matrix functions

Consider the function $f(x)$ with Taylor series expansion

$$f(x) = \sum_{k=0}^{\infty} \frac{f^{(k)}(0)}{k!} x^k.$$

A matrix function can be defined using this Taylor series for the square matrix **A** as

$$f(\mathbf{A}) = \sum_{k=0}^{\infty} \frac{f^{(k)}(0)}{k!} \mathbf{A}^k.$$

Note
While this serves as a useful representation of a matrix function, it is rarely the best way to calculate a matrix function. In particular, if the matrix is not diagonalizable, results may be inaccurate.

Exponential and logarithm functions

The matrix exponential is one of the more common matrix functions. The preferred method for implementing the matrix exponential is to use scaling and a Padé approximation for e^x. This algorithm is implemented as `linalg.expm`.

The inverse of the matrix exponential is the matrix logarithm defined as the inverse of the matrix exponential:

$$\mathbf{A} \equiv \exp\left(\log\left(\mathbf{A}\right)\right).$$

The matrix logarithm can be obtained with `linalg.logm`.

Trigonometric functions

The trigonometric functions, sin, cos, and tan, are implemented for matrices in `linalg.sinm`, `linalg.cosm`, and `linalg.tanm`, respectively. The matrix sine and cosine can be defined using Euler's identity as

$$\begin{aligned}\sin\left(\mathbf{A}\right) &= \frac{e^{j\mathbf{A}} - e^{-j\mathbf{A}}}{2j} \\ \cos\left(\mathbf{A}\right) &= \frac{e^{j\mathbf{A}} + e^{-j\mathbf{A}}}{2}.\end{aligned}$$

The tangent is

$$\tan\left(x\right) = \frac{\sin\left(x\right)}{\cos\left(x\right)} = \left[\cos\left(x\right)\right]^{-1}\sin\left(x\right)$$

and so the matrix tangent is defined as

$$\left[\cos\left(\mathbf{A}\right)\right]^{-1}\sin\left(\mathbf{A}\right).$$

Hyperbolic trigonometric functions

The hyperbolic trigonometric functions, sinh, cosh, and tanh, can also be defined for matrices using the familiar definitions:

$$\begin{aligned}\sinh\left(\mathbf{A}\right) &= \frac{e^{\mathbf{A}} - e^{-\mathbf{A}}}{2} \\ \cosh\left(\mathbf{A}\right) &= \frac{e^{\mathbf{A}} + e^{-\mathbf{A}}}{2} \\ \tanh\left(\mathbf{A}\right) &= \left[\cosh\left(\mathbf{A}\right)\right]^{-1}\sinh\left(\mathbf{A}\right).\end{aligned}$$

These matrix functions can be found using `linalg.sinhm`, `linalg.coshm`, and `linalg.tanhm`.

Arbitrary function

Finally, any arbitrary function that takes one complex number and returns a complex number can be called as a matrix function using the command `linalg.funm`. This command takes the matrix and an arbitrary Python function. It then implements an algorithm from Golub and Van Loan's book "Matrix Computations" to compute the function applied to the matrix using a Schur decomposition. Note that *the function needs to accept complex numbers* as input in order to work with this algorithm. For example, the following code computes the zeroth-order Bessel function applied to a matrix.

```
>>> from scipy import special, linalg
>>> rng = np.random.default_rng()
>>> A = rng.random((3, 3))
>>> B = linalg.funm(A, lambda x: special.jv(0, x))
>>> A
array([[0.06369197, 0.90647174, 0.98024544],
       [0.68752227, 0.5604377 , 0.49142032],
       [0.86754578, 0.9746787 , 0.37932682]])
>>> B
array([[ 0.6929219 , -0.29728805, -0.15930896],
       [-0.16226043,  0.71967826, -0.22709386],
       [-0.19945564, -0.33379957,  0.70259022]])
>>> linalg.eigvals(A)
array([ 1.94835336+0.j, -0.72219681+0.j, -0.22270006+0.j])
>>> special.jv(0, linalg.eigvals(A))
array([0.25375345+0.j, 0.87379738+0.j, 0.98763955+0.j])
>>> linalg.eigvals(B)
array([0.25375345+0.j, 0.87379738+0.j, 0.98763955+0.j])
```

Note how, by virtue of how matrix analytic functions are defined, the Bessel function has acted on the matrix eigenvalues.

Special matrices

SciPy and NumPy provide several functions for creating special matrices that are frequently used in engineering and science.

Type	Function	Description
block diagonal	`scipy.linalg.block_diag`	Create a block diagonal matrix from the provided arrays.
circulant	`scipy.linalg.circulant`	Create a circulant matrix.
companion	`scipy.linalg.companion`	Create a companion matrix.
convolution	`scipy.linalg.convolution_matrix`	Create a convolution matrix.
Discrete Fourier	`scipy.linalg.dft`	Create a discrete Fourier transform matrix.
Fiedler	`scipy.linalg.fiedler`	Create a symmetric Fiedler matrix.
Fiedler Companion	`scipy.linalg.fiedler_companion`	Create a Fiedler companion matrix.
Hadamard	`scipy.linalg.hadamard`	Create a Hadamard matrix.

Type	Function	Description
Hankel	`scipy.linalg.hankel`	Create a Hankel matrix.
Helmert	`scipy.linalg.helmert`	Create a Helmert matrix.
Hilbert	`scipy.linalg.hilbert`	Create a Hilbert matrix.
Inverse Hilbert	`scipy.linalg.invhilbert`	Create the inverse of a Hilbert matrix.
Leslie	`scipy.linalg.leslie`	Create a Leslie matrix.
Pascal	`scipy.linalg.pascal`	Create a Pascal matrix.
Inverse Pascal	`scipy.linalg.invpascal`	Create the inverse of a Pascal matrix.
Toeplitz	`scipy.linalg.toeplitz`	Create a Toeplitz matrix.
Van der Monde	`numpy.vander`	Create a Van der Monde matrix.

For examples of the use of these functions, see their respective docstrings.

Advanced Features

Batch Support

Some of SciPy's linear algebra functions can process batches of scalars, 1D-, or 2D-arrays given N-d array input. For example, a linear algebra function that typically accepts a (2D) matrix may accept an array of shape `(4, 3, 2)`, which it would interpret as a batch of four 3-by-2 matrices. In this case, we say that the the *core shape* of the input is (3, 2) and the *batch shape* is `(4,)`. Likewise, a linear algebra function that typically accepts a (1D) vector would treat a `(4, 3, 2)` array as a `(4, 3)` batch of vectors, in which case the *core shape* of the input is `(2,)` and the *batch shape* is `(4, 3)`. The length of the core shape is also referred to as the *core dimension*. In these cases, the final shape of the output is the batch shape of the input concatenated with the core shape of the output (i.e., the shape of the output when the batch shape of the input is `()`). For more information, see linalg_batch.

`overwrite_*` arguments

Many linear algebra functions have `overwrite_*` arguments to signal that an operation is allowed to work in-place and overwrite the input array (the naming convention is that an array argument `a` has a matching `overwrite_a`; a `b` argument has a matching `overwrite_b` and so on).

By default, `scipy.linalg` functions preserve their inputs and make copies internally. In some cases, working in-place may improve performance or avoid running out of memory---it is however advisable to measure and make sure you actually see improvements for your specific workloads.

Note however that setting `overwrite_a=True` only indicates that it *may* work in-place; whether it actually *does* depends on whether additional requirements are met. For most functions, an in-place operation requires that all of the following conditions are true:

- the dtype of the input array is LAPACK-compatible: only single and double precision float (`float{32,64}`) and complex (`complex{64,128}`) arrays are compatible;
- the array is Fortran-ordered;

- currently only two-dimensional arrays, `ndim == 2`, are compatible; batched arrays use an internal buffer of the size of the core shaped array. This however may change in a future SciPy version.

If any of these conditions are violated, a copy is made under the hood, regardless of whether `overwrite_a` is `True` or `False`.

Some functions may relax some of these requirements or impose additional constraints for working in-place.

Note that this all is completely opaque: if you set `overwrite_a=True` but the function has to operate on a copy, no warnings or diagnostic output will be generated.

As an illustration, consider a simple inversion operation:

```
>>> import numpy as np
>>> from scipy.linalg import inv
>>> a = np.asarray([[2, 1], [0, 1]])
>>> a_inv = inv(a)
>>> a_inv
array([[ 0.5, -0.5],
       [ 0. ,  1. ]])
>>> a
array([[2, 1],
       [0, 1]])
```

Since `a` is of integer dtype, the `scipy.linalg.inv` function internally makes a floating-point copy, which it feeds into the LAPACK's inversion routine. Thus setting `overwrite_a` to `True` makes no difference:

```
>>> a_inv = inv(a, overwrite_a=True)
>>> a_inv
array([[ 0.5, -0.5],
       [ 0. ,  1. ]])
>>> a
array([[2, 1],
       [0, 1]])
```

To force it work in-place, we need to manually convert the `a` array to be Fortran-ordered and dtype-compatible:

```
>>> af = np.asarray(a, order='F', dtype=np.float64)
>>> a_inv = inv(af, overwrite_a=True)
>>> a_inv
array([[ 0.5, -0.5],
       [ 0. ,  1. ]])
>>> af
array([[ 0.5, -0.5],
       [ 0. ,  1. ]])
```

Note that the input array, `af`, has now been overwritten. We can check that the input and output arrays actually use the same memory buffer:

```
>>> np.shares_memory(af, a_inv)
True
```

In general, it may or may not hold that `a_inv is af`: the only guarantee is that inputs and outputs share *some* part of the memory buffer.

Finally, we recommend that you only use these `overwrite_a` arguments if performance / memory measurements show a significant improvement. In the vast majority of usages, adding them is a premature optimization.

Sparse arrays (scipy.sparse)

Source: https://docs.scipy.org/doc/scipy/tutorial/sparse.html

Introduction

`scipy.sparse` and its submodules provide tools for working with *sparse arrays*. Sparse arrays are arrays where only a few locations in the array have any data, most of the locations are considered as "empty". Sparse arrays are useful because they allow for simpler, faster, and/or less memory-intensive algorithms for linear algebra (`scipy.sparse.linalg`) or graph-based computations (`scipy.sparse.csgraph`), but they are generally less flexible for operations like slicing, reshaping, or assignment. This guide will introduce the basics of sparse arrays in `scipy.sparse`, explain the unique aspects of sparse data structures, and refer onward for other sections of the user guide explaining sparse linear algebra and graph methods.

Getting started with sparse arrays

Sparse arrays are a special kind of array where only a few locations in the array have data. This allows for *compressed* representations of the data to be used, where only the locations where data exists are recorded. There are many different sparse array formats, each of which makes a different tradeoff between compression and functionality. To start, let's build a very simple sparse array, the Coordinate (COO) array (`coo_array`) and compare it to a dense array:

```
>>> import scipy as sp
>>> import numpy as np
>>> dense = np.array([[1, 0, 0, 2], [0, 4, 1, 0], [0, 0, 5, 0]])
>>> sparse = sp.sparse.coo_array(dense)
>>> dense
array([[1, 0, 0, 2],
    [0, 4, 1, 0],
    [0, 0, 5, 0]])
>>> sparse
<COOrdinate sparse array of dtype 'int64'
     with 5 stored elements and shape (3, 4)>
```

Note that in our dense array, we have five nonzero values. For example, `2` is at location `0,3`, and `4` is at location `1,1`. All of the other values are zero. The sparse array records these five values *explicitly* (see the `5 stored elements and shape (3, 4)`), and then represents all of the remaining zeros as *implicit* values.

Most sparse array methods work in a similar fashion to dense array methods:

```
>>> sparse.max()
5
>>> dense.max()
5
>>> sparse.argmax()
10
>>> dense.argmax()
10
>>> sparse.mean()
1.0833333333333333
```

```
>>> dense.mean()
1.0833333333333333
```

A few ”extra” properties, such as `.nnz` which returns the number of stored values, are present on sparse arrays as well:

```
>>> sparse.nnz
5
```

Most of the reduction operations, such as `.mean()`, `.sum()`, or `.max()` will return a numpy array when applied over an axis of the sparse array:

```
>>> sparse.mean(axis=1)
array([0.75, 1.25, 1.25])
```

This is because reductions over sparse arrays are often dense.

Understanding sparse array formats

Different kinds of sparse arrays have different capabilities. For example, DIA arrays cannot be subscripted or sliced:

```
>>> dense[2, 2]
5
>>> sparse.todia()[2, 2]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'dia_array' object is not subscriptable
```

But, other formats, such as the Compressed Sparse Row (CSR) `csr_array()` support slicing and element indexing:

```
>>> sparse.tocsr()[2, 2]
5
```

Sometimes, `scipy.sparse` will return a different sparse matrix format than the input sparse matrix format. For example, the dot product of two sparse arrays in COO format will be a CSR format array:

```
>>> sparse @ sparse.T
<Compressed Sparse Row sparse array of dtype 'int64'
     with 5 stored elements and shape (3, 3)>
```

This change occurs because `scipy.sparse` will change the format of input sparse arrays in order to use the most efficient computational method.

The `scipy.sparse` module contains the following formats, each with their own distinct advantages and disadvantages:

- Block Sparse Row (BSR) arrays `scipy.sparse.bsr_array()`, which are most appropriate when the parts of the array with data occur in contiguous blocks.
- Coordinate (COO) arrays `scipy.sparse.coo_array()`, which provide a simple way to construct sparse arrays and modify them in place. COO can also be quickly converted into other formats, such CSR, CSC, or BSR.
- Compressed Sparse Row (CSR) arrays `scipy.sparse.csr_array()`, which are most useful for fast arithmetic, vector products, and slicing by row.
- Compressed Sparse Column (CSC) arrays `scipy.sparse.csc_array()`, which are most useful for fast arithmetic, vector products, and slicing by column.

- Diagonal (DIA) arrays `scipy.sparse.dia_array()`, which are useful for efficient storage and fast arithmetic so long as the data primarily occurs along diagonals of the array.
- Dictionary of Keys (DOK) arrays `scipy.sparse.dok_array()`, which are useful for fast construction and single-element access.
- List of Lists (LIL) arrays `scipy.sparse.lil_array()`, which are useful for fast construction and modification of sparse arrays.

More information on the strengths and weaknesses of each of the sparse array formats can be found in their documentation.

All formats of `scipy.sparse` arrays can be constructed directly from a `numpy.ndarray`. However, some sparse formats can be constructed in different ways, too. Each sparse array format has different strengths, and these strengths are documented in each class. For example, one of the most common methods for constructing sparse arrays is to build a sparse array from the individual `row`, `column`, and `data` values. For our array from before:

```
>>> dense
array([[1, 0, 0, 2],
    [0, 4, 1, 0],
    [0, 0, 5, 0]])
```

The `row`, `column`, and `data` arrays describe the rows, columns, and values where our sparse array has entries:

```
>>> row = [0,0,1,1,2]
>>> col = [0,3,1,2,2]
>>> data = [1,2,4,1,5]
```

Using these, we can now define a sparse array without building a dense array first:

```
>>> csr = sp.sparse.csr_array((data, (row, col)))
>>> csr
<Compressed Sparse Row sparse array of dtype 'int64'
     with 5 stored elements and shape (3, 4)>
```

Different classes have different constructors, but the `scipy.sparse.csr_array`, `scipy.sparse.csc_array`, and `scipy.sparse.coo_array` allow for this style of construction.

Sparse arrays, implicit zeros, and duplicates

Sparse arrays are useful because they represent much of their values *implicitly*, without storing an actual placeholder value. In `scipy.sparse`, the value used to represent "no data" is an *implicit zero*. This can be confusing when *explicit zeros* are required. For example, in graph methods from `scipy.sparse.csgraph`, we often need to be able to distinguish between (A) a link connecting nodes `i` and `j` with zero weight and (B) no link between `i` and `j`. Sparse matrices can do this, so long as we keep the *explicit* and *implicit* zeros in mind.

For example, in our previous `csr` array, we could include an explicit zero by including it in the `data` list. Let's treat the final entry in the array at the bottom row and last column as an *explicit zero*:

```
>>> row = [0,0,1,1,2,2]
>>> col = [0,3,1,2,2,3]
>>> data = [1,2,4,1,5,0]
```

Then, our sparse array will have *six* stored elements, not five:

```
>>> csr = sp.sparse.csr_array((data, (row, col)))
>>> csr
```

```
<Compressed Sparse Row sparse array of dtype 'int64'
     with 6 stored elements and shape (3, 4)>
```

The "extra" element is our *explicit zero*. The two are still identical when converted back into a dense array, because dense arrays represent *everything* explicitly:

```
>>> csr.todense()
array([[1, 0, 0, 2],
    [0, 4, 1, 0],
    [0, 0, 5, 0]])
>>> dense
array([[1, 0, 0, 2],
    [0, 4, 1, 0],
    [0, 0, 5, 0]])
```

But, for sparse arithmetic, linear algebra, and graph methods, the value at `2,3` will be considered an *explicit zero*. To remove this explicit zero, we can use the `csr.eliminate_zeros()` method. This operates on the sparse array *in place*, and removes any zero-value stored elements:

```
>>> csr
<Compressed Sparse Row sparse array of dtype 'int64'
     with 6 stored elements and shape (3, 4)>
>>> csr.eliminate_zeros()
>>> csr
<Compressed Sparse Row sparse array of dtype 'int64'
     with 5 stored elements and shape (3, 4)>
```

Before `csr.eliminate_zeros()`, there were six stored elements. After, there are only five stored elements.

Another point of complication arises from how *duplicates* are processed when constructing a sparse array. A *duplicate* can occur when we have two or more entries at `row,col` when constructing a sparse array. This often occurs when building sparse arrays using the `data`, `row`, and `col` vectors. For example, we might represent our previous array with a duplicate value at `1,1`:

```
>>> row = [0,0,1,1,1,2]
>>> col = [0,3,1,1,2,2]
>>> data = [1,2,1,3,1,5]
```

In this case, we can see that there are *two* `data` values that correspond to the `1,1` location in our final array. `scipy.sparse` will store these values separately:

```
>>> dupes = sp.sparse.coo_array((data, (row, col)))
>>> dupes
<COOrdinate sparse array of dtype 'int64'
     with 6 stored elements and shape (3, 4)>
```

Note that there are six stored elements in this sparse array, despite only having five unique locations where data occurs. When these arrays are converted back to dense arrays, the duplicate values are summed. So, at location `1,1`, the dense array will contain the sum of duplicate stored entries, `1 + 3`:

```
>>> dupes.todense()
array([[1, 0, 0, 2],
      [0, 4, 1, 0],
      [0, 0, 5, 0]])
```

To remove duplicate values within the sparse array itself and thus reduce the number of stored elements, we can use the `.sum_duplicates()` method:

```
>>> dupes.sum_duplicates()
>>> dupes
<COOrdinate sparse array of dtype 'int64'
     with 5 stored elements and shape (3, 4)>
```

Now there are only five stored elements in our sparse array, and it is identical to the array we have been working with throughout this guide:

```
>>> dupes.todense()
array([[1, 0, 0, 2],
       [0, 4, 1, 0],
       [0, 0, 5, 0]])
```

Canonical formats

Several sparse array formats have ”canonical formats” to allow for more efficient operations. Generally these consist of added restrictions like:

- No duplicate entries for any value
- Sorted indices

Classes with a canonical form include: `coo_array`, `csr_array`, `csc_array`, and `bsr_array`. See the docstrings of these classes for details on each canonical representation.

To check if an instance of these classes is in canonical form, use the `.has_canonical_format` attribute:

```
>>> coo = sp.sparse.coo_array(([1, 1, 1], ([0, 2, 1], [0, 1, 2])))
>>> coo.has_canonical_format
False
```

To convert an instance to canonical form, use the `.sum_duplicates()` method:

```
>>> coo.sum_duplicates()
>>> coo.has_canonical_format
True
```

Next steps with sparse arrays

Sparse array types are most helpful when working with large, nearly empty arrays. Specifically, sparse linear algebra and sparse graph methods see the largest improvements in efficiency in these circumstances.

Spatial Data Structures and Algorithms (`scipy.spatial`) ==========

`scipy.spatial` can compute triangulations, Voronoi diagrams, and convex hulls of a set of points, by leveraging the Qhull library.

Moreover, it contains `KDTree` implementations for nearest-neighbor point queries, and utilities for distance computations in various metrics.

Spatial data structures and algorithms (scipy.spatial)

Source: https://docs.scipy.org/doc/scipy/tutorial/spatial.html

The Delaunay triangulation is a subdivision of a set of points into a non-overlapping set of triangles, such that no point is inside the circumcircle of any triangle. In practice, such triangulations tend to avoid triangles with small angles.

Delaunay triangulation can be computed using `scipy.spatial` as follows:

```
>>> from scipy.spatial import Delaunay
>>> import numpy as np
>>> points = np.array([[0, 0], [0, 1.1], [1, 0], [1, 1]])
>>> tri = Delaunay(points)
```

We can visualize it:

```
>>> import matplotlib.pyplot as plt
>>> plt.triplot(points[:,0], points[:,1], tri.simplices)
>>> plt.plot(points[:,0], points[:,1], 'o')
```

And add some further decorations:

```
>>> for j, p in enumerate(points):
...     plt.text(p[0]-0.03, p[1]+0.03, j, ha='right') # label the points
>>> for j, s in enumerate(tri.simplices):
...     p = points[s].mean(axis=0)
...     plt.text(p[0], p[1], '#%d' % j, ha='center') # label triangles
>>> plt.xlim(-0.5, 1.5); plt.ylim(-0.5, 1.5)
>>> plt.show()
```

The structure of the triangulation is encoded in the following way: the `simplices` attribute contains the indices of the points in the `points` array that make up the triangle. For instance:

```
>>> i = 1
>>> tri.simplices[i,:]
array([3, 1, 0], dtype=int32)
>>> points[tri.simplices[i,:]]
array([[ 1. ,  1. ],
       [ 0. ,  1.1],
       [ 0. ,  0. ]])
```

Moreover, neighboring triangles can also be found:

```
>>> tri.neighbors[i]
array([-1,  0, -1], dtype=int32)
```

What this tells us is that this triangle has triangle #0 as a neighbor, but no other neighbors. Moreover, it tells us that neighbor 0 is opposite the vertex 1 of the triangle:

```
>>> points[tri.simplices[i, 1]]
array([ 0. ,  1.1])
```

Indeed, from the figure, we see that this is the case.

Qhull can also perform tessellations to simplices for higher-dimensional point sets (for instance, subdivision into tetrahedra in 3-D).

Coplanar points

It is important to note that not *all* points necessarily appear as vertices of the triangulation, due to numerical precision issues in forming the triangulation. Consider the above with a duplicated point:

```
>>> points = np.array([[0, 0], [0, 1], [1, 0], [1, 1], [1, 1]])
>>> tri = Delaunay(points)
>>> np.unique(tri.simplices.ravel())
array([0, 1, 2, 3], dtype=int32)
```

Observe that point #4, which is a duplicate, does not occur as a vertex of the triangulation. That this happened is recorded:

```
>>> tri.coplanar
array([[4, 0, 3]], dtype=int32)
```

This means that point 4 resides near triangle 0 and vertex 3, but is not included in the triangulation.

Note that such degeneracies can occur not only because of duplicated points, but also for more complicated geometrical reasons, even in point sets that at first sight seem well-behaved.

However, Qhull has the ”QJ” option, which instructs it to perturb the input data randomly until degeneracies are resolved:

```
>>> tri = Delaunay(points, qhull_options="QJ Pp")
>>> points[tri.simplices]
array([[[1, 0],
        [1, 1],
        [0, 0]],
       [[1, 1],
        [1, 1],
        [1, 0]],
       [[1, 1],
        [0, 1],
        [0, 0]],
       [[0, 1],
        [1, 1],
        [1, 1]]])
```

Two new triangles appeared. However, we see that they are degenerate and have zero area.

Convex hulls

A convex hull is the smallest convex object containing all points in a given point set.

These can be computed via the Qhull wrappers in `scipy.spatial` as follows:

```
>>> from scipy.spatial import ConvexHull
>>> rng = np.random.default_rng()
>>> points = rng.random((30, 2))   # 30 random points in 2-D
>>> hull = ConvexHull(points)
```

The convex hull is represented as a set of N 1-D simplices, which in 2-D means line segments. The storage scheme is exactly the same as for the simplices in the Delaunay triangulation discussed

above.

We can illustrate the above result:

```
>>> import matplotlib.pyplot as plt
>>> plt.plot(points[:,0], points[:,1], 'o')
>>> for simplex in hull.simplices:
...     plt.plot(points[simplex,0], points[simplex,1], 'k-')
>>> plt.show()
```

The same can be achieved with `scipy.spatial.convex_hull_plot_2d`.

Voronoi diagrams

A Voronoi diagram is a subdivision of the space into the nearest neighborhoods of a given set of points.

There are two ways to approach this object using `scipy.spatial`. First, one can use the `KDTree` to answer the question "which of the points is closest to this one", and define the regions that way:

```
>>> from scipy.spatial import KDTree
>>> points = np.array([[0, 0], [0, 1], [0, 2], [1, 0], [1, 1], [1, 2],
...                    [2, 0], [2, 1], [2, 2]])
>>> tree = KDTree(points)
>>> tree.query([0.1, 0.1])
(0.14142135623730953, 0)
```

So the point `(0.1, 0.1)` belongs to region `0`. In color:

```
>>> x = np.linspace(-0.5, 2.5, 31)
>>> y = np.linspace(-0.5, 2.5, 33)
>>> xx, yy = np.meshgrid(x, y)
>>> xy = np.c_[xx.ravel(), yy.ravel()]
>>> import matplotlib.pyplot as plt
>>> dx_half, dy_half = np.diff(x[:2])[0] / 2., np.diff(y[:2])[0] / 2.
>>> x_edges = np.concatenate((x - dx_half, [x[-1] + dx_half]))
>>> y_edges = np.concatenate((y - dy_half, [y[-1] + dy_half]))
>>> plt.pcolormesh(x_edges, y_edges, tree.query(xy)[1].reshape(33, 31), shading='flat')
>>> plt.plot(points[:,0], points[:,1], 'ko')
>>> plt.show()
```

This does not, however, give the Voronoi diagram as a geometrical object.

The representation in terms of lines and points can be again obtained via the Qhull wrappers in `scipy.spatial`:

```
>>> from scipy.spatial import Voronoi
>>> vor = Voronoi(points)
>>> vor.vertices
array([[0.5, 0.5],
       [0.5, 1.5],
       [1.5, 0.5],
       [1.5, 1.5]])
```

The Voronoi vertices denote the set of points forming the polygonal edges of the Voronoi regions. In this case, there are 9 different regions:

```
>>> vor.regions
[[], [-1, 0], [-1, 1], [1, -1, 0], [3, -1, 2], [-1, 3], [-1, 2], [0, 1, 3, 2], [2, -1, 0], [3, -1, 1]]
```

Negative value `-1` again indicates a point at infinity. Indeed, only one of the regions, `[0, 1, 3, 2]`, is bounded. Note here that due to similar numerical precision issues as in Delaunay triangulation above, there may be fewer Voronoi regions than input points.

The ridges (lines in 2-D) separating the regions are described as a similar collection of simplices as the convex hull pieces:

```
>>> vor.ridge_vertices
[[-1, 0], [-1, 0], [-1, 1], [-1, 1], [0, 1], [-1, 3], [-1, 2], [2, 3], [-1, 3], [-1, 2], [1, 3], [0, 2]]
```

These numbers present the indices of the Voronoi vertices making up the line segments. `-1` is again a point at infinity --- only 4 of the 12 lines are a bounded line segment, while others extend to infinity.

The Voronoi ridges are perpendicular to the lines drawn between the input points. To which two points each ridge corresponds is also recorded:

```
>>> vor.ridge_points
array([[0, 3],
       [0, 1],
       [2, 5],
       [2, 1],
       [1, 4],
       [7, 8],
       [7, 6],
       [7, 4],
       [8, 5],
       [6, 3],
       [4, 5],
       [4, 3]], dtype=int32)
```

This information, taken together, is enough to construct the full diagram.

We can plot it as follows. First, the points and the Voronoi vertices:

```
>>> plt.plot(points[:, 0], points[:, 1], 'o')
>>> plt.plot(vor.vertices[:, 0], vor.vertices[:, 1], '*')
>>> plt.xlim(-1, 3); plt.ylim(-1, 3)
```

Plotting the finite line segments goes as for the convex hull, but now we have to guard for the infinite edges:

```
>>> for simplex in vor.ridge_vertices:
...     simplex = np.asarray(simplex)
...     if np.all(simplex >= 0):
...         plt.plot(vor.vertices[simplex, 0], vor.vertices[simplex, 1], 'k-')
```

The ridges extending to infinity require a bit more care:

```
>>> center = points.mean(axis=0)
>>> for pointidx, simplex in zip(vor.ridge_points, vor.ridge_vertices):
...     simplex = np.asarray(simplex)
...     if np.any(simplex < 0):
...         i = simplex[simplex >= 0][0] # finite end Voronoi vertex
...         t = points[pointidx[1]] - points[pointidx[0]]  # tangent
...         t = t / np.linalg.norm(t)
...         n = np.array([-t[1], t[0]]) # normal
...         midpoint = points[pointidx].mean(axis=0)
...         far_point = vor.vertices[i] + np.sign(np.dot(midpoint - center, n)) * n * 100
...         plt.plot([vor.vertices[i,0], far_point[0]],
...                  [vor.vertices[i,1], far_point[1]], 'k--')
>>> plt.show()
```

This plot can also be created using `scipy.spatial.voronoi_plot_2d`.

Voronoi diagrams can be used to create interesting generative art. Try playing with the settings of this `mandala` function to create your own!

```
>>> import numpy as np
>>> from scipy import spatial
>>> import matplotlib.pyplot as plt

>>> def mandala(n_iter, n_points, radius):
...     """Creates a mandala figure using Voronoi tessellations.
...
...     Parameters
...     ----------
...     n_iter : int
...         Number of iterations, i.e. how many times the equidistant points will
...         be generated.
...     n_points : int
...         Number of points to draw per iteration.
...     radius : scalar
...         The radial expansion factor.
...
...     Returns
...     -------
...     fig : matplotlib.Figure instance
...
...     Notes
...     -----
...     This code is adapted from the work of Audrey Roy Greenfeld [1]_ and Carlos
...     Focil-Espinosa [2]_, who created beautiful mandalas with Python code.  That
...     code in turn was based on Antonio Sánchez Chinchón's R code [3]_.
...
...     References
...     ----------
...     .. [1] https://www.codemakesmehappy.com/2019/09/voronoi-mandalas.html
...
...     .. [2] https://github.com/CarlosFocil/mandalapy
...
...     .. [3] https://github.com/aschinchon/mandalas
...
...     """
...     fig = plt.figure(figsize=(10, 10))
...     ax = fig.add_subplot(111)
...     ax.set_axis_off()
...     ax.set_aspect('equal', adjustable='box')
...
...     angles = np.linspace(0, 2*np.pi * (1 - 1/n_points), num=n_points) + np.pi/2
...     # Starting from a single center point, add points iteratively
...     xy = np.array([[0, 0]])
...     for k in range(n_iter):
...         t1 = np.array([])
...         t2 = np.array([])
...         # Add ``n_points`` new points around each existing point in this iteration
...         for i in range(xy.shape[0]):
...             t1 = np.append(t1, xy[i, 0] + radius**k * np.cos(angles))
...             t2 = np.append(t2, xy[i, 1] + radius**k * np.sin(angles))
...
...         xy = np.column_stack((t1, t2))
...
```

```
...     # Create the Mandala figure via a Voronoi plot
...     spatial.voronoi_plot_2d(spatial.Voronoi(xy), ax=ax)
...
...     return fig

>>> # Modify the following parameters in order to get different figures
>>> n_iter = 3
>>> n_points = 6
>>> radius = 4

>>> fig = mandala(n_iter, n_points, radius)
>>> plt.show()
```

API quick reference

Generated from the docstrings of the installed SciPy 1.18.1 package, which are the text the official API reference pages are built from. Full reference: https://docs.scipy.org/doc/scipy/reference/index.html

Statistics: describing and testing

scipy.stats.describe

```
scipy.stats.describe(a, axis=0, ddof=1, bias=True, nan_policy='propagate')
```

Compute several descriptive statistics of the passed array.

Parameters

a : *array_like* Input data.

axis : *int or None, optional* Axis along which statistics are calculated. Default is 0. If None, compute over the whole array a.

ddof : *int, optional* Delta degrees of freedom (only for variance). Default is 1.

bias : *bool, optional* If False, then the skewness and kurtosis calculations are corrected for statistical bias.

nan_policy : *{'propagate', 'raise', 'omit'}, optional* Defines how to handle when input contains nan. The following options are available (default is 'propagate'):

- 'propagate': returns nan
- 'raise': throws an error
- 'omit': performs the calculations ignoring nan values

Returns

nobs : *int or ndarray of ints* Number of observations (length of data along axis). When 'omit' is chosen as nan_policy, the length along each axis slice is counted separately.

minmax: tuple of ndarrays or floats Minimum and maximum value of a along the given axis.

mean : *ndarray or float* Arithmetic mean of a along the given axis.

variance : *ndarray or float* Unbiased variance of a along the given axis; denominator is number of observations minus one.

skewness : *ndarray or float* Skewness of a along the given axis, based on moment calculations with denominator equal to the number of observations, i.e. no degrees of freedom correction.

kurtosis : *ndarray or float* Kurtosis (Fisher) of a along the given axis. The kurtosis is normalized so that it is zero for the normal distribution. No degrees of freedom are used.

Raises

ValueError If size of a is 0.

See also: skew, kurtosis

Notes

Array API Standard Support

`describe` has experimental support for Python Array API Standard compatible backends in addition to NumPy. Please consider testing these features by setting an environment variable `SCIPY_ARRAY_API=1` and providing CuPy, PyTorch, JAX, or Dask arrays as array arguments. The following combinations of backend and device (or other capability) are supported.

(Notes truncated; see the full reference page.)

Examples

```
>>> from scipy import stats
>>> a = np.arange(10)
>>> stats.describe(a)
DescribeResult(nobs=10, minmax=(0, 9), mean=4.5,
               variance=9.166666666666666, skewness=0.0,
               kurtosis=-1.2242424242424244)
>>> b = [[1, 2], [3, 4]]
>>> stats.describe(b)
DescribeResult(nobs=2, minmax=(array([1, 2]), array([3, 4])),
               mean=array([2., 3.]), variance=array([2., 2.]),
               skewness=array([0., 0.]), kurtosis=array([-2., -2.]))
```

scipy.stats.zscore

```
scipy.stats.zscore(a, axis=0, ddof=0, nan_policy='propagate')
```

Compute the z score.

Compute the z score of each value in the sample, relative to the sample mean and standard deviation.

Parameters

`a` : *array_like* An array like object containing the sample data.

`axis` : *int or None, optional* Axis along which to operate. Default is 0. If None, compute over the whole array `a`.

`ddof` : *int, optional* Degrees of freedom correction in the calculation of the standard deviation. Default is 0.

`nan_policy` : *{'propagate', 'raise', 'omit'}, optional* Defines how to handle when input contains nan. 'propagate' returns nan, 'raise' throws an error, 'omit' performs the calculations ignoring nan values. Default is 'propagate'. Note that when the value is 'omit', nans in the input also propagate to the output, but they do not affect the z-scores computed for the non-nan values.

Returns

`zscore` : *array_like* The z-scores, standardized by mean and standard deviation of input array `a`.

See also: `numpy.mean`, `numpy.std`, `scipy.stats.gzscore`

Notes

This function preserves ndarray subclasses, and works also with matrices and masked arrays (it uses `asanyarray` instead of `asarray` for parameters).

Array API Standard Support

`zscore` has experimental support for Python Array API Standard compatible backends in addition to NumPy. Please consider testing these features by setting an environment variable

SCIPY_ARRAY_API=1 and providing CuPy, PyTorch, JAX, or Dask arrays as array arguments. The following combinations of backend and device (or other capability) are supported.

(Notes truncated; see the full reference page.)

Examples

```
>>> a = np.array([ 0.7972,  0.0767,  0.4383,  0.7866,  0.8091,
...                0.1954,  0.6307,  0.6599,  0.1065,  0.0508])
>>> from scipy import stats
>>> stats.zscore(a)
array([ 1.1273, -1.247 , -0.0552,  1.0923,  1.1664, -0.8559,  0.5786,
        0.6748, -1.1488, -1.3324])
```

Computing along a specified axis, using n-1 degrees of freedom (ddof=1) to calculate the standard deviation:

```
>>> b = np.array([[ 0.3148,  0.0478,  0.6243,  0.4608],
...               [ 0.7149,  0.0775,  0.6072,  0.9656],
...               [ 0.6341,  0.1403,  0.9759,  0.4064],
...               [ 0.5918,  0.6948,  0.904 ,  0.3721],
...               [ 0.0921,  0.2481,  0.1188,  0.1366]])
>>> stats.zscore(b, axis=1, ddof=1)
array([[-0.19264823, -1.28415119,  1.07259584,  0.40420358],
       [ 0.33048416, -1.37380874,  0.04251374,  1.00081084],
       [ 0.26796377, -1.12598418,  1.23283094, -0.37481053],
       [-0.22095197,  0.24468594,  1.19042819, -1.21416216],
       [-0.82780366,  1.4457416 , -0.43867764, -0.1792603 ]])
...
```

scipy.stats.ttest_ind

```
scipy.stats.ttest_ind(a, b, *, axis=0, equal_var=True, nan_policy='propagate', alternative='two-sided',
↪  trim=0, method=None, keepdims=False)
```

Calculate the T-test for the means of *two independent* samples of scores.

This is a test for the null hypothesis that 2 independent samples have identical average (expected) values. This test assumes that the populations have identical variances by default.

Parameters

a, b : *array_like* The arrays must have the same shape, except in the dimension corresponding to axis (the first, by default).

axis : *int or None, default: 0* If an int, the axis of the input along which to compute the statistic. The statistic of each axis-slice (e.g. row) of the input will appear in a corresponding element of the output. If None, the input will be raveled before computing the statistic.

equal_var : *bool, optional* If True (default), perform a standard independent 2 sample test that assumes equal population variances. If False, perform Welch's t-test, which does not assume equal population variance.

New in version 0.11.0.

nan_policy : *{'propagate', 'omit', 'raise'}* Defines how to handle input NaNs.

- propagate: if a NaN is present in the axis slice (e.g. row) along which the statistic is computed, the corresponding entry of the output will be NaN.
- omit: NaNs will be omitted when performing the calculation. If insufficient data remains in the axis slice along which the statistic is computed, the corresponding entry of the output will be NaN.

- `raise`: if a NaN is present, a `ValueError` will be raised.

`alternative` : *{'two-sided', 'less', 'greater'}, optional* Defines the alternative hypothesis. The following options are available (default is 'two-sided'):

- 'two-sided': the means of the distributions underlying the samples are unequal.
- 'less': the mean of the distribution underlying the first sample is less than the mean of the distribution underlying the second sample.
- 'greater': the mean of the distribution underlying the first sample is greater than the mean of the distribution underlying the second sample.

`trim` : *float, optional* If nonzero, performs a trimmed (Yuen's) t-test. Defines the fraction of elements to be trimmed from each end of the input samples. If 0 (default), no elements will be trimmed from either side. The number of trimmed elements from each tail is the floor of the trim times the number of elements. Valid range is [0, .5).

`method` : *ResamplingMethod, optional* Defines the method used to compute the p-value. If `method` is an instance of `PermutationMethod`/`MonteCarloMethod`, the p-value is computed using `scipy.stats.permutation_test`/`scipy.stats.monte_carlo_test` with the provided configuration options and other appropriate settings. Otherwise, the p-value is computed by comparing the test statistic against a theoretical t-distribution.

New in version 1.15.0.

`keepdims` : *bool, default: False* If this is set to True, the axes which are reduced are left in the result as dimensions with size one. With this option, the result will broadcast correctly against the input array.

Returns

`result` : *"TtestResult"* An object with the following attributes:

statistic : float or ndarray The t-statistic.

pvalue : float or ndarray The p-value associated with the given alternative.

df : float or ndarray The number of degrees of freedom used in calculation of the t-statistic.

New in version 1.11.0.

The object also has the following method:

confidence_interval(confidence_level=0.95) Computes a confidence interval around the difference in population means for the given confidence level. The confidence interval is returned in a `namedtuple` with fields `low` and `high`.

New in version 1.11.0.

Notes

Suppose we observe two independent samples, e.g. flower petal lengths, and we are considering whether the two samples were drawn from the same population (e.g. the same species of flower or two species with similar petal characteristics) or two different populations.

(Notes truncated; see the full reference page.)

Examples

```
>>> from scipy import stats
>>> rng = np.random.default_rng()
```

Test with sample with identical means:

```
>>> rvs1 = stats.norm.rvs(loc=5, scale=10, size=500, random_state=rng)
>>> rvs2 = stats.norm.rvs(loc=5, scale=10, size=500, random_state=rng)
>>> stats.ttest_ind(rvs1, rvs2)
TtestResult(statistic=-0.4390847099199348,
            pvalue=0.6606952038870015,
            df=998.0)
>>> stats.ttest_ind(rvs1, rvs2, equal_var=False)
TtestResult(statistic=-0.4390847099199348,
            pvalue=0.6606952553131064,
            df=997.4602304121448)
```

`ttest_ind` underestimates p for unequal variances: ...

scipy.stats.ttest_rel

```
scipy.stats.ttest_rel(a, b, axis=0, nan_policy='propagate', alternative='two-sided', *, keepdims=False)
```

Calculate the t-test on TWO RELATED samples of scores, a and b.

This is a test for the null hypothesis that two related or repeated samples have identical average (expected) values.

Parameters

`a, b` : *array_like* The arrays must have the same shape.

`axis` : *int or None, default: 0* If an int, the axis of the input along which to compute the statistic. The statistic of each axis-slice (e.g. row) of the input will appear in a corresponding element of the output. If `None`, the input will be raveled before computing the statistic.

`nan_policy` : *{'propagate', 'omit', 'raise'}* Defines how to handle input NaNs.

- `propagate`: if a NaN is present in the axis slice (e.g. row) along which the statistic is computed, the corresponding entry of the output will be NaN.
- `omit`: NaNs will be omitted when performing the calculation. If insufficient data remains in the axis slice along which the statistic is computed, the corresponding entry of the output will be NaN.
- `raise`: if a NaN is present, a `ValueError` will be raised.

`alternative` : *{'two-sided', 'less', 'greater'}, optional* Defines the alternative hypothesis. The following options are available (default is 'two-sided'):

- 'two-sided': the means of the distributions underlying the samples are unequal.
- 'less': the mean of the distribution underlying the first sample is less than the mean of the distribution underlying the second sample.
- 'greater': the mean of the distribution underlying the first sample is greater than the mean of the distribution underlying the second sample.

New in version 1.6.0.

`keepdims` : *bool, default: False* If this is set to True, the axes which are reduced are left in the result as dimensions with size one. With this option, the result will broadcast correctly against the input array.

Returns

`result` : *“TtestResult“* An object with the following attributes:

statistic : float or array The t-statistic.

pvalue : float or array The p-value associated with the given alternative.

df : float or array The number of degrees of freedom used in calculation of the t-statistic; this is one less than the size of the sample (`a.shape[axis]`).

New in version 1.10.0.

The object also has the following method:

confidence_interval(confidence_level=0.95) Computes a confidence interval around the difference in population means for the given confidence level. The confidence interval is returned in a `namedtuple` with fields `low` and `high`.

New in version 1.10.0.

Notes

Examples for use are scores of the same set of student in different exams, or repeated sampling from the same units. The test measures whether the average score differs significantly across samples (e.g. exams). If we observe a large p-value, for example greater than 0.05 or 0.1 then we cannot reject the null hypothesis of identical average scores. If the p-value is smaller than the threshold, e.g. 1%, 5% or 10%, then we reject the null hypothesis of equal averages. Small p-values are associated with large t-statistics.

(Notes truncated; see the full reference page.)

Examples

```
>>> from scipy import stats
>>> rng = np.random.default_rng()
```

```
>>> rvs1 = stats.norm.rvs(loc=5, scale=10, size=500, random_state=rng)
>>> rvs2 = (stats.norm.rvs(loc=5, scale=10, size=500, random_state=rng)
...         + stats.norm.rvs(scale=0.2, size=500, random_state=rng))
>>> stats.ttest_rel(rvs1, rvs2)
TtestResult(statistic=-0.4549717054410304, pvalue=0.6493274702088672, df=499)
>>> rvs3 = (stats.norm.rvs(loc=8, scale=10, size=500, random_state=rng)
...         + stats.norm.rvs(scale=0.2, size=500, random_state=rng))
>>> stats.ttest_rel(rvs1, rvs3)
TtestResult(statistic=-5.879467544540889, pvalue=7.540777129099917e-09, df=499)
```

scipy.stats.mannwhitneyu

```
scipy.stats.mannwhitneyu(x, y, use_continuity=True, alternative='two-sided', axis=0, method='auto', *,
↪  nan_policy='propagate', keepdims=False)
```

Perform the Mann-Whitney U rank test on two independent samples.

The Mann-Whitney U test is a nonparametric test of the null hypothesis that the distribution underlying sample `x` is the same as the distribution underlying sample `y`. It is often used as a test of difference in location between distributions.

Parameters

`x, y` : *array-like* N-d arrays of samples. The arrays must be broadcastable except along the dimension given by `axis`.

`use_continuity` : *bool, optional* Whether a continuity correction (1/2) should be applied. Default is True when `method` is `'asymptotic'`; has no effect otherwise.

`alternative` : *{'two-sided', 'less', 'greater'}, optional* Defines the alternative hypothesis. Default is 'two-sided'. Let *SX(u)* and *SY(u)* be the survival functions of the distributions underlying `x` and `y`, respectively. Then the following alternative hypotheses are available:

- 'two-sided': the distributions are not equal, i.e. *SX(u) ≠ SY(u)* for at least one *u*.
- 'less': the distribution underlying `x` is stochastically less than the distribution underlying `y`, i.e. *SX(u) < SY(u)* for all *u*.
- 'greater': the distribution underlying `x` is stochastically greater than the distribution underlying `y`, i.e. *SX(u) > SY(u)* for all *u*.

Under a more restrictive set of assumptions, the alternative hypotheses can be expressed in terms of the locations of the distributions; see section 5.1.

`axis` : *int or None, default: 0* If an int, the axis of the input along which to compute the statistic. The statistic of each axis-slice (e.g. row) of the input will appear in a corresponding element of the output. If `None`, the input will be raveled before computing the statistic.

`method` : *{'auto', 'asymptotic', 'exact'} or "PermutationMethod" instance, optional* Selects the method used to calculate the *p*-value. Default is 'auto'. The following options are available.

- `'asymptotic'`: compares the standardized test statistic against the normal distribution, correcting for ties.
- `'exact'`: computes the exact *p*-value by comparing the observed U statistic against the exact distribution of the U statistic under the null hypothesis. No correction is made for ties.
- `'auto'`: chooses `'exact'` when the size of one of the samples is less than or equal to 8 and there are no ties; chooses `'asymptotic'` otherwise.
- `PermutationMethod` instance. In this case, the p-value is computed using `permutation_test` with the provided configuration options and other appropriate settings.

`nan_policy` : *{'propagate', 'omit', 'raise'}* Defines how to handle input NaNs.

- `propagate`: if a NaN is present in the axis slice (e.g. row) along which the statistic is computed, the corresponding entry of the output will be NaN.
- `omit`: NaNs will be omitted when performing the calculation. If insufficient data remains in the axis slice along which the statistic is computed, the corresponding entry of the output will be NaN.
- `raise`: if a NaN is present, a `ValueError` will be raised.

`keepdims` : *bool, default: False* If this is set to True, the axes which are reduced are left in the result as dimensions with size one. With this option, the result will broadcast correctly against the input array.

Returns

`res` : *MannwhitneyuResult* An object containing attributes:

statistic : float The Mann-Whitney U statistic corresponding with sample `x`. See Notes for the test statistic corresponding with sample `y`.

pvalue : float The associated *p*-value for the chosen `alternative`.

zstatistic : array_like When `method = 'approx'`, this is the normalized z-statistic:

```
z = (U - mn - d) / se
```

where `U` is `statistic` as defined above, `mn` is the mean of the distribution under the null hypothesis, `d` is a continuity correction, and `se` is the standard error. When `method != 'asymptotic'`, this attribute is not available.

See also: `scipy.stats.wilcoxon`, `scipy.stats.ttest_ind`

Notes

If `U1` is the statistic corresponding with sample `x`, then the statistic corresponding with sample `y` is `U2 = x.shape[axis] * y.shape[axis] - U1`.

The p-value and standardized z-statistic produced by the Mann-Whitney U test is identical to that produced by a Wilcoxon *rank sum* test, but different raw statistics are associated with the tests. Specifically, the rank sum statistic corresponding with the first sample is `R1 = U1 + x.shape[axis] * (x.shape[axis] + 1) / 2`.

`mannwhitneyu` is for independent samples. For related / paired samples, consider `scipy.stats.wilcoxon`.

(Notes truncated; see the full reference page.)

Examples

We follow the example from: nine randomly sampled young adults were diagnosed with type II diabetes at the ages below.

```
>>> males = [19, 22, 16, 29, 24]
>>> females = [20, 11, 17, 12]
```

We use the Mann-Whitney U test to assess whether there is a statistically significant difference in the diagnosis age of males and females. The null hypothesis is that the distribution of male diagnosis ages is the same as the distribution of female diagnosis ages. We decide that a confidence level of 95% is required to reject the null hypothesis in favor of the alternative that the distributions are different. Since the number of samples is very small and there are no ties in the data, we can compare the observed test statistic against the *exact* distribution of the test statistic under the null hypothesis.

```
>>> from scipy.stats import mannwhitneyu
>>> U1, p = mannwhitneyu(males, females, method="exact")
>>> print(U1)
17.0
...
```

scipy.stats.chi2_contingency

```
scipy.stats.chi2_contingency(observed, correction=True, lambda_=None, *, method=None)
```

Chi-square test of independence of variables in a contingency table.

This function computes the chi-square statistic and p-value for the hypothesis test of independence of the observed frequencies in the contingency table `observed`. The expected frequencies are computed based on the marginal sums under the assumption of independence; see `scipy.stats.contingency.expected_freq`. The number of degrees of freedom is (expressed using numpy functions and attributes):

```
dof = observed.size - sum(observed.shape) + observed.ndim - 1
```

Parameters

`observed` : ***array_like*** The contingency table. The table contains the observed frequencies (i.e. number of occurrences) in each category. In the two-dimensional case, the table is often described as an ”R x C table”.

`correction` : ***bool, optional*** If True, *and* the degrees of freedom is 1, apply Yates' correction for continuity. The effect of the correction is to adjust each observed value by 0.5 towards the corresponding expected value.

`lambda_` : ***float or str, optional*** By default, the statistic computed in this test is Pearson's chi-squared statistic. `lambda_` allows a statistic from the Cressie-Read power divergence family to be used instead. See `scipy.stats.power_divergence` for details.

method : *ResamplingMethod, optional* Defines the method used to compute the p-value. Compatible only with `correction=False`, default `lambda_`, and two-way tables. If `method` is an instance of `scipy.stats.PermutationMethod` or `scipy.stats.MonteCarloMethod`, the p-value is computed using `scipy.stats.permutation_test`/`scipy.stats.monte_carlo_test` with the provided configuration options and other appropriate settings. Otherwise, the p-value is computed as documented in the notes. Note that if `method` is an instance of `scipy.stats.MonteCarloMethod`, the `rvs` attribute must be left unspecified; Monte Carlo samples are always drawn using the `rvs` method of `scipy.stats.random_table`.

New in version 1.15.0.

Returns

res : *Chi2ContingencyResult* An object containing attributes:

statistic : float The test statistic.

pvalue : float The p-value of the test.

dof : int The degrees of freedom. NaN if `method` is not `None`.

expected_freq : ndarray, same shape as `observed` The expected frequencies, based on the marginal sums of the table.

See also: `scipy.stats.contingency.expected_freq`, `scipy.stats.fisher_exact`, `scipy.stats.chisquare`, `scipy.stats.power_divergence`, `scipy.stats.barnard_exact`, `scipy.stats.boschloo_exact`, `hypothesis_chi2_contingency`

Notes

An often quoted guideline for the validity of this calculation is that the test should be used only if the observed and expected frequencies in each cell are at least 5.

This is a test for the independence of different categories of a population. The test is only meaningful when the dimension of `observed` is two or more. Applying the test to a one-dimensional table will always result in `expected` equal to `observed` and a chi-square statistic equal to 0.

This function does not handle masked arrays, because the calculation does not make sense with missing values.

(Notes truncated; see the full reference page.)

Examples

A two-way example (2 x 3):

```
>>> from scipy.stats import chi2_contingency
>>> obs = np.array([[10, 10, 20], [20, 20, 20]])
>>> res = chi2_contingency(obs)
>>> res.statistic
2.7777777777777777
>>> res.pvalue
0.24935220877729619
>>> res.dof
2
>>> res.expected_freq
array([[ 12.,  12.,  16.],
       [ 18.,  18.,  24.]])
```

Perform the test using the log-likelihood ratio (i.e. the ”G-test”) instead of Pearson's chi-squared statistic. ...

scipy.stats.shapiro

```
scipy.stats.shapiro(x, *, axis=None, nan_policy='propagate', keepdims=False)
```

Perform the Shapiro-Wilk test for normality.

The Shapiro-Wilk test tests the null hypothesis that the data was drawn from a normal distribution.

Parameters

x : *array_like* Array of sample data. Must contain at least three observations.

axis : *int or None, default: None* If an int, the axis of the input along which to compute the statistic. The statistic of each axis-slice (e.g. row) of the input will appear in a corresponding element of the output. If `None`, the input will be raveled before computing the statistic.

nan_policy : *{'propagate', 'omit', 'raise'}* Defines how to handle input NaNs.

- `propagate`: if a NaN is present in the axis slice (e.g. row) along which the statistic is computed, the corresponding entry of the output will be NaN.
- `omit`: NaNs will be omitted when performing the calculation. If insufficient data remains in the axis slice along which the statistic is computed, the corresponding entry of the output will be NaN.
- `raise`: if a NaN is present, a `ValueError` will be raised.

keepdims : *bool, default: False* If this is set to True, the axes which are reduced are left in the result as dimensions with size one. With this option, the result will broadcast correctly against the input array.

Returns

statistic : *float* The test statistic.

p-value : *float* The p-value for the hypothesis test.

See also: `anderson`, `kstest`, `hypothesis_shapiro`

Notes

The algorithm used is described in, but censoring parameters as described are not implemented. For N > 5000 the W test statistic is accurate, but the p-value may not be.

Beginning in SciPy 1.9, `np.matrix` inputs (not recommended for new code) are converted to `np.ndarray` before the calculation is performed. In this case, the output will be a scalar or `np.ndarray` of appropriate shape rather than a 2D `np.matrix`. Similarly, while masked elements of masked arrays are ignored, the output will be a scalar or `np.ndarray` rather than a masked array with `mask=False`.

Array API Standard Support

(Notes truncated; see the full reference page.)

Examples

```
>>> from scipy import stats
>>> rng = np.random.default_rng()
>>> x = stats.norm.rvs(loc=5, scale=3, size=100, random_state=rng)
>>> shapiro_test = stats.shapiro(x)
>>> shapiro_test
ShapiroResult(statistic=0.9813305735588074, pvalue=0.16855233907699585)
>>> shapiro_test.statistic
0.9813305735588074
>>> shapiro_test.pvalue
0.16855233907699585
```

For a more detailed example, see hypothesis_shapiro.

scipy.stats.ks_2samp

```
scipy.stats.ks_2samp(data1, data2, alternative='two-sided', method='auto', *, axis=0,
↪  nan_policy='propagate', keepdims=False)
```

Performs the two-sample Kolmogorov-Smirnov test for goodness of fit.

This test compares the underlying continuous distributions F(x) and G(x) of two independent samples. See Notes for a description of the available null and alternative hypotheses.

Parameters

`data1, data2` : *array_like, 1-Dimensional* Two arrays of sample observations assumed to be drawn from a continuous distribution, sample sizes can be different.

`alternative` : *{'two-sided', 'less', 'greater'}, optional* Defines the null and alternative hypotheses. Default is 'two-sided'. Please see explanations in the Notes below.

`method` : *{'auto', 'exact', 'asymp'}, optional* Defines the method used for calculating the p-value. The following options are available (default is 'auto'):

- 'auto' : use 'exact' for small size arrays, 'asymp' for large
- 'exact' : use exact distribution of test statistic
- 'asymp' : use asymptotic distribution of test statistic

`axis` : *int or None, default: 0* If an int, the axis of the input along which to compute the statistic. The statistic of each axis-slice (e.g. row) of the input will appear in a corresponding element of the output. If `None`, the input will be raveled before computing the statistic.

`nan_policy` : *{'propagate', 'omit', 'raise'}* Defines how to handle input NaNs.

- `propagate`: if a NaN is present in the axis slice (e.g. row) along which the statistic is computed, the corresponding entry of the output will be NaN.
- `omit`: NaNs will be omitted when performing the calculation. If insufficient data remains in the axis slice along which the statistic is computed, the corresponding entry of the output will be NaN.
- `raise`: if a NaN is present, a `ValueError` will be raised.

`keepdims` : *bool, default: False* If this is set to True, the axes which are reduced are left in the result as dimensions with size one. With this option, the result will broadcast correctly against the input array.

Returns

res: KstestResult An object containing attributes:

statistic : float KS test statistic.

pvalue : float One-tailed or two-tailed p-value.

statistic_location : float Value from `data1` or `data2` corresponding with the KS statistic; i.e., the distance between the empirical distribution functions is measured at this observation.

statistic_sign : int +1 if the empirical distribution function of data1 exceeds the empirical distribution function of `data2` at `statistic_location`, otherwise -1.

See also: `kstest`, `ks_1samp`, `epps_singleton_2samp`, `anderson_ksamp`

Notes

There are three options for the null and corresponding alternative hypothesis that can be selected using the `alternative` parameter.

- `less`: The null hypothesis is that F(x) >= G(x) for all x; the alternative is that F(x) < G(x) for at least one x. The statistic is the magnitude of the minimum (most negative) difference between the empirical distribution functions of the samples.
- `greater`: The null hypothesis is that F(x) <= G(x) for all x; the alternative is that F(x) > G(x) for at least one x. The statistic is the maximum (most positive) difference between the empirical distribution functions of the samples.

(Notes truncated; see the full reference page.)

Examples

Suppose we wish to test the null hypothesis that two samples were drawn from the same distribution. We choose a confidence level of 95%; that is, we will reject the null hypothesis in favor of the alternative if the p-value is less than 0.05.

If the first sample were drawn from a uniform distribution and the second were drawn from the standard normal, we would expect the null hypothesis to be rejected.

```
>>> from scipy import stats
>>> rng = np.random.default_rng()
>>> sample1 = stats.uniform.rvs(size=100, random_state=rng)
>>> sample2 = stats.norm.rvs(size=110, random_state=rng)
>>> stats.ks_2samp(sample1, sample2)
KstestResult(statistic=0.5454545454545454,
             pvalue=7.37417839555191e-15,
             statistic_location=-0.014071496412861274,
             statistic_sign=-1)
...
```

scipy.stats.pearsonr

```
scipy.stats.pearsonr(x, y, *, alternative='two-sided', method=None, axis=0)
```

Pearson correlation coefficient and p-value for testing non-correlation.

The Pearson correlation coefficient measures the linear relationship between two datasets. Like other correlation coefficients, this one varies between -1 and +1 with 0 implying no correlation. Correlations of -1 or +1 imply an exact linear relationship. Positive correlations imply that as x increases, so does y. Negative correlations imply that as x increases, y decreases.

This function also performs a test of the null hypothesis that the distributions underlying the samples are uncorrelated and normally distributed. (See Kowalski for a discussion of the effects of non-normality of the input on the distribution of the correlation coefficient.) The p-value roughly indicates the probability of an uncorrelated system producing datasets that have a Pearson correlation at least as extreme as the one computed from these datasets.

Parameters

x : *array_like* Input array.

y : *array_like* Input array.

alternative : *{'two-sided', 'greater', 'less'}, optional* Defines the alternative hypothesis. Default is 'two-sided'. The following options are available:

- 'two-sided': the correlation is nonzero
- 'less': the correlation is negative (less than zero)
- 'greater': the correlation is positive (greater than zero)

New in version 1.9.0.

method : *ResamplingMethod, optional* Defines the method used to compute the p-value. If `method` is an instance of `PermutationMethod`/`MonteCarloMethod`, the p-value is computed using `scipy.stats.permutation_test`/`scipy.stats.monte_carlo_test` with the provided configuration options and other appropriate settings. Otherwise, the p-value is computed as documented in the notes.

New in version 1.11.0.

axis : *int or None, default* Axis along which to perform the calculation. Default is 0. If None, ravel both arrays before performing the calculation.

New in version 1.14.0.

Returns

result : *"PearsonRResult"* An object with the following attributes:

statistic : float Pearson product-moment correlation coefficient.

pvalue : float The p-value associated with the chosen alternative.

The object has the following method:

confidence_interval(confidence_level, method) This computes the confidence interval of the correlation coefficient `statistic` for the given confidence level. The confidence interval is returned in a `namedtuple` with fields `low` and `high`. If `method` is not provided, the confidence interval is computed using the Fisher transformation . If `method` is an instance of `BootstrapMethod`, the confidence interval is computed using `scipy.stats.bootstrap` with the provided configuration options and other appropriate settings. In some cases, confidence limits may be NaN due to a degenerate resample, and this is typical for very small samples (~6 observations).

Raises

ValueError If `x` and `y` do not have length at least 2.

See also: `spearmanr`, `kendalltau`, `hypothesis_pearsonr`

Notes

The correlation coefficient is calculated as follows:

$$r = \frac{\sum(x - m_x)(y - m_y)}{\sqrt{\sum(x - m_x)^2 \sum(y - m_y)^2}}$$

where m_x is the mean of the vector x and m_y is the mean of the vector y.

(Notes truncated; see the full reference page.)

Examples

```
>>> from scipy import stats
>>> x, y = [1, 2, 3, 4, 5, 6, 7], [10, 9, 2.5, 6, 4, 3, 2]
>>> res = stats.pearsonr(x, y)
>>> res
PearsonRResult(statistic=-0.828503883588428, pvalue=0.021280260007523286)
```

To perform an exact permutation version of the test:

```
>>> rng = np.random.default_rng(7796654889291491997)
>>> method = stats.PermutationMethod(n_resamples=np.inf, random_state=rng)
>>> stats.pearsonr(x, y, method=method)
PearsonRResult(statistic=-0.828503883588428, pvalue=0.028174603174603175)
```

To perform the test under the null hypothesis that the data were drawn from *uniform* distributions:

```
>>> method = stats.MonteCarloMethod(rvs=(rng.uniform, rng.uniform))
>>> stats.pearsonr(x, y, method=method)
PearsonRResult(statistic=-0.828503883588428, pvalue=0.0188)
...
```

scipy.stats.spearmanr

```
scipy.stats.spearmanr(a, b=None, axis=0, nan_policy='propagate', alternative='two-sided')
```

Calculate a Spearman correlation coefficient with associated p-value.

The Spearman rank-order correlation coefficient is a nonparametric measure of the monotonicity of the relationship between two datasets. Like other correlation coefficients, this one varies between -1 and +1 with 0 implying no correlation. Correlations of -1 or +1 imply an exact monotonic relationship. Positive correlations imply that as x increases, so does y. Negative correlations imply that as x increases, y decreases.

The p-value roughly indicates the probability of an uncorrelated system producing datasets that have a Spearman correlation at least as extreme as the one computed from these datasets. Although calculation of the p-value does not make strong assumptions about the distributions underlying the samples, it is only accurate for very large samples (>500 observations). For smaller sample sizes, consider a permutation test (see Examples section below).

Parameters

a, b : *1D or 2D array_like, b is optional* One or two 1-D or 2-D arrays containing multiple variables and observations. When these are 1-D, each represents a vector of observations of a single variable. For the behavior in the 2-D case, see under `axis`, below. Both arrays need to have the same length in the `axis` dimension.

axis : *int or None, optional* If axis=0 (default), then each column represents a variable, with observations in the rows. If axis=1, the relationship is transposed: each row represents a variable, while the columns contain observations. If axis=None, then both arrays will be raveled.

nan_policy : *{'propagate', 'raise', 'omit'}, optional* Defines how to handle when input contains nan. The following options are available (default is 'propagate'):

- 'propagate': returns nan
- 'raise': throws an error
- 'omit': performs the calculations ignoring nan values

alternative : *{'two-sided', 'less', 'greater'}, optional* Defines the alternative hypothesis. Default is 'two-sided'. The following options are available:

- 'two-sided': the correlation is nonzero
- 'less': the correlation is negative (less than zero)
- 'greater': the correlation is positive (greater than zero)

New in version 1.7.0.

Returns

res : *SignificanceResult* An object containing attributes:

statistic : float or ndarray (2-D square) Spearman correlation matrix or correlation coefficient (if only 2 variables are given as parameters). Correlation matrix is square with length equal to total number of variables (columns or rows) in `a` and `b` combined.

pvalue : float The p-value for a hypothesis test whose null hypothesis is that two samples have no ordinal correlation. See `alternative` above for alternative hypotheses. `pvalue` has the same shape as `statistic`.

Raises

ValueError If `axis` is not 0, 1 or None, or if the number of dimensions of `a` is greater than 2, or if `b` is None and the number of dimensions of `a` is less than 2.

See also: `hypothesis_spearmanr`

Examples

```
>>> from scipy import stats
>>> res = stats.spearmanr([1, 2, 3, 4, 5], [5, 6, 7, 8, 7])
>>> res.statistic
0.8207826816681233
>>> res.pvalue
0.08858700531354381
```

```
>>> rng = np.random.default_rng()
>>> x2n = rng.standard_normal((100, 2))
>>> y2n = rng.standard_normal((100, 2))
>>> res = stats.spearmanr(x2n)
>>> res.statistic, res.pvalue
(-0.07960396039603959, 0.4311168705769747)
```

```
>>> res = stats.spearmanr(x2n[:, 0], x2n[:, 1])
>>> res.statistic, res.pvalue
(-0.07960396039603959, 0.4311168705769747)
...
```

scipy.stats.linregress

```
scipy.stats.linregress(x, y, alternative='two-sided', *, axis=0, nan_policy='propagate', keepdims=False)
```

Calculate a linear least-squares regression for two sets of measurements.

Parameters

x, y : *array_like* Two sets of measurements. Both arrays should have the same length N.

alternative : *{'two-sided', 'less', 'greater'}, optional* Defines the alternative hypothesis. Default is 'two-sided'. The following options are available:

- 'two-sided': the slope of the regression line is nonzero
- 'less': the slope of the regression line is less than zero
- 'greater': the slope of the regression line is greater than zero

New in version 1.7.0.

axis : *int or None, default: 0* If an int, the axis of the input along which to compute the statistic. The statistic of each axis-slice (e.g. row) of the input will appear in a corresponding element of the output. If `None`, the input will be raveled before computing the statistic.

nan_policy : *{'propagate', 'omit', 'raise'}* Defines how to handle input NaNs.

- `propagate`: if a NaN is present in the axis slice (e.g. row) along which the statistic is computed, the corresponding entry of the output will be NaN.
- `omit`: NaNs will be omitted when performing the calculation. If insufficient data remains in the axis slice along which the statistic is computed, the corresponding entry of the output will be NaN.
- `raise`: if a NaN is present, a `ValueError` will be raised.

`keepdims` : *bool, default: False* If this is set to True, the axes which are reduced are left in the result as dimensions with size one. With this option, the result will broadcast correctly against the input array.

Returns

`result` : *"LinregressResult" instance* The return value is an object with the following attributes:

slope : float Slope of the regression line.

intercept : float Intercept of the regression line.

rvalue : float The Pearson correlation coefficient. The square of `rvalue` is equal to the coefficient of determination.

pvalue : float The p-value for a hypothesis test whose null hypothesis is that the slope is zero, using Wald Test with t-distribution of the test statistic. See `alternative` above for alternative hypotheses.

stderr : float Standard error of the estimated slope (gradient), under the assumption of residual normality.

intercept_stderr : float Standard error of the estimated intercept, under the assumption of residual normality.

See also: `scipy.optimize.curve_fit`, `scipy.optimize.leastsq`

Notes

For compatibility with older versions of SciPy, the return value acts like a `namedtuple` of length 5, with fields `slope`, `intercept`, `rvalue`, `pvalue` and `stderr`, so one can continue to write:

```
slope, intercept, r, p, se = linregress(x, y)
```

With that style, however, the standard error of the intercept is not available. To have access to all the computed values, including the standard error of the intercept, use the return value as an object with attributes, e.g.:

```
result = linregress(x, y)
print(result.intercept, result.intercept_stderr)
```

(Notes truncated; see the full reference page.)

Examples

```
>>> import matplotlib.pyplot as plt
>>> from scipy import stats
>>> rng = np.random.default_rng()
```

Generate some data:

```
>>> x = rng.random(10)
>>> y = 1.6*x + rng.random(10)
```

Perform the linear regression:

```
>>> res = stats.linregress(x, y)
```

Coefficient of determination (R-squared):

```
>>> print(f"R-squared: {res.rvalue**2:.6f}")
R-squared: 0.717533
...
```

scipy.stats.bootstrap

```
scipy.stats.bootstrap(data, statistic, *, n_resamples=9999, batch=None, vectorized=None, paired=False,
↪  axis=0, confidence_level=0.95, alternative='two-sided', method='BCa', bootstrap_result=None,
↪  rng=None, random_state=None)
```

Compute a two-sided bootstrap confidence interval of a statistic.

When `method` is `'percentile'` and `alternative` is `'two-sided'`, a bootstrap confidence interval is computed according to the following procedure.

1. Resample the data: for each sample in `data` and for each of `n_resamples`, take a random sample of the original sample (with replacement) of the same size as the original sample.
2. Compute the bootstrap distribution of the statistic: for each set of resamples, compute the test statistic.
3. Determine the confidence interval: find the interval of the bootstrap distribution that is
   - symmetric about the median and
   - contains `confidence_level` of the resampled statistic values.

While the `'percentile'` method is the most intuitive, it is rarely used in practice. Two more common methods are available, `'basic'` ('reverse percentile') and `'BCa'` ('bias-corrected and accelerated'); they differ in how step 3 is performed.

If the samples in `data` are taken at random from their respective distributions n times, the confidence interval returned by `bootstrap` will contain the true value of the statistic for those distributions approximately `confidence_level` $\times n$ times.

Parameters

`data` : *sequence of array-like* Each element of `data` is a sample containing scalar observations from an underlying distribution. Elements of `data` must be broadcastable to the same shape (with the possible exception of the dimension specified by `axis`).

`statistic` : *callable* Statistic for which the confidence interval is to be calculated. `statistic` must be a callable that accepts `len(data)` samples as separate arguments and returns the resulting statistic. If `vectorized` is set `True`, `statistic` must also accept a keyword argument `axis` and be vectorized to compute the statistic along the provided `axis`.

`n_resamples` : *int, default: "9999"* The number of resamples performed to form the bootstrap distribution of the statistic.

`batch` : *int, optional* The number of resamples to process in each vectorized call to `statistic`. Memory usage is O(`batch` * `n`), where `n` is the sample size. Default is `None`, in which case `batch = n_resamples` (or `batch = max(n_resamples, n)` for `method='BCa'`).

`vectorized` : *bool, optional* If `vectorized` is set `False`, `statistic` will not be passed keyword argument `axis` and is expected to calculate the statistic only for 1D samples. If `True`, `statistic` will be passed keyword argument `axis` and is expected to calculate the statistic along `axis` when passed an ND sample array. If `None` (default), `vectorized` will be set `True` if `axis` is a parameter of `statistic`. Use of a vectorized statistic typically reduces computation time.

paired : *bool, default: “False“* Whether the statistic treats corresponding elements of the samples in `data` as paired. If True, `bootstrap` resamples an array of *indices* and uses the same indices for all arrays in `data`; otherwise, `bootstrap` independently resamples the elements of each array.

axis : *int, default: “0“* The axis of the samples in `data` along which the `statistic` is calculated.

confidence_level : *float, default: “0.95“* The confidence level of the confidence interval.

alternative : *{'two-sided', 'less', 'greater'}, default: “'two-sided'“* Choose `'two-sided'` (default) for a two-sided confidence interval, `'less'` for a one-sided confidence interval with the lower bound at `-np.inf`, and `'greater'` for a one-sided confidence interval with the upper bound at `np.inf`. The other bound of the one-sided confidence intervals is the same as that of a two-sided confidence interval with `confidence_level` twice as far from 1.0; e.g. the upper bound of a 95% `'less'` confidence interval is the same as the upper bound of a 90% `'two-sided'` confidence interval.

method : *{'percentile', 'basic', 'bca'}, default: “'BCa'“* Whether to return the 'percentile' bootstrap confidence interval (`'percentile'`), the 'basic' (AKA 'reverse') bootstrap confidence interval (`'basic'`), or the bias-corrected and accelerated bootstrap confidence interval (`'BCa'`).

bootstrap_result : *BootstrapResult, optional* Provide the result object returned by a previous call to `bootstrap` to include the previous bootstrap distribution in the new bootstrap distribution. This can be used, for example, to change `confidence_level`, change `method`, or see the effect of performing additional resampling without repeating computations.

rng : *{None, int, “numpy.random.Generator“}, optional* If `rng` is passed by keyword, types other than `numpy.random.Generator` are passed to `numpy.random.default_rng` to instantiate a `Generator`. If `rng` is already a `Generator` instance, then the provided instance is used. Specify `rng` for repeatable function behavior.

If this argument is passed by position or `random_state` is passed by keyword, legacy behavior for the argument `random_state` applies:

- If `random_state` is None (or `numpy.random`), the `numpy.random.RandomState` singleton is used.
- If `random_state` is an int, a new `RandomState` instance is used, seeded with `random_state`.
- If `random_state` is already a `Generator` or `RandomState` instance then that instance is used.

Changed in version 1.15.0. As part of the SPEC-007 transition from use of `numpy.random.RandomState` to `numpy.random.Generator`, this keyword was changed from `random_state` to `rng`. For an interim period, both keywords will continue to work, although only one may be specified at a time. After the interim period, function calls using the `random_state` keyword will emit warnings. The behavior of both `random_state` and `rng` are outlined above, but only the `rng` keyword should be used in new code.

Returns

res : *BootstrapResult* An object with attributes:

confidence_interval : ConfidenceInterval The bootstrap confidence interval as an instance of `collections.namedtuple` with attributes `low` and `high`.

bootstrap_distribution : ndarray The bootstrap distribution, that is, the value of `statistic` for each resample. The last dimension corresponds with the resamples (e.g. `res.bootstrap_distribution.shape[-1] == n_resamples`).

standard_error : float or ndarray The bootstrap standard error, that is, the sample standard deviation of the bootstrap distribution.

Notes

Elements of the confidence interval may be NaN for `method='BCa'` if the bootstrap distribution is degenerate (e.g. all elements are identical). In this case, consider using another `method` or inspecting `data` for indications that other analysis may be more appropriate (e.g. all observations are identical).

Array API Standard Support

`bootstrap` has experimental support for Python Array API Standard compatible backends in addition to NumPy. Please consider testing these features by setting an environment variable `SCIPY_ARRAY_API=1` and providing CuPy, PyTorch, JAX, or Dask arrays as array arguments. The following combinations of backend and device (or other capability) are supported.

(Notes truncated; see the full reference page.)

Examples

Suppose we have sampled data from an unknown distribution.

```
>>> rng = np.random.default_rng()
>>> from scipy.stats import norm
>>> dist = norm(loc=2, scale=4)  # our "unknown" distribution
>>> data = dist.rvs(size=100, random_state=rng)
```

We are interested in the standard deviation of the distribution.

```
>>> std_true = dist.std()      # the true value of the statistic
>>> print(std_true)
4.0
>>> std_sample = np.std(data)  # the sample statistic
>>> print(std_sample)
3.9460644295563863
```

The bootstrap is used to approximate the variability we would expect if we were to repeatedly sample from the unknown distribution and calculate the statistic of the sample each time. It does this by repeatedly resampling values *from the original sample* with replacement and calculating the statistic of each resample. This results in a ”bootstrap distribution” of the statistic. ...

scipy.stats.gaussian_kde

```
scipy.stats.gaussian_kde(dataset, bw_method=None, weights=None)
```

Representation of a kernel-density estimate using Gaussian kernels.

Kernel density estimation is a way to estimate the probability density function (PDF) of a random variable in a non-parametric way. `gaussian_kde` works for both uni-variate and multi-variate data. It includes automatic bandwidth determination. The estimation works best for a unimodal distribution; bimodal or multi-modal distributions tend to be oversmoothed.

Parameters

`dataset` : *array_like* Datapoints to estimate from. In case of univariate data this is a 1-D array, otherwise a 2-D array with shape (# of dims, # of data).

`bw_method` : *str, scalar or callable, optional* The method used to calculate the bandwidth factor. This can be 'scott', 'silverman', a scalar constant or a callable. If a scalar, this will be used directly as `factor`. If a callable, it should take a `gaussian_kde` instance as only parameter and return a scalar. If None (default), 'scott' is used. See Notes for more details.

`weights` : *array_like, optional* weights of datapoints. This must be the same shape as dataset. If None (default), the samples are assumed to be equally weighted

Notes

Bandwidth selection strongly influences the estimate obtained from the KDE (much more so than the actual shape of the kernel). Bandwidth selection can be done by a "rule of thumb", by cross-validation, by "plug-in methods" or by other means; see, for reviews. `gaussian_kde` uses a rule of thumb, the default is Scott's Rule.

Scott's Rule, implemented as `scotts_factor`, is:

```
n**(-1./(d+4)),
```

with `n` the number of data points and `d` the number of dimensions. In the case of unequally weighted points, `scotts_factor` becomes:

```
(Notes truncated; see the full reference page.)
```

Examples

Generate some random two-dimensional data:

```
>>> from scipy import stats
>>> def measure(n):
...     "Measurement model, return two coupled measurements."
...     m1 = np.random.normal(size=n)
...     m2 = np.random.normal(scale=0.5, size=n)
...     return m1+m2, m1-m2
```

```
>>> m1, m2 = measure(2000)
>>> xmin = m1.min()
>>> xmax = m1.max()
>>> ymin = m2.min()
>>> ymax = m2.max()
```

Perform a kernel density estimate on the data: ...

Optimization

scipy.optimize.minimize

```
scipy.optimize.minimize(fun, x0, args=(), method=None, jac=None, hess=None, hessp=None, bounds=None,
↪  constraints=(), tol=None, callback=None, options=None)
```

Minimization of scalar function of one or more variables.

Parameters

`fun` : *callable* The objective function to be minimized:

```
fun(x, *args) -> float
```

where `x` is a 1-D array with shape (n,) and `args` is a tuple of the fixed parameters needed to completely specify the function.

Suppose the callable has signature `f0(x, *my_args, **my_kwargs)`, where `my_args` and `my_kwargs` are required positional and keyword arguments. Rather than passing `f0` as the callable, wrap it to accept only `x`; e.g., pass `fun=lambda x: f0(x, *my_args, **my_kwargs)` as the callable, where `my_args` (tuple) and `my_kwargs` (dict) have been gathered before invoking this function.

`x0` : *ndarray, shape (n,)* Initial guess. Array of real elements of size (n,), where `n` is the number of independent variables.

args : *tuple, optional* Extra arguments passed to the objective function and its derivatives (`fun`, `jac` and `hess` functions).

method : *str or callable, optional* Type of solver. Should be one of

- 'Nelder-Mead' (see here)
- 'Powell' (see here)
- 'CG' (see here)
- 'BFGS' (see here)
- 'Newton-CG' (see here)
- 'L-BFGS-B' (see here)
- 'TNC' (see here)
- 'COBYLA' (see here)
- 'COBYQA' (see here)
- 'SLSQP' (see here)
- 'trust-constr'(see here)
- 'dogleg' (see here)
- 'trust-ncg' (see here)
- 'trust-exact' (see here)
- 'trust-krylov' (see here)
- custom - a callable object, see below for description.

If not given, chosen to be one of `BFGS`, `L-BFGS-B`, `SLSQP`, depending on whether or not the problem has constraints or bounds.

jac : *{callable, '2-point', '3-point', 'cs', bool}, optional* Method for computing the gradient vector. Only for CG, BFGS, Newton-CG, L-BFGS-B, TNC, SLSQP, dogleg, trust-ncg, trust-krylov, trust-exact and trust-constr. If it is a callable, it should be a function that returns the gradient vector:

```
jac(x, *args) -> array_like, shape (n,)
```

where `x` is an array with shape (n,) and `args` is a tuple with the fixed parameters. If `jac` is a Boolean and is True, `fun` is assumed to return a tuple `(f, g)` containing the objective function and the gradient. Methods 'Newton-CG', 'trust-ncg', 'dogleg', 'trust-exact', and 'trust-krylov' require that either a callable be supplied, or that `fun` return the objective and gradient. If None or False, the gradient will be estimated using 2-point finite difference estimation with an absolute step size. Alternatively, the keywords {'2-point', '3-point', 'cs'} can be used to select a finite difference scheme for numerical estimation of the gradient with a relative step size. These finite difference schemes obey any specified `bounds`.

hess : *{callable, '2-point', '3-point', 'cs', HessianUpdateStrategy}, optional* Method for computing the Hessian matrix. Only for Newton-CG, dogleg, trust-ncg, trust-krylov, trust-exact and trust-constr. If it is callable, it should return the Hessian matrix:

```
hess(x, *args) -> {LinearOperator, spmatrix, array}, (n, n)
```

where `x` is a (n,) ndarray and `args` is a tuple with the fixed parameters. The keywords {'2-point', '3-point', 'cs'} can also be used to select a finite difference scheme for numerical estimation of the hessian. Alternatively, objects implementing the `HessianUpdateStrategy` interface can be used to approximate the Hessian. Available quasi-Newton methods implementing this interface are:

- `BFGS`
- `SR1`

Not all of the options are available for each of the methods; for availability refer to the notes.

hessp : *callable, optional* Hessian of objective function times an arbitrary vector p. Only for Newton-CG, trust-ncg, trust-krylov, trust-constr. Only one of `hessp` or `hess` needs to be given.

If `hess` is provided, then `hessp` will be ignored. `hessp` must compute the Hessian times an arbitrary vector:

```
hessp(x, p, *args) ->  ndarray shape (n,)
```

where `x` is a (n,) ndarray, `p` is an arbitrary vector with dimension (n,) and `args` is a tuple with the fixed parameters.

`bounds` : *sequence or "Bounds", optional* Bounds on variables for Nelder-Mead, L-BFGS-B, TNC, SLSQP, Powell, trust-constr, COBYLA, and COBYQA methods. There are two ways to specify the bounds:

1. Instance of `Bounds` class.
2. Sequence of `(min, max)` pairs for each element in `x`. None is used to specify no bound.

`constraints` : *{Constraint, dict} or List of {Constraint, dict}, optional* Constraints definition. Only for COBYLA, COBYQA, SLSQP and trust-constr.

Constraints for 'trust-constr', 'cobyqa', and 'cobyla' are defined as a single object or a list of objects specifying constraints to the optimization problem. Available constraints are:

- `LinearConstraint`
- `NonlinearConstraint`

Constraints for COBYLA, SLSQP are defined as a list of dictionaries. Each dictionary with fields:

type : str Constraint type: 'eq' for equality, 'ineq' for inequality.

fun : callable The function defining the constraint.

jac : callable, optional The Jacobian of `fun` (only for SLSQP).

args : sequence, optional Extra arguments to be passed to the function and Jacobian.

Equality constraint means that the constraint function result is to be zero whereas inequality means that it is to be non-negative.

`tol` : *float, optional* Tolerance for termination. When `tol` is specified, the selected minimization algorithm sets some relevant solver-specific tolerance(s) equal to `tol`. For detailed control, use solver-specific options.

`callback` : *callable, optional* A callable called after each iteration.

All methods except TNC support a callable with the signature:

```
callback(intermediate_result: OptimizeResult)
```

where `intermediate_result` is a keyword parameter containing an `OptimizeResult` with attributes `x` and `fun`, the present values of the parameter vector and objective function. Not all attributes of `OptimizeResult` may be present. The name of the parameter must be `intermediate_result` for the callback to be passed an `OptimizeResult`. These methods will also terminate if the callback raises `StopIteration`.

All methods except trust-constr (also) support a signature like:

```
callback(xk)
```

where `xk` is the current parameter vector.

Introspection is used to determine which of the signatures above to invoke.

`options` : *dict, optional* A dictionary of solver options. All methods except `TNC` accept the following generic options:

maxiter : int Maximum number of iterations to perform. Depending on the method each iteration may use several function evaluations.

For `TNC` use `maxfun` instead of `maxiter`.

disp : bool Set to True to print convergence messages.

For method-specific options, see `show_options()`.

Returns

res : *OptimizeResult* The optimization result represented as a `OptimizeResult` object. Important attributes are: `x` the solution array, `success` a Boolean flag indicating if the optimizer exited successfully and `message` which describes the cause of the termination. See `OptimizeResult` for a description of other attributes.

See also: `minimize_scalar`, `show_options`

Notes

This section describes the available solvers that can be selected by the 'method' parameter. The default method is *BFGS*.

Unconstrained minimization

Method CG uses a nonlinear conjugate gradient algorithm by Polak and Ribiere, a variant of the Fletcher-Reeves method described in pp.120-122. Only the first derivatives are used.

(Notes truncated; see the full reference page.)

Examples

Let us consider the problem of minimizing the Rosenbrock function. This function (and its respective derivatives) is implemented in `rosen` (resp. `rosen_der`, `rosen_hess`) in the `scipy.optimize`.

```
>>> from scipy.optimize import minimize, rosen, rosen_der
```

A simple application of the *Nelder-Mead* method is:

```
>>> x0 = [1.3, 0.7, 0.8, 1.9, 1.2]
>>> res = minimize(rosen, x0, method='Nelder-Mead', tol=1e-6)
>>> res.x
array([ 1.,  1.,  1.,  1.,  1.])
```

Now using the *BFGS* algorithm, using the first derivative and a few options:

```
>>> res = minimize(rosen, x0, method='BFGS', jac=rosen_der,
...                options={'gtol': 1e-6, 'disp': True})
Optimization terminated successfully.
         Current function value: 0.000000
         Iterations: 26
         Function evaluations: 31
         Gradient evaluations: 31
>>> res.x
...
```

scipy.optimize.curve_fit

```
scipy.optimize.curve_fit(f, xdata, ydata, p0=None, sigma=None, absolute_sigma=False, check_finite=None,
↪  bounds=(-inf, inf), method=None, jac=None, *, full_output=False, nan_policy=None, **kwargs)
```

Use non-linear least squares to fit a function, f, to data.

Assumes `ydata = f(xdata, *params) + eps`.

Parameters

f : *callable* The model function, f(x, ...). It must take the independent variable as the first argument and the parameters to fit as separate remaining arguments.

xdata : *array_like* The independent variable where the data is measured. Should usually be an M-length sequence or an (k,M)-shaped array for functions with k predictors, and each element should be float convertible if it is an array like object.

ydata : *array_like* The dependent data, a length M array - nominally `f(xdata, ...)`.

p0 : *array_like, optional* Initial guess for the parameters (length N). If None, then the initial values will all be 1 (if the number of parameters for the function can be determined using introspection, otherwise a ValueError is raised).

sigma : *None or scalar or M-length sequence or MxM array, optional* Determines the uncertainty in `ydata`. If we define residuals as `r = ydata - f(xdata, *popt)`, then the interpretation of `sigma` depends on its number of dimensions:

- A scalar or 1-D `sigma` should contain values of standard deviations of errors in `ydata`. In this case, the optimized function is `chisq = sum((r / sigma) ** 2)`.
- A 2-D `sigma` should contain the covariance matrix of errors in `ydata`. In this case, the optimized function is `chisq = r.T @ inv(sigma) @ r`.

  New in version 0.19.

None (default) is equivalent of 1-D `sigma` filled with ones.

absolute_sigma : *bool, optional* If True, `sigma` is used in an absolute sense and the estimated parameter covariance `pcov` reflects these absolute values.

If False (default), only the relative magnitudes of the `sigma` values matter. The returned parameter covariance matrix `pcov` is based on scaling `sigma` by a constant factor. This constant is set by demanding that the reduced `chisq` for the optimal parameters `popt` when using the *scaled* `sigma` equals unity. In other words, `sigma` is scaled to match the sample variance of the residuals after the fit. Default is False. Mathematically, `pcov(absolute_sigma=False) = pcov(absolute_sigma=True) * chisq(popt)/(M-N)`

check_finite : *bool, optional* If True, check that the input arrays do not contain nans of infs, and raise a ValueError if they do. Setting this parameter to False may silently produce nonsensical results if the input arrays do contain nans. Default is True if `nan_policy` is not specified explicitly and False otherwise.

bounds : *2-tuple of array_like or "Bounds", optional* Lower and upper bounds on parameters. Defaults to no bounds. There are two ways to specify the bounds:

- Instance of `Bounds` class.
- 2-tuple of array_like: Each element of the tuple must be either an array with the length equal to the number of parameters, or a scalar (in which case the bound is taken to be the same for all parameters). Use `np.inf` with an appropriate sign to disable bounds on all or some parameters.

method : *{'lm', 'trf', 'dogbox'}, optional* Method to use for optimization. See `least_squares` for more details. Default is 'lm' for unconstrained problems and 'trf' if `bounds` are provided. The method 'lm' won't work when the number of observations is less than the number of variables, use 'trf' or 'dogbox' in this case.

New in version 0.17.

jac : *callable, str or None, optional* Function with signature `jac(x, ...)` which computes the Jacobian matrix of the model function with respect to parameters as a dense array_like structure. It will be scaled according to provided `sigma`. If None (default), the Jacobian will be estimated numerically. String keywords for 'trf' and 'dogbox' methods can be used to select a finite difference scheme, see `least_squares`.

New in version 0.18.

full_output : *bool, optional* If True, this function returns additional information: `infodict`, `mesg`, and `ier`.

New in version 1.9.

nan_policy : *{'raise', 'omit', None}, optional* Defines how to handle when input contains nan. The following options are available (default is None):

- 'raise': throws an error
- 'omit': performs the calculations ignoring nan values
- None: no special handling of NaNs is performed (except what is done by check_finite); the behavior when NaNs are present is implementation-dependent and may change.

Note that if this value is specified explicitly (not None), `check_finite` will be set as False.

New in version 1.11.

****kwargs** Keyword arguments passed to `leastsq` for `method='lm'` or `least_squares` otherwise.

Returns

popt : *array* Optimal values for the parameters so that the sum of the squared residuals of `f(xdata, *popt) - ydata` is minimized.

pcov : *2-D array* The estimated approximate covariance of popt. The diagonals provide the variance of the parameter estimate. To compute one standard deviation errors on the parameters, use `perr = np.sqrt(np.diag(pcov))`. Note that the relationship between `cov` and parameter error estimates is derived based on a linear approximation to the model function around the optimum. When this approximation becomes inaccurate, `cov` may not provide an accurate measure of uncertainty.

How the `sigma` parameter affects the estimated covariance depends on `absolute_sigma` argument, as described above.

If the Jacobian matrix at the solution doesn't have a full rank, then 'lm' method returns a matrix filled with `np.inf`, on the other hand 'trf' and 'dogbox' methods use Moore-Penrose pseudoinverse to compute the covariance matrix. Covariance matrices with large condition numbers (e.g. computed with `numpy.linalg.cond`) may indicate that results are unreliable.

infodict : *dict (returned only if "full_output" is True)* a dictionary of optional outputs with the keys:

nfev The number of function calls. Methods 'trf' and 'dogbox' do not count function calls for numerical Jacobian approximation, as opposed to 'lm' method.

fvec The residual values evaluated at the solution, for a 1-D `sigma` this is `(f(x, *popt) - ydata)/sigma`.

fjac A permutation of the R matrix of a QR factorization of the final approximate Jacobian matrix, stored column wise. Together with ipvt, the covariance of the estimate can be approximated. Method 'lm' only provides this information.

ipvt An integer array of length N which defines a permutation matrix, p, such that fjac*p = q*r, where r is upper triangular with diagonal elements of nonincreasing magnitude. Column j of p is column ipvt(j) of the identity matrix. Method 'lm' only provides this information.

qtf The vector (transpose(q) * fvec). Method 'lm' only provides this information.

New in version 1.9.

mesg : *str (returned only if "full_output" is True)* A string message giving information about the solution.

New in version 1.9.

ier : *int (returned only if "full_output" is True)* An integer flag. If it is equal to 1, 2, 3 or 4, the solution was found. Otherwise, the solution was not found. In either case, the optional output variable `mesg` gives more information.

New in version 1.9.

Raises

ValueError if either `ydata` or `xdata` contain NaNs, or if incompatible options are used.

RuntimeError if the least-squares minimization fails.

TypeError if the number of data points is fewer than the number of parameters

OptimizeWarning if covariance of the parameters can not be estimated.

See also: `least_squares`, `scipy.stats.linregress`

Notes

Users should ensure that inputs `xdata`, `ydata`, and the output of `f` are `float64`, or else the optimization may return incorrect results.

With `method='lm'`, the algorithm uses the Levenberg-Marquardt algorithm through `leastsq`. Note that this algorithm can only deal with unconstrained problems.

Box constraints can be handled by methods 'trf' and 'dogbox'. Refer to the docstring of `least_squares` for more information.

(Notes truncated; see the full reference page.)

Examples

```
>>> import matplotlib.pyplot as plt
>>> from scipy.optimize import curve_fit
```

```
>>> def func(x, a, b, c):
...     return a * np.exp(-b * x) + c
```

Define the data to be fit with some noise:

```
>>> xdata = np.linspace(0, 4, 50)
>>> y = func(xdata, 2.5, 1.3, 0.5)
>>> rng = np.random.default_rng()
>>> y_noise = 0.2 * rng.normal(size=xdata.size)
>>> ydata = y + y_noise
>>> plt.plot(xdata, ydata, 'b-', label='data')
```

Fit for the parameters a, b, c of the function `func`: ...

scipy.optimize.least_squares

```
scipy.optimize.least_squares(fun, x0, jac='2-point', bounds=(-inf, inf), method='trf', ftol=1e-08,
↪  xtol=1e-08, gtol=1e-08, x_scale=None, loss='linear', f_scale=1.0, diff_step=None, tr_solver=None,
↪  tr_options=None, jac_sparsity=None, max_nfev=None, verbose=0, args=(), kwargs=None, callback=None,
↪  workers=None)
```

Solve a nonlinear least-squares problem with bounds on the variables.

Given the residuals f(x) (an m-D real function of n real variables) and the loss function rho(s) (a scalar function), `least_squares` finds a local minimum of the cost function F(x):

```
minimize F(x) = 0.5 * sum(rho(f_i(x)**2), i = 0, ..., m - 1)
subject to lb <= x <= ub
```

The purpose of the loss function rho(s) is to reduce the influence of outliers on the solution.

Parameters

fun : *callable* Function which computes the vector of residuals, with the signature `fun(x, *args, **kwargs)`, i.e., the minimization proceeds with respect to its first argument. The argument `x` passed to this function is an ndarray of shape (n,) (never a scalar, even for n=1). It must allocate and return a 1-D array_like of shape (m,) or a scalar. If the argument `x` is complex or the function `fun` returns complex residuals, it must be wrapped in a real function of real arguments, as shown at the end of the Examples section.

x0 : *array_like with shape (n,) or float* Initial guess on independent variables. If float, it will be treated as a 1-D array with one element. When `method` is 'trf', the initial guess might be slightly adjusted to lie sufficiently within the given `bounds`.

jac : *{'2-point', '3-point', 'cs', callable}, optional* Method of computing the Jacobian matrix (an m-by-n matrix, where element (i, j) is the partial derivative of f[i] with respect to x[j]). The keywords select a finite difference scheme for numerical estimation. The scheme '3-point' is more accurate, but requires twice as many operations as '2-point' (default). The scheme 'cs' uses complex steps, and while potentially the most accurate, it is applicable only when `fun` correctly handles complex inputs and can be analytically continued to the complex plane. If callable, it is used as `jac(x, *args, **kwargs)` and should return a good approximation (or the exact value) for the Jacobian as an array_like (np.atleast_2d is applied), a sparse array (csr_array preferred for performance) or a `scipy.sparse.linalg.LinearOperator`.

Changed in version 1.16.0. An ability to use the '3-point', 'cs' keywords with the 'lm' method. Previously 'lm' was limited to '2-point' and callable.

bounds : *2-tuple of array_like or "Bounds", optional* There are two ways to specify bounds:

1. Instance of `Bounds` class
2. Lower and upper bounds on independent variables. Defaults to no bounds. Each array must match the size of `x0` or be a scalar, in the latter case a bound will be the same for all variables. Use `np.inf` with an appropriate sign to disable bounds on all or some variables.

method : *{'trf', 'dogbox', 'lm'}, optional* Algorithm to perform minimization.

- 'trf' : Trust Region Reflective algorithm, particularly suitable for large sparse problems with bounds. Generally robust method.
- 'dogbox' : dogleg algorithm with rectangular trust regions, typical use case is small problems with bounds. Not recommended for problems with rank-deficient Jacobian.
- 'lm' : Levenberg-Marquardt algorithm as implemented in MINPACK. Doesn't handle bounds and sparse Jacobians. Usually the most efficient method for small unconstrained problems.

Default is 'trf'. See Notes for more information.

ftol : *float or None, optional* Tolerance for termination by the change of the cost function. Default is 1e-8. The optimization process is stopped when `dF < ftol * F`, and there was an adequate agreement between a local quadratic model and the true model in the last step.

If None and 'method' is not 'lm', the termination by this condition is disabled. If 'method' is 'lm', this tolerance must be higher than machine epsilon.

xtol : *float or None, optional* Tolerance for termination by the change of the independent variables. Default is 1e-8. The exact condition depends on the `method` used:

- For 'trf' and 'dogbox' : `norm(dx) < xtol * (xtol + norm(x))`.

- For 'lm' : `Delta < xtol * norm(xs)`, where `Delta` is a trust-region radius and `xs` is the value of `x` scaled according to `x_scale` parameter (see below).

If None and 'method' is not 'lm', the termination by this condition is disabled. If 'method' is 'lm', this tolerance must be higher than machine epsilon.

`gtol` : *float or None, optional* Tolerance for termination by the norm of the gradient. Default is 1e-8. The exact condition depends on a `method` used:

- For 'trf' : `norm(g_scaled, ord=np.inf) < gtol`, where `g_scaled` is the value of the gradient scaled to account for the presence of the bounds [STIR].
- For 'dogbox' : `norm(g_free, ord=np.inf) < gtol`, where `g_free` is the gradient with respect to the variables which are not in the optimal state on the boundary.
- For 'lm' : the maximum absolute value of the cosine of angles between columns of the Jacobian and the residual vector is less than `gtol`, or the residual vector is zero.

If None and 'method' is not 'lm', the termination by this condition is disabled. If 'method' is 'lm', this tolerance must be higher than machine epsilon.

`x_scale` : *{None, array_like, 'jac'}, optional* Characteristic scale of each variable. Setting `x_scale` is equivalent to reformulating the problem in scaled variables `xs = x / x_scale`. An alternative view is that the size of a trust region along jth dimension is proportional to `x_scale[j]`. Improved convergence may be achieved by setting `x_scale` such that a step of a given size along any of the scaled variables has a similar effect on the cost function. If set to 'jac', the scale is iteratively updated using the inverse norms of the columns of the Jacobian matrix (as described in [JJMore]). The default scaling for each method (i.e. if `x_scale is None`) is as follows:

- For 'trf' : `x_scale == 1`
- For 'dogbox' : `x_scale == 1`
- For 'lm' : `x_scale == 'jac'`

Changed in version 1.16.0. The default keyword value is changed from 1 to None to indicate that a default approach to scaling is used. For the 'lm' method the default scaling is changed from 1 to 'jac'. This has been found to give better performance, and is the same scaling as performed by `leastsq`.

`loss` : *str or callable, optional* Determines the loss function. The following keyword values are allowed:

- 'linear' (default) : `rho(z) = z`. Gives a standard least-squares problem.
- 'soft_l1' : `rho(z) = 2 * ((1 + z)**0.5 - 1)`. The smooth approximation of l1 (absolute value) loss. Usually a good choice for robust least squares.
- 'huber' : `rho(z) = z if z <= 1 else 2*z**0.5 - 1`. Works similarly to 'soft_l1'.
- 'cauchy' : `rho(z) = ln(1 + z)`. Severely weakens outliers influence, but may cause difficulties in optimization process.
- 'arctan' : `rho(z) = arctan(z)`. Limits a maximum loss on a single residual, has properties similar to 'cauchy'.

If callable, it must take a 1-D ndarray `z=f**2` and return an array_like with shape (3, m) where row 0 contains function values, row 1 contains first derivatives and row 2 contains second derivatives. Method 'lm' supports only 'linear' loss.

`f_scale` : *float, optional* Value of soft margin between inlier and outlier residuals, default is 1.0. The loss function is evaluated as follows `rho_(f**2) = C**2 * rho(f**2 / C**2)`, where `C` is `f_scale`, and `rho` is determined by `loss` parameter. This parameter has no effect with `loss='linear'`, but for other `loss` values it is of crucial importance.

`diff_step` : *None or array_like, optional* Determines the relative step size for the finite difference approximation of the Jacobian. The actual step is computed as `x * diff_step`. If None

(default), then `diff_step` is taken to be a conventional ”optimal” power of machine epsilon for the finite difference scheme used [NR].

`tr_solver` : *{None, 'exact', 'lsmr'}, optional* Method for solving trust-region subproblems, relevant only for 'trf' and 'dogbox' methods.

- 'exact' is suitable for not very large problems with dense Jacobian matrices. The computational complexity per iteration is comparable to a singular value decomposition of the Jacobian matrix.
- 'lsmr' is suitable for problems with sparse and large Jacobian matrices. It uses the iterative procedure `scipy.sparse.linalg.lsmr` for finding a solution of a linear least-squares problem and only requires matrix-vector product evaluations.

If None (default), the solver is chosen based on the type of Jacobian returned on the first iteration.

`tr_options` : *dict, optional* Keyword options passed to trust-region solver.

- `tr_solver='exact'`: `tr_options` are ignored.
- `tr_solver='lsmr'`: options for `scipy.sparse.linalg.lsmr`. Additionally, `method='trf'` supports 'regularize' option (bool, default is True), which adds a regularization term to the normal equation, which improves convergence if the Jacobian is rank-deficient [Byrd] (eq. 3.4).

`jac_sparsity` : *{None, array_like, sparse array}, optional* Defines the sparsity structure of the Jacobian matrix for finite difference estimation, its shape must be (m, n). If the Jacobian has only few non-zero elements in *each* row, providing the sparsity structure will greatly speed up the computations [Curtis]. A zero entry means that a corresponding element in the Jacobian is identically zero. If provided, forces the use of 'lsmr' trust-region solver. If None (default), then dense differencing will be used. Has no effect for 'lm' method.

`max_nfev` : *None or int, optional* For all methods this parameter controls the maximum number of function evaluations used by each method, separate to those used in numerical approximation of the jacobian. If None (default), the value is chosen automatically as 100 * n.

Changed in version 1.16.0. The default for the 'lm' method is changed to 100 * n, for both a callable and a numerically estimated jacobian. Previously the default when using an estimated jacobian was 100 * n * (n + 1), because the method included evaluations used in the estimation.

`verbose` : *{0, 1, 2}, optional* Level of algorithm's verbosity:

- 0 (default) : work silently.
- 1 : display a termination report.
- 2 : display progress during iterations (not supported by 'lm' method).

`args, kwargs` : *tuple and dict, optional* Additional arguments passed to `fun` and `jac`. Both empty by default. The calling signature is `fun(x, *args, **kwargs)` and the same for `jac`.

`callback` : *None or callable, optional* Callback function that is called by the algorithm on each iteration. This can be used to print or plot the optimization results at each step, and to stop the optimization algorithm based on some user-defined condition. Only implemented for the `trf` and `dogbox` methods.

The signature is `callback(intermediate_result: OptimizeResult)`

intermediate_result is a scipy.optimize.OptimizeResult‘ which contains the intermediate results of the optimization at the current iteration.

The callback also supports a signature like: `callback(x)`

Introspection is used to determine which of the signatures is invoked.

If the `callback` function raises `StopIteration` the optimization algorithm will stop and return with status code -2.

New in version 1.16.0.

`workers` : *map-like callable, optional* A map-like callable, such as `multiprocessing.Pool.map` for evaluating any numerical differentiation in parallel. This evaluation is carried out as `workers(fun, iterable)`.

New in version 1.16.0.

Returns

`result` : *OptimizeResult* `OptimizeResult` with the following fields defined:

x : ndarray, shape (n,) Solution found.

cost : float Value of the cost function at the solution.

fun : ndarray, shape (m,) Vector of residuals at the solution.

jac : ndarray, sparse array or LinearOperator, shape (m, n) Modified Jacobian matrix at the solution, in the sense that J^T J is a Gauss-Newton approximation of the Hessian of the cost function. The type is the same as the one used by the algorithm.

grad : ndarray, shape (m,) Gradient of the cost function at the solution.

optimality : float First-order optimality measure. In unconstrained problems, it is always the uniform norm of the gradient. In constrained problems, it is the quantity which was compared with `gtol` during iterations.

active_mask : ndarray of int, shape (n,) Each component shows whether a corresponding constraint is active (that is, whether a variable is at the bound):

- 0 : a constraint is not active.
- -1 : a lower bound is active.
- 1 : an upper bound is active.

Might be somewhat arbitrary for 'trf' method as it generates a sequence of strictly feasible iterates and `active_mask` is determined within a tolerance threshold.

nfev : int Number of function evaluations done. This number does not include the function calls used for numerical Jacobian approximation.

Changed in version 1.16.0. For the 'lm' method the number of function calls used in numerical Jacobian approximation is no longer included. This is to bring all methods into line.

njev : int or None Number of Jacobian evaluations done. If numerical Jacobian approximation is used in 'lm' method, it is set to None.

status : int The reason for algorithm termination:

- -2 : terminated because callback raised StopIteration.
- -1 : improper input parameters status returned from MINPACK.
- 0 : the maximum number of function evaluations is exceeded.
- 1 : `gtol` termination condition is satisfied.
- 2 : `ftol` termination condition is satisfied.
- 3 : `xtol` termination condition is satisfied.
- 4 : Both `ftol` and `xtol` termination conditions are satisfied.

message : str Verbal description of the termination reason.

success : bool True if one of the convergence criteria is satisfied (`status` > 0).

See also: `leastsq`, `curve_fit`

Notes

Method 'lm' (Levenberg-Marquardt) calls a wrapper over a least-squares algorithm implemented in MINPACK (lmder). It runs the Levenberg-Marquardt algorithm formulated as a trust-region type algorithm. The implementation is based on paper [JJMore], it is very robust and efficient with a lot of smart tricks. It should be your first choice for unconstrained problems. Note that it doesn't support bounds. Also, it doesn't work when m < n.

(Notes truncated; see the full reference page.)

Examples

In this example we find a minimum of the Rosenbrock function without bounds on independent variables.

```
>>> def fun_rosenbrock(x):
...     return np.array([10 * (x[1] - x[0]**2), (1 - x[0])])
```

Notice that we only provide the vector of the residuals. The algorithm constructs the cost function as a sum of squares of the residuals, which gives the Rosenbrock function. The exact minimum is at `x = [1.0, 1.0]`.

```
>>> from scipy.optimize import least_squares
>>> x0_rosenbrock = np.array([2, 2])
>>> res_1 = least_squares(fun_rosenbrock, x0_rosenbrock)
>>> res_1.x
array([ 1.,  1.])
>>> res_1.cost
9.8669242910846867e-30
>>> res_1.optimality
8.8928864934219529e-14
...
```

scipy.optimize.root_scalar

```
scipy.optimize.root_scalar(f, args=(), method=None, bracket=None, fprime=None, fprime2=None, x0=None,
↪  x1=None, xtol=None, rtol=None, maxiter=None, options=None)
```

Find a root of a scalar function.

Parameters

`f` : *callable* A function to find a root of.

Suppose the callable has signature `f0(x, *my_args, **my_kwargs)`, where `my_args` and `my_kwargs` are required positional and keyword arguments. Rather than passing `f0` as the callable, wrap it to accept only `x`; e.g., pass `fun=lambda x: f0(x, *my_args, **my_kwargs)` as the callable, where `my_args` (tuple) and `my_kwargs` (dict) have been gathered before invoking this function.

`args` : *tuple, optional* Extra arguments passed to the objective function and its derivative(s).

`method` : *str, optional* Type of solver. Should be one of

- 'bisect' (see here)
- 'brentq' (see here)
- 'brenth' (see here)
- 'ridder' (see here)
- 'toms748' (see here)
- 'newton' (see here)

- 'secant' (see here)
- 'halley' (see here)

bracket : *A sequence of 2 floats, optional* An interval bracketing a root. `f(x, *args)` must have different signs at the two endpoints.

fprime : *bool or callable, optional* If `fprime` is a boolean and is True, `f` is assumed to return the value of the objective function and of the derivative. `fprime` can also be a callable returning the derivative of `f`. In this case, it must accept the same arguments as `f`.

fprime2 : *bool or callable, optional* If `fprime2` is a boolean and is True, `f` is assumed to return the value of the objective function and of the first and second derivatives. `fprime2` can also be a callable returning the second derivative of `f`. In this case, it must accept the same arguments as `f`.

x0 : *float, optional* Initial guess.

x1 : *float, optional* A second guess.

xtol : *float, optional* Tolerance (absolute) for termination.

rtol : *float, optional* Tolerance (relative) for termination.

maxiter : *int, optional* Maximum number of iterations.

options : *dict, optional* A dictionary of solver options. E.g., `k`, see `show_options()` for details.

Returns

sol : *RootResults* The solution represented as a `RootResults` object. Important attributes are: `root` the solution , `converged` a boolean flag indicating if the algorithm exited successfully and `flag` which describes the cause of the termination. See `RootResults` for a description of other attributes.

See also: `show_options`, `root`

Notes

This section describes the available solvers that can be selected by the 'method' parameter.

The default is to use the best method available for the situation presented. If a bracket is provided, it may use one of the bracketing methods. If a derivative and an initial value are specified, it may select one of the derivative-based methods. If no method is judged applicable, it will raise an Exception.

Arguments for each method are as follows (x=required, o=optional).

(Notes truncated; see the full reference page.)

Examples

Find the root of a simple cubic

```
>>> from scipy import optimize
>>> def f(x):
...     return (x**3 - 1)  # only one real root at x = 1
```

```
>>> def fprime(x):
...     return 3*x**2
```

The `brentq` method takes as input a bracket

```
>>> sol = optimize.root_scalar(f, bracket=[0, 3], method='brentq')
>>> sol.root, sol.iterations, sol.function_calls
(1.0, 10, 11)
```

The `newton` method takes as input a single point and uses the derivative(s). ...

scipy.optimize.linprog

```
scipy.optimize.linprog(c, A_ub=None, b_ub=None, A_eq=None, b_eq=None, bounds=(0, None), method='highs',
↪  callback=None, options=None, x0=None, integrality=None)
```

Linear programming: minimize a linear objective function subject to linear equality and inequality constraints.

Linear programming solves problems of the following form:

$$\begin{aligned} \min_x \ & c^T x \\ \text{such that } & A_{ub} x \leq b_{ub}, \\ & A_{eq} x = b_{eq}, \\ & l \leq x \leq u, \end{aligned}$$

where x is a vector of decision variables; c, b_{ub}, b_{eq}, l, and u are vectors; and A_{ub} and A_{eq} are matrices.

Alternatively, that's:

- minimize :

  ```
  c @ x
  ```

- such that :

  ```
  A_ub @ x <= b_ub
  A_eq @ x == b_eq
  lb <= x <= ub
  ```

Note that by default `lb = 0` and `ub = None`. Other bounds can be specified with `bounds`.

Parameters

`c` : *1-D array* The coefficients of the linear objective function to be minimized.

`A_ub` : *2-D array, optional* The inequality constraint matrix. Each row of `A_ub` specifies the coefficients of a linear inequality constraint on `x`.

`b_ub` : *1-D array, optional* The inequality constraint vector. Each element represents an upper bound on the corresponding value of `A_ub @ x`.

`A_eq` : *2-D array, optional* The equality constraint matrix. Each row of `A_eq` specifies the coefficients of a linear equality constraint on `x`.

`b_eq` : *1-D array, optional* The equality constraint vector. Each element of `A_eq @ x` must equal the corresponding element of `b_eq`.

`bounds` : *sequence, optional* A sequence of `(min, max)` pairs for each element in `x`, defining the minimum and maximum values of that decision variable. If a single tuple `(min, max)` is provided, then `min` and `max` will serve as bounds for all decision variables. Use `None` to indicate that there is no bound. For instance, the default bound `(0, None)` means that all decision variables are non-negative, and the pair `(None, None)` means no bounds at all, i.e. all variables are allowed to be any real.

`method` : *str, optional* The algorithm used to solve the standard form problem. The following are supported.

- 'highs' (default)

- 'highs-ds'
- 'highs-ipm'
- 'interior-point' (legacy)
- 'revised simplex' (legacy)
- 'simplex' (legacy)

The legacy methods are deprecated and will be removed in SciPy 1.11.0.

`callback` : *callable, optional* If a callback function is provided, it will be called at least once per iteration of the algorithm. The callback function must accept a single `scipy.optimize.OptimizeResult` consisting of the following fields:

x : 1-D array The current solution vector.

fun : float The current value of the objective function `c @ x`.

success : bool `True` when the algorithm has completed successfully.

slack : 1-D array The (nominally positive) values of the slack, `b_ub - A_ub @ x`.

con : 1-D array The (nominally zero) residuals of the equality constraints, `b_eq - A_eq @ x`.

phase : int The phase of the algorithm being executed.

status : int An integer representing the status of the algorithm.

`0` : Optimization proceeding nominally.

`1` : Iteration limit reached.

`2` : Problem appears to be infeasible.

`3` : Problem appears to be unbounded.

`4` : Numerical difficulties encountered.

nit : int The current iteration number.

message : str A string descriptor of the algorithm status.

Callback functions are not currently supported by the HiGHS methods.

`options` : *dict, optional* A dictionary of solver options. All methods accept the following options:

maxiter : int Maximum number of iterations to perform. Default: see method-specific documentation.

disp : bool Set to `True` to print convergence messages. Default: `False`.

presolve : bool Set to `False` to disable automatic presolve. Default: `True`.

All methods except the HiGHS solvers also accept:

tol : float A tolerance which determines when a residual is ”close enough” to zero to be considered exactly zero.

autoscale : bool Set to `True` to automatically perform equilibration. Consider using this option if the numerical values in the constraints are separated by several orders of magnitude. Default: `False`.

rr : bool Set to `False` to disable automatic redundancy removal. Default: `True`.

rr_method : str Method used to identify and remove redundant rows from the equality constraint matrix after presolve. For problems with dense input, the available methods for redundancy removal are:

SVD: Repeatedly performs singular value decomposition on the matrix, detecting redundant rows based on nonzeros in the left singular vectors that correspond with zero singular values. May be fast when the matrix is nearly full rank.

pivot: Uses the algorithm presented in to identify redundant rows.

ID: Uses a randomized interpolative decomposition. Identifies columns of the matrix transpose not used in a full-rank interpolative decomposition of the matrix.

None: Uses `svd` if the matrix is nearly full rank, that is, the difference between the matrix rank and the number of rows is less than five. If not, uses `pivot`. The behavior of this default is subject to change without prior notice.

Default: None. For problems with sparse input, this option is ignored, and the pivot-based algorithm presented in is used.

For method-specific options, see `show_options('linprog')`.

x0 : ***1-D array, optional*** Guess values of the decision variables, which will be refined by the optimization algorithm. This argument is currently used only by the 'revised simplex' method, and can only be used if `x0` represents a basic feasible solution.

integrality : ***1-D array or int, optional*** Indicates the type of integrality constraint on each decision variable.

`0` : Continuous variable; no integrality constraint.

`1` : Integer variable; decision variable must be an integer within `bounds`.

`2` : Semi-continuous variable; decision variable must be within `bounds` or `0`.

`3` : Semi-integer variable; decision variable must be an integer within `bounds` or `0`.

By default, all variables are continuous.

For mixed integrality constraints, supply an array of shape `c.shape`. To infer a constraint on each decision variable from shorter inputs, the argument will be broadcast to `c.shape` using `numpy.broadcast_to`.

This argument is currently used only by the 'highs' method and is ignored otherwise.

Returns

res : ***OptimizeResult*** A `scipy.optimize.OptimizeResult` consisting of the fields below. Note that the return types of the fields may depend on whether the optimization was successful, therefore it is recommended to check `OptimizeResult.status` before relying on the other fields:

x : 1-D array The values of the decision variables that minimizes the objective function while satisfying the constraints.

fun : float The optimal value of the objective function `c @ x`.

slack : 1-D array The (nominally positive) values of the slack variables, `b_ub - A_ub @ x`.

con : 1-D array The (nominally zero) residuals of the equality constraints, `b_eq - A_eq @ x`.

success : bool `True` when the algorithm succeeds in finding an optimal solution.

status : int An integer representing the exit status of the algorithm.

`0` : Optimization terminated successfully.

`1` : Iteration limit reached.

`2` : Problem appears to be infeasible.

`3` : Problem appears to be unbounded.

`4` : Numerical difficulties encountered.

nit : int The total number of iterations performed in all phases.

message : str A string descriptor of the exit status of the algorithm.

See also: `show_options`

Notes

This section describes the available solvers that can be selected by the 'method' parameter.

'highs-ds', and 'highs-ipm' are interfaces to the HiGHS simplex and interior-point method solvers, respectively. 'highs' (default) chooses between the two automatically. These are the fastest linear programming solvers in SciPy, especially for large, sparse problems; which of these two is faster is problem-dependent. The other solvers are legacy methods and will be removed when `callback` is supported by the HiGHS methods.

(Notes truncated; see the full reference page.)

Examples

Consider the following problem:

The problem is not presented in the form accepted by `linprog`. This is easily remedied by converting the "greater than" inequality constraint to a "less than" inequality constraint by multiplying both sides by a factor of -1. Note also that the last constraint is really the simple bound $-3 \leq x_1 \leq \infty$. Finally, since there are no bounds on x_0, we must explicitly specify the bounds $-\infty \leq x_0 \leq \infty$, as the default is for variables to be non-negative. After collecting coeffecients into arrays and tuples, the input for this problem is: ...

Integration and interpolation

scipy.integrate.quad

```
scipy.integrate.quad(func, a, b, args=(), full_output=0, epsabs=1.49e-08, epsrel=1.49e-08, limit=50,
↪  points=None, weight=None, wvar=None, wopts=None, maxp1=50, limlst=50, complex_func=False)
```

Compute a definite integral.

Integrate func from `a` to `b` (possibly infinite interval) using a technique from the Fortran library QUADPACK.

Parameters

`func` : *{function, scipy.LowLevelCallable}* A Python function or method to integrate. If `func` takes many arguments, it is integrated along the axis corresponding to the first argument.

If the user desires improved integration performance, then `f` may be a `scipy.LowLevelCallable` with one of the signatures:

```
double func(double x)
double func(double x, void *user_data)
double func(int n, double *xx)
double func(int n, double *xx, void *user_data)
```

The `user_data` is the data contained in the `scipy.LowLevelCallable`. In the call forms with `xx`, `n` is the length of the `xx` array which contains `xx[0] == x` and the rest of the items are numbers contained in the `args` argument of quad.

In addition, certain ctypes call signatures are supported for backward compatibility, but those should not be used in new code.

a : *float* Lower limit of integration (use -numpy.inf for -infinity).

b : *float* Upper limit of integration (use numpy.inf for +infinity).

args : *tuple, optional* Extra arguments to pass to `func`.

full_output : *int, optional* Non-zero to return a dictionary of integration information. If non-zero, warning messages are also suppressed and the message is appended to the output tuple.

Returns

y : *float* The integral of func from `a` to `b`.

abserr : *float* An estimate of the absolute error in the result.

infodict : *dict* A dictionary containing additional information.

message A convergence message.

explain Appended only with 'cos' or 'sin' weighting and infinite integration limits, it contains an explanation of the codes in infodict['ierlst']

See also: `dblquad`, `tplquad`, `nquad`, `fixed_quad`, `simpson`, `romb`, `scipy.special`

Notes

For valid results, the integral must converge; behavior for divergent integrals is not guaranteed.

Extra information for quad() inputs and outputs

If full_output is non-zero, then the third output argument (infodict) is a dictionary with entries as tabulated below. For infinite limits, the range is transformed to (0,1) and the optional outputs are given with respect to this transformed range. Let M be the input argument limit and let K be infodict['last']. The entries are:

(Notes truncated; see the full reference page.)

Examples

Calculate $\int_0^4 x^2 dx$ and compare with an analytic result

```
>>> from scipy import integrate
>>> x2 = lambda x: x**2
>>> integrate.quad(x2, 0, 4)
(21.333333333333332, 2.3684757858670003e-13)
>>> print(4**3 / 3.)  # analytical result
21.3333333333
```

Calculate $\int_0^\infty e^{-x} dx$

```
>>> invexp = lambda x: np.exp(-x)
>>> integrate.quad(invexp, 0, np.inf)
(1.0, 5.842605999138044e-11)
```

Calculate $\int_0^1 ax\,dx$ for $a = 1, 3$ ...

scipy.integrate.solve_ivp

```
scipy.integrate.solve_ivp(fun, t_span, y0, method='RK45', t_eval=None, dense_output=False, events=None,
↪  vectorized=False, args=None, **options)
```

Solve an initial value problem for a system of ODEs.

This function numerically integrates a system of ordinary differential equations given an initial value:

```
dy / dt = f(t, y)
y(t0) = y0
```

Here t is a 1-D independent variable (time), y(t) is an N-D vector-valued function (state), and an N-D vector-valued function f(t, y) determines the differential equations. The goal is to find y(t) approximately satisfying the differential equations, given an initial value y(t0)=y0.

Some of the solvers support integration in the complex domain, but note that for stiff ODE solvers, the right-hand side must be complex-differentiable (satisfy Cauchy-Riemann equations). To solve a problem in the complex domain, pass y0 with a complex data type. Another option always available is to rewrite your problem for real and imaginary parts separately.

Parameters

`fun` : *callable* Right-hand side of the system: the time derivative of the state `y` at time `t`. The calling signature is `fun(t, y)`, where `t` is a scalar and `y` is an ndarray with `len(y) = len(y0)`. Additional arguments need to be passed if `args` is used (see documentation of `args` argument). `fun` must return an array of the same shape as `y`. See `vectorized` for more information.

`t_span` : *2-member sequence* Interval of integration (t0, tf). The solver starts with t=t0 and integrates until it reaches t=tf. Both t0 and tf must be floats or values interpretable by the float conversion function.

`y0` : *array_like, shape (n,)* Initial state. For problems in the complex domain, pass `y0` with a complex data type (even if the initial value is purely real).

`method` : *str or "OdeSolver", optional* Integration method to use:

- **'RK45' (default)**: Explicit Runge-Kutta method of order 5(4). The error is controlled assuming accuracy of the fourth-order method, but steps are taken using the fifth-order accurate formula (local extrapolation is done). A quartic interpolation polynomial is used for the dense output. Can be applied in the complex domain.
- **'RK23'**: Explicit Runge-Kutta method of order 3(2). The error is controlled assuming accuracy of the second-order method, but steps are taken using the third-order accurate formula (local extrapolation is done). A cubic Hermite polynomial is used for the dense output. Can be applied in the complex domain.
- **'DOP853'**: Explicit Runge-Kutta method of order 8. Python implementation of the "DOP853" algorithm originally written in Fortran. A 7-th order interpolation polynomial accurate to 7-th order is used for the dense output. Can be applied in the complex domain.
- **'Radau'**: Implicit Runge-Kutta method of the Radau IIA family of order 5. The error is controlled with a third-order accurate embedded formula. A cubic polynomial which satisfies the collocation conditions is used for the dense output.
- **'BDF'**: Implicit multi-step variable-order (1 to 5) method based on a backward differentiation formula for the derivative approximation. The implementation follows the one described in. A quasi-constant step scheme is used and accuracy is enhanced using the NDF modification. Can be applied in the complex domain.
- **'LSODA'**: Adams/BDF method with automatic stiffness detection and switching,. This is a wrapper of the Fortran solver from ODEPACK.

Explicit Runge-Kutta methods ('RK23', 'RK45', 'DOP853') should be used for non-stiff problems and implicit methods ('Radau', 'BDF') for stiff problems. Among Runge-Kutta methods, 'DOP853' is recommended for solving with high precision (low values of `rtol` and `atol`).

If not sure, first try to run 'RK45'. If it makes unusually many iterations, diverges, or fails, your problem is likely to be stiff and you should use 'Radau' or 'BDF'. 'LSODA' can also be a good universal choice, but it might be somewhat less convenient to work with as it wraps old Fortran code.

You can also pass an arbitrary class derived from `OdeSolver` which implements the solver.

`t_eval` : *array_like or None, optional* Times at which to store the computed solution, must be sorted and lie within `t_span`. If None (default), use points selected by the solver.

`dense_output` : *bool, optional* Whether to compute a continuous solution. Default is False.

`events` : *callable, or list of callables, optional* Events to track. If None (default), no events will be tracked. Each event occurs at the zeros of a continuous function of time and state. Each function must have the signature `event(t, y)` where additional argument have to be passed if `args` is used (see documentation of `args` argument). Each function must return a float. The solver will find an accurate value of `t` at which `event(t, y(t)) = 0` using a root-finding algorithm. By default, all zeros will be found. The solver looks for a sign change over each step, so if multiple zero crossings occur within one step, events may be missed. Additionally each `event` function might have the following attributes:

terminal: bool or int, optional When boolean, whether to terminate integration if this event occurs. When integral, termination occurs after the specified the number of occurrences of this event. Implicitly False if not assigned.

direction: float, optional Direction of a zero crossing. If `direction` is positive, `event` will only trigger when going from negative to positive, and vice versa if `direction` is negative. If 0, then either direction will trigger event. Implicitly 0 if not assigned.

You can assign attributes like `event.terminal = True` to any function in Python.

`vectorized` : *bool, optional* Whether `fun` can be called in a vectorized fashion. Default is False.

If `vectorized` is False, `fun` will always be called with `y` of shape `(n,)`, where `n = len(y0)`.

If `vectorized` is True, `fun` may be called with `y` of shape `(n, k)`, where `k` is an integer. In this case, `fun` must behave such that `fun(t, y)[:, i] == fun(t, y[:, i])` (i.e. each column of the returned array is the time derivative of the state corresponding with a column of `y`).

Setting `vectorized=True` allows for faster finite difference approximation of the Jacobian by methods 'Radau' and 'BDF', but will result in slower execution for other methods and for 'Radau' and 'BDF' in some circumstances (e.g. small `len(y0)`).

`args` : *tuple, optional* Additional arguments to pass to the user-defined functions. If given, the additional arguments are passed to all user-defined functions. So if, for example, `fun` has the signature `fun(t, y, a, b, c)`, then `jac` (if given) and any event functions must have the same signature, and `args` must be a tuple of length 3.

**`**options` Options passed to a chosen solver. All options available for already implemented solvers are listed below.

first_step : float or None, optional Initial step size. Default is `None` which means that the algorithm should choose.

max_step : float, optional Maximum allowed step size. Default is np.inf, i.e., the step size is not bounded and determined solely by the solver.

rtol, atol : float or array_like, optional Relative and absolute tolerances. The solver keeps the local error estimates less than `atol + rtol * abs(y)`. Here `rtol` controls a relative accuracy (number of correct digits), while `atol` controls absolute accuracy (number of correct decimal places). To achieve the desired `rtol`, set `atol` to be smaller than the smallest value that can be expected from `rtol * abs(y)` so that `rtol` dominates the allowable error. If `atol` is larger than `rtol * abs(y)` the number of correct digits is not guaranteed. Conversely, to achieve the desired `atol` set `rtol` such that `rtol * abs(y)` is always smaller than `atol`. If components of y have different scales, it might be beneficial to set different `atol` values for different components by passing array_like with shape (n,) for `atol`. Default values are 1e-3 for `rtol` and 1e-6 for `atol`.

jac : array_like, sparse_matrix, callable or None, optional Jacobian matrix of the right-hand side of the system with respect to y, required by the 'Radau', 'BDF' and 'LSODA' method. The Jacobian matrix has shape (n, n) and its element (i, j) is equal to `d f_i / d y_j`. There are three ways to define the Jacobian:

- If array_like or sparse_matrix, the Jacobian is assumed to be constant. Not supported by 'LSODA'.
- If callable, the Jacobian is assumed to depend on both t and y; it will be called as `jac(t, y)`, as necessary. Additional arguments have to be passed if `args` is used (see documentation of `args` argument). For 'Radau' and 'BDF' methods, the return value might be a sparse matrix.
- If None (default), the Jacobian will be approximated by finite differences.

It is generally recommended to provide the Jacobian rather than relying on a finite-difference approximation.

jac_sparsity : array_like, sparse matrix or None, optional Defines a sparsity structure of the Jacobian matrix for a finite-difference approximation. Its shape must be (n, n). This argument is ignored if `jac` is not `None`. If the Jacobian has only few non-zero elements in *each* row, providing the sparsity structure will greatly speed up the computations. A zero entry means that a corresponding element in the Jacobian is always zero. If None (default), the Jacobian is assumed to be dense. Not supported by 'LSODA', see `lband` and `uband` instead.

lband, uband : int or None, optional Parameters defining the bandwidth of the Jacobian for the 'LSODA' method, i.e., `jac[i, j] != 0 only for i - lband <= j <= i + uband`. Default is None. Setting these requires your jac routine to return the Jacobian in the packed format: the returned array must have `n` columns and `uband + lband + 1` rows in which Jacobian diagonals are written. Specifically `jac_packed[uband + i - j , j] = jac[i, j]`. The same format is used in `scipy.linalg.solve_banded` (check for an illustration). These parameters can be also used with `jac=None` to reduce the number of Jacobian elements estimated by finite differences.

min_step : float, optional The minimum allowed step size for 'LSODA' method. By default `min_step` is zero.

Returns

`result` : *OdeResult* Bunch object with the following fields defined:

t : ndarray, shape (n_points,) Time points.

y : ndarray, shape (n, n_points) Values of the solution at `t`.

sol : `OdeSolution` or None Found solution as `OdeSolution` instance; None if `dense_output` was set to False.

t_events : list of ndarray or None Contains for each event type a list of arrays at which an event of that type event was detected. None if `events` was None.

y_events : list of ndarray or None For each value of `t_events`, the corresponding value of the solution. None if `events` was None.

nfev : int Number of evaluations of the right-hand side.

njev : int Number of evaluations of the Jacobian.

nlu : int Number of LU decompositions.

status : int Reason for algorithm termination:

- -1: Integration step failed.
- 0: The solver successfully reached the end of `tspan`.
- 1: A termination event occurred.

message : str Human-readable description of the termination reason.

success : bool True if the solver reached the interval end or a termination event occurred (`status >= 0`).

Examples

Basic exponential decay showing automatically chosen time points.

```
>>> from scipy.integrate import solve_ivp
>>> def exponential_decay(t, y): return -0.5 * y
>>> sol = solve_ivp(exponential_decay, [0, 10], [2, 4, 8])
>>> print(sol.t)
[ 0.          0.11487653  1.26364188  3.06061781  4.81611105  6.57445806
  8.33328988 10.        ]
>>> print(sol.y)
[[2.         1.88836035 1.06327177 0.43319312 0.18017253 0.07483045
  0.03107158 0.01350781]
 [4.         3.7767207  2.12654355 0.86638624 0.36034507 0.14966091
  0.06214316 0.02701561]
 [8.         7.5534414  4.25308709 1.73277247 0.72069014 0.29932181
  0.12428631 0.05403123]]
```

Specifying points where the solution is desired. ...

scipy.interpolate.make_interp_spline

```
scipy.interpolate.make_interp_spline(x, y, k=3, t=None, bc_type=None, axis=0, check_finite=True)
```

Create an interpolating B-spline with specified degree and boundary conditions.

Parameters

x : *array_like, shape (n,)* Abscissas.

y : *array_like, shape (n, ...)* Ordinates.

k : *int, optional* B-spline degree. Default is cubic, `k = 3`.

t : *array_like, shape (nt + k + 1,), optional* Knots. The number of knots needs to agree with the number of data points and the number of derivatives at the edges. Specifically, `nt - n` must equal `len(deriv_l) + len(deriv_r)`.

bc_type : *2-tuple or None* Boundary conditions. Default is None, which means choosing the boundary conditions automatically. Otherwise, it must be a length-two tuple where the first element (`deriv_l`) sets the boundary conditions at `x[0]` and the second element (`deriv_r`) sets the boundary conditions at `x[-1]`. Each of these must be an iterable of pairs `(order, value)` which gives the values of derivatives of specified orders at the given edge of the interpolation interval. Alternatively, the following string aliases are recognized:

- `"clamped"`: The first derivatives at the ends are zero. This is equivalent to `bc_type=([(1, 0.0)], [(1, 0.0)])`.
- `"natural"`: The second derivatives at ends are zero. This is equivalent to `bc_type=([(2, 0.0)], [(2, 0.0)])`.
- `"not-a-knot"` (default): The first and second segments are the same polynomial. This is equivalent to having `bc_type=None`.
- `"periodic"`: The values and the first `k-1` derivatives at the ends are equivalent.

axis : *int, optional* Interpolation axis. Default is 0.

check_finite : *bool, optional* Whether to check that the input arrays contain only finite numbers. Disabling may give a performance gain, but may result in problems (crashes, non-termination) if the inputs do contain infinities or NaNs. Default is True.

Returns

b : *"BSpline" object* A `BSpline` object of the degree `k` and with knots `t`.

See also: `BSpline`, `CubicSpline`, `make_lsq_spline`, `UnivariateSpline`, `splrep`

Notes

Array API Standard Support

`make_interp_spline` has experimental support for Python Array API Standard compatible backends in addition to NumPy. Please consider testing these features by setting an environment variable `SCIPY_ARRAY_API=1` and providing CuPy, PyTorch, JAX, or Dask arrays as array arguments. The following combinations of backend and device (or other capability) are supported.

(Notes truncated; see the full reference page.)

Examples

Use cubic interpolation on Chebyshev nodes:

```
>>> import matplotlib.pyplot as plt
>>> def cheb_nodes(N):
...     jj = 2.*np.arange(N) + 1
...     x = np.cos(np.pi * jj / 2 / N)[::-1]
...     return x
```

```
>>> x = cheb_nodes(20)
>>> y = np.sqrt(1 - x**2)
```

```
>>> from scipy.interpolate import BSpline, make_interp_spline
>>> b = make_interp_spline(x, y)
>>> np.allclose(b(x), y)
True
```

Note that the default is a cubic spline with a not-a-knot boundary condition ...

scipy.interpolate.CubicSpline

```
scipy.interpolate.CubicSpline(x, y, axis=0, bc_type='not-a-knot', extrapolate=None)
```

Piecewise cubic interpolator to fit values (C2 smooth).

Interpolate data with a piecewise cubic polynomial which is twice continuously differentiable. The result is represented as a `PPoly` instance with breakpoints matching the given data.

Parameters

x : *array_like, shape (n,)* 1-D array containing values of the independent variable. Values must be real, finite and in strictly increasing order.

y : *array_like* Array containing values of the dependent variable. It can have arbitrary number of dimensions, but the length along `axis` (see below) must match the length of `x`. Values must be finite.

axis : *int, optional* Axis along which `y` is assumed to be varying. Meaning that for `x[i]` the corresponding values are `np.take(y, i, axis=axis)`. Default is 0.

bc_type : *str or 2-tuple, optional* Boundary condition type. Two additional equations, given by the boundary conditions, are required to determine all coefficients of polynomials on each segment.

If `bc_type` is a string, then the specified condition will be applied at both ends of a spline. Available conditions are:

- 'not-a-knot' (default): The first and second segment at a curve end are the same polynomial. It is a good default when there is no information on boundary conditions.
- 'periodic': The interpolated functions is assumed to be periodic of period `x[-1] - x[0]`. The first and last value of `y` must be identical: `y[0] == y[-1]`. This boundary condition will result in `y'[0] == y'[-1]` and `y''[0] == y''[-1]`.
- 'clamped': The first derivative at curves ends are zero. Assuming a 1D `y`, `bc_type=((1, 0.0), (1, 0.0))` is the same condition.
- 'natural': The second derivative at curve ends are zero. Assuming a 1D `y`, `bc_type=((2, 0.0), (2, 0.0))` is the same condition.

If `bc_type` is a 2-tuple, the first and the second value will be applied at the curve start and end respectively. The tuple values can be one of the previously mentioned strings (except 'periodic') or a tuple `(order, deriv_values)` allowing to specify arbitrary derivatives at curve ends:

- `order`: the derivative order, 1 or 2.
- `deriv_value`: array_like containing derivative values, shape must be the same as `y`, excluding `axis` dimension. For example, if `y` is 1-D, then `deriv_value` must be a scalar. If `y` is 3-D with the shape (n0, n1, n2) and axis=2, then `deriv_value` must be 2-D and have the shape (n0, n1).

extrapolate : *{bool, 'periodic', None}, optional* If bool, determines whether to extrapolate to out-of-bounds points based on first and last intervals, or to return NaNs. If 'periodic', periodic extrapolation is used. If None (default), `extrapolate` is set to 'periodic' for `bc_type='periodic'` and to True otherwise. See tutorial-interpolate_out_of_bounds.

See also: `Akima1DInterpolator`, `PchipInterpolator`, `PPoly`

Notes

Parameters `bc_type` and `extrapolate` work independently, i.e. the former controls only construction of a spline, and the latter only evaluation.

When a boundary condition is 'not-a-knot' and n = 2, it is replaced by a condition that the first derivative is equal to the linear interpolant slope. When both boundary conditions are 'not-a-knot' and n = 3, the solution is sought as a parabola passing through given points.

When 'not-a-knot' boundary conditions is applied to both ends, the resulting spline will be the same as returned by `splrep` (with `s=0`) and `InterpolatedUnivariateSpline`, but these two methods use a representation in B-spline basis.

(Notes truncated; see the full reference page.)

Examples

In this example the cubic spline is used to interpolate a sampled sinusoid. You can see that the spline continuity property holds for the first and second derivatives and violates only for the third derivative.

```
>>> from scipy.interpolate import CubicSpline
>>> import matplotlib.pyplot as plt
>>> x = np.arange(10)
>>> y = np.sin(x)
>>> cs = CubicSpline(x, y)
>>> xs = np.arange(-0.5, 9.6, 0.1)
```

```
>>> fig, ax = plt.subplots(figsize=(6.5, 4))
>>> ax.plot(x, y, 'o', label='data')
>>> ax.plot(xs, np.sin(xs), label='true')
>>> ax.plot(xs, cs(xs), label="S")
>>> ax.plot(xs, cs(xs, 1), label="S'")
>>> ax.plot(xs, cs(xs, 2), label="S''")
>>> ax.plot(xs, cs(xs, 3), label="S'''")
>>> ax.set_xlim(-0.5, 9.5)
>>> ax.legend(loc='lower left', ncol=2)
>>> plt.show()
...
```

Linear algebra, sparse and spatial

scipy.linalg.solve

```
scipy.linalg.solve(a, b, lower=False, overwrite_a=False, overwrite_b=False, check_finite=True,
↪  assume_a=None, transposed=False)
```

Solve the equation `a @ x = b` for `x`, where `a` is a square matrix.

If the data matrix is known to be a particular type then supplying the corresponding string to `assume_a` key chooses the dedicated solver. The available options are

diagonal	'diagonal'
tridiagonal	'tridiagonal'
banded	'banded'
upper triangular	'upper triangular'
lower triangular	'lower triangular'
symmetric	'symmetric' (or 'sym')
hermitian	'hermitian' (or 'her')
symmetric positive definite	'positive definite' (or 'pos')
general	'general' (or 'gen')

Array argument(s) of this function may have additional ”batch” dimensions prepended to the core shape. In this case, the array is treated as a batch of lower-dimensional slices; see linalg_batch for details.

Parameters

`a` : *array_like, shape (..., N, N)* Square left-hand side matrix or a batch of matrices.

`b` : *(..., N, NRHS) array_like* Input data for the right hand side or a batch of right-hand sides.

`lower` : *bool, default: False* Ignored unless `assume_a` is one of `'sym'`, `'her'`, or `'pos'`. If True, the calculation uses only the data in the lower triangle of `a`; entries above the diagonal are ignored. If False (default), the calculation uses only the data in the upper triangle of `a`; entries below the diagonal are ignored.

`overwrite_a` : *bool, optional* Allow overwriting data in `a` (may enhance performance). Default is False. See tutorial_linalg_overwrite for details.

`overwrite_b` : *bool, optional* Allow overwriting data in `b` (may enhance performance). Default is False. See tutorial_linalg_overwrite for details.

check_finite : *bool, default: True* Whether to check that the input matrices contain only finite numbers. Disabling may give a performance gain, but may result in problems (crashes, non-termination) if the inputs do contain infinities or NaNs.

assume_a : *str, optional* Valid entries are described above. If omitted or `None`, checks are performed to identify structure so the appropriate solver can be called.

transposed : *bool, default: False* If True, solve `a.T @ x == b`. Raises `NotImplementedError` for complex `a`.

Returns

x : *ndarray, shape (N, NRHS) or (..., N)* The solution array.

Raises

ValueError If size mismatches detected or input a is not square.

LinAlgError If the computation fails because of matrix singularity.

LinAlgWarning If an ill-conditioned input a is detected.

NotImplementedError If transposed is True and input a is a complex matrix.

Notes

If the input b matrix is a 1-D array with N elements, when supplied together with an NxN input a, it is assumed as a valid column vector despite the apparent size mismatch. This is compatible with the numpy.dot() behavior and the returned result is still 1-D array.

The general, symmetric, Hermitian and positive definite solutions are obtained via calling ?GETRF/?GETRS, ?SYSV, ?HESV, and ?POTRF/?POTRS routines of LAPACK respectively.

The datatype of the arrays define which solver is called regardless of the values. In other words, even when the complex array entries have precisely zero imaginary parts, the complex solver will be called based on the data type of the array.

Examples

Given `a` and `b`, solve for `x`:

```
>>> a = np.array([[3, 2, 0], [1, -1, 0], [0, 5, 1]])
>>> b = np.array([2, 4, -1])
>>> from scipy.linalg import solve
>>> x = solve(a, b)
>>> x
array([ 2., -2.,  9.])
>>> a @ x == b
array([ True,  True,  True], dtype=bool)
```

Batches of matrices are supported, with and without structure detection: (See linalg_batch for further details of handling batched inputs.)

```
>>> a = np.arange(12).reshape(3, 2, 2)   # a batch of 3 2x2 matrices
>>> A = a.transpose(0, 2, 1) @ a    # A is a batch of 3 positive definite matrices
>>> b = np.ones(2)
>>> solve(A, b)      # this automatically detects that A is pos.def.
array([[ 1. , -0.5],
       [ 3. , -2.5],
       [ 5. , -4.5]])
>>> solve(A, b, assume_a='pos')   # bypass structucture detection
array([[ 1. , -0.5],
       [ 3. , -2.5],
...
```

scipy.linalg.lstsq

```
scipy.linalg.lstsq(a, b, cond=None, overwrite_a=False, overwrite_b=False, check_finite=True,
↪  lapack_driver=None)
```

Compute least-squares solution to the equation `a @ x = b`.

Compute a vector x such that the 2-norm `|b - A x|` is minimized.

Parameters

a : ***(..., M, N) array_like*** Left-hand side array

b : ***(M,) or (..., M, K) array_like*** Right hand side array

cond : ***float, optional*** Cutoff for 'small' singular values; used to determine effective rank of a. Singular values smaller than `cond * largest_singular_value` are considered zero.

overwrite_a : ***bool, optional*** Whether to overwrite data in `a` (may improve performance). Default is False. See tutorial_linalg_overwrite for details.

overwrite_b : ***bool, optional*** Whether to overwrite data in `b` (may improve performance). Default is False. See tutorial_linalg_overwrite for details.

check_finite : ***bool, optional*** Whether to check that the input matrices contain only finite numbers. Disabling may give a performance gain, but may result in problems (crashes, nontermination) if the inputs do contain infinities or NaNs.

lapack_driver : ***str, optional*** Which LAPACK driver is used to solve the least-squares problem. Options are `'gelsd'`, `'gelsy'`, `'gelss'`. Default (`'gelsd'`) is a good choice. However, `'gelsy'` can be slightly faster on many problems. `'gelss'` was used historically. It is generally slow but uses less memory.

New in version 0.17.0.

Returns

x : ***(N,) or (..., N, K) ndarray*** Least-squares solution.

residues : ***(K,) ndarray or float*** If `lapack_driver` is `'gelss'` or `'gelsd'` this contains the square of the 2-norm for each column in `b - a x` if `M > N` and `rank == N`. If the rank condition is violated, `NaN` is returned instead. If `lapack_driver` if `'gelsy'` or `M <= N` a (0,)-shaped array is returned.

rank : ***int*** Effective rank of `a`.

s : ***(min(M, N),) ndarray or None*** Singular values of `a`. The condition number of `a` is `s[0] / s[-1]`.

Raises

LinAlgError If computation does not converge.

ValueError When parameters are not compatible.

See also: `scipy.optimize.nnls`

Notes

When `'gelsy'` is used as a driver, `s` is always `None`.

Array arguments of this function, `a` and `b`, may have additional ”batch” dimensions prepended to the core shape. In this case, the array is treated as a batch of lower-dimensional slices; see linalg_batch for details.

Examples

```
>>> from scipy.linalg import lstsq
>>> import matplotlib.pyplot as plt
```

Suppose we have the following data:

```
>>> x = np.array([1, 2.5, 3.5, 4, 5, 7, 8.5])
>>> y = np.array([0.3, 1.1, 1.5, 2.0, 3.2, 6.6, 8.6])
```

We want to fit a quadratic polynomial of the form `y = a + b*x**2` to this data. We first form the "design matrix" M, with a constant column of 1s and a column containing `x**2`:

```
>>> M = x[:, np.newaxis]**[0, 2]
>>> M
array([[  1.  ,   1.  ],
       [  1.  ,   6.25],
       [  1.  ,  12.25],
       [  1.  ,  16.  ],
       [  1.  ,  25.  ],
       [  1.  ,  49.  ],
       [  1.  ,  72.25]])
...
```

scipy.linalg.svd

```
scipy.linalg.svd(a, full_matrices=True, compute_uv=True, overwrite_a=False, check_finite=True,
↪  lapack_driver='gesdd')
```

Singular Value Decomposition.

Factorizes the matrix `a` into two unitary matrices `U` and `Vh`, and a 1-D array `s` of singular values (real, non-negative) such that `a == U @ S @ Vh`, where `S` is a suitably shaped matrix of zeros with main diagonal `s`.

Parameters

`a` : *(..., M, N) array_like* Matrix to decompose.

`full_matrices` : *bool, optional* If True (default), `U` and `Vh` are of shape `(M, M)`, `(N, N)`. If False, the shapes are `(M, K)` and `(K, N)`, where `K = min(M, N)`.

`compute_uv` : *bool, optional* Whether to compute also `U` and `Vh` in addition to `s`. Default is True.

`overwrite_a` : *bool, optional* Whether to overwrite data in `a` (may improve performance). Default is False. See tutorial_linalg_overwrite for details.

`check_finite` : *bool, optional* Whether to check that the input matrix contains only finite numbers. Disabling may give a performance gain, but may result in problems (crashes, non-termination) if the inputs do contain infinities or NaNs.

`lapack_driver` : *{'gesdd', 'gesvd'}, optional* Whether to use the more efficient divide-and-conquer approach (`'gesdd'`) or general rectangular approach (`'gesvd'`) to compute the SVD. MATLAB and Octave use the `'gesvd'` approach. Default is `'gesdd'`.

Returns

`U` : *ndarray* Unitary matrix having left singular vectors as columns. Of shape `(M, M)` or `(M, K)`, depending on `full_matrices`. Only present when `compute_uv=True`.

`s` : *ndarray* The singular values, sorted in non-increasing order. Of shape (K,), with `K = min(M, N)`.

Vh : *ndarray* Unitary matrix having right singular vectors as rows. Of shape `(N, N)` or `(K, N)` depending on `full_matrices`. Only present when `compute_uv=True`.

Raises

LinAlgError If SVD computation does not converge.

See also: `svdvals`, `diagsvd`

Notes

The array argument of this function, `a`, may have additional "batch" dimensions prepended to the core shape. In this case, the array is treated as a batch of lower-dimensional slices; see linalg_batch for details.

Examples

```
>>> from scipy import linalg
>>> rng = np.random.default_rng()
>>> m, n = 9, 6
>>> a = rng.standard_normal((m, n)) + 1.j*rng.standard_normal((m, n))
>>> U, s, Vh = linalg.svd(a)
>>> U.shape,  s.shape, Vh.shape
((9, 9), (6,), (6, 6))
```

Reconstruct the original matrix from the decomposition:

```
>>> sigma = np.zeros((m, n))
>>> for i in range(min(m, n)):
...     sigma[i, i] = s[i]
>>> a1 = U @ sigma @ Vh
>>> np.allclose(a, a1)
True
...
```

scipy.sparse.csr_array

```
scipy.sparse.csr_array(arg1, shape=None, dtype=None, copy=False, *, maxprint=None)
```

Compressed Sparse Row array.

This can be instantiated in several ways: csr_array(D) where D is a 2-D ndarray

csr_array(S) with another sparse array or matrix S (equivalent to S.tocsr())

csr_array((M, N), [dtype]) to construct an empty array with shape (M, N) dtype is optional, defaulting to dtype='d'.

csr_array((data, (row_ind, col_ind)), [shape=(M, N)]) where `data`, `row_ind` and `col_ind` satisfy the relationship `a[row_ind[k], col_ind[k]] = data[k]`.

csr_array((data, indices, indptr), [shape=(M, N)]) is the standard CSR representation where the column indices for row i are stored in `indices[indptr[i]:indptr[i+1]]` and their corresponding values are stored in `data[indptr[i]:indptr[i+1]]`. If the shape parameter is not supplied, the array dimensions are inferred from the index arrays.

Notes

Sparse arrays can be used in arithmetic operations: they support addition, subtraction, multiplication, division, and matrix power.

Advantages of the CSR format

- efficient arithmetic operations CSR + CSR, CSR * CSR, etc.

- efficient row slicing
- fast matrix vector products

Disadvantages of the CSR format

- slow column slicing operations (consider CSC)
- changes to the sparsity structure are expensive (consider LIL or DOK)

(Notes truncated; see the full reference page.)

Examples

```
>>> from scipy.sparse import csr_array
>>> csr_array((3, 4), dtype=np.int8).toarray()
array([[0, 0, 0, 0],
       [0, 0, 0, 0],
       [0, 0, 0, 0]], dtype=int8)
```

```
>>> row = np.array([0, 0, 1, 2, 2, 2])
>>> col = np.array([0, 2, 2, 0, 1, 2])
>>> data = np.array([1, 2, 3, 4, 5, 6])
>>> csr_array((data, (row, col)), shape=(3, 3)).toarray()
array([[1, 0, 2],
       [0, 0, 3],
       [4, 5, 6]])
```

```
>>> indptr = np.array([0, 2, 3, 6])
>>> indices = np.array([0, 2, 2, 0, 1, 2])
>>> data = np.array([1, 2, 3, 4, 5, 6])
>>> csr_array((data, indices, indptr), shape=(3, 3)).toarray()
array([[1, 0, 2],
       [0, 0, 3],
       [4, 5, 6]])
...
```

scipy.spatial.KDTree

```
scipy.spatial.KDTree(data, leafsize=10, compact_nodes=True, copy_data=False, balanced_tree=True,
↪  boxsize=None)
```

kd-tree for quick nearest-neighbor lookup.

This class provides an index into a set of k-dimensional points which can be used to rapidly look up the nearest neighbors of any point.

Parameters

`data` : *array_like, shape (n,m)* The n data points of dimension m to be indexed. This array is not copied unless this is necessary to produce a contiguous array of doubles. The data are also copied if the kd-tree is built with copy_data=True. Concurrently modifying the data while constructing a KDTree is not well-defined and may lead to crashes or data corruption. If no copy happens, modifying the data *after* creating the KDTree may lead to data corruption or incorrect answers when searching the tree.

`leafsize` : *positive int, optional* The number of points at which the algorithm switches over to brute-force. Default: 10.

`compact_nodes` : *bool, optional* If True, the kd-tree is built to shrink the hyperrectangles to the actual data range. This usually gives a more compact tree that is robust against degenerated input data and gives faster queries at the expense of longer build time. Default: True.

copy_data : *bool, optional* If True the data is always copied to protect the kd-tree against data corruption. Default: False.

balanced_tree : *bool, optional* If True, the median is used to split the hyperrectangles instead of the midpoint. This usually gives a more compact tree and faster queries at the expense of longer build time. Default: True.

boxsize : *array_like or scalar, optional* Apply an m-d toroidal topology to the KDTree. The topology is generated by $x_i + n_i L_i$ where n_i are integers and L_i is the boxsize along i-th dimension. The input data shall be wrapped into $[0, L_i)$. A ValueError is raised if any of the data is outside of this bound.

Notes

The algorithm used is described in. The general idea is that the kd-tree is a binary tree, each of whose nodes represents an axis-aligned hyperrectangle. Each node specifies an axis and splits the set of points based on whether their coordinate along that axis is greater than or less than a particular value.

During construction, the axis and splitting point are chosen by the ”sliding midpoint” rule, which ensures that the cells do not all become long and thin.

(Notes truncated; see the full reference page.)

scipy.spatial.distance.cdist

```
scipy.spatial.distance.cdist(XA, XB, metric='euclidean', *, out=None, **kwargs)
```

Compute distance between each pair of the two collections of inputs.

See Notes for common calling conventions.

Parameters

XA : *array_like* An m_A by n array of m_A original observations in an n-dimensional space. Inputs are converted to float type.

XB : *array_like* An m_B by n array of m_B original observations in an n-dimensional space. Inputs are converted to float type.

metric : *str or callable, optional* The distance metric to use. If a string, the distance function can be 'braycurtis', 'canberra', 'chebyshev', 'cityblock', 'correlation', 'cosine', 'dice', 'euclidean', 'hamming', 'jaccard', 'jensenshannon', 'mahalanobis', 'matching', 'minkowski', 'rogerstanimoto', 'russellrao', 'seuclidean', 'sokalsneath', 'sqeuclidean', 'yule'.

out : *ndarray, optional* Output array in which to place the result. It must have the same shape as the expected output.

****kwargs : *dict, optional*** Extra arguments to `metric`: refer to each metric documentation for a list of all possible arguments.

Some possible arguments:

p : scalar The p-norm to apply for Minkowski, weighted and unweighted. Default: 2.

w : array_like The weight vector for metrics that support weights (e.g., Minkowski).

V : array_like The variance vector for standardized Euclidean. Default: var(vstack([XA, XB]), axis=0, ddof=1)

VI : array_like The inverse of the covariance matrix for Mahalanobis. Default: inv(cov(vstack([XA, XB].T))).T

out : ndarray The output array If not None, the distance matrix Y is stored in this array.

Returns

Y : *ndarray* A m_A by m_B distance matrix is returned. For each i and j, the metric `dist(u=XA[i], v=XB[j])` is computed and stored in the ij th entry.

Raises

ValueError An exception is thrown if `XA` and `XB` do not have the same number of columns.

Notes

The following are common calling conventions:

1. `Y = cdist(XA, XB, 'euclidean')`

   Computes the distance between m points using Euclidean distance (2-norm) as the distance metric between the points. The points are arranged as m n-dimensional row vectors in the matrix X.

2. `Y = cdist(XA, XB, 'minkowski', p=2.)`

(Notes truncated; see the full reference page.)

Examples

Find the Euclidean distances between four 2-D coordinates:

```
>>> from scipy.spatial import distance
>>> coords = [(35.0456, -85.2672),
...           (35.1174, -89.9711),
...           (35.9728, -83.9422),
...           (36.1667, -86.7833)]
>>> distance.cdist(coords, coords, 'euclidean')
array([[ 0.    ,  4.7044,  1.6172,  1.8856],
       [ 4.7044,  0.    ,  6.0893,  3.3561],
       [ 1.6172,  6.0893,  0.    ,  2.8477],
       [ 1.8856,  3.3561,  2.8477,  0.    ]])
```

Find the Manhattan distance from a 3-D point to the corners of the unit cube:

```
>>> a = np.array([[0, 0, 0],
...               [0, 0, 1],
...               [0, 1, 0],
...               [0, 1, 1],
...               [1, 0, 0],
...               [1, 0, 1],
...               [1, 1, 0],
...               [1, 1, 1]])
...
```